
Theory of Static Analysis

- *Temporal Logics & Model Checking*

Farn Wang

Dept. of Electrical Engineering

National Taiwan University

Specifications, descriptions, & verification

- specification (property):
 - The user's requirement
- description (implementation, model):
 - The user's description of the systems
 - No strict line between description and specification.
- verification:
 - Does the description satisfy the specification ?

Formal specification & automated verification

- formal specification:
 - specification with rigorous mathematical notations
- automated verification:
 - verification with support from computer tools.

Why formal specifications ?

- to make the engineers/users understand the system to design through rigorous mathematical notations.
- to avoid ambiguity/confusion/misunderstanding in communication/discussion/reading.
- to specify the system precisely.
- to generate mathematical models for automated analysis.
- *But according to Goedel's incompleteness theorem, it is impossible to come up with a complete specification.*

Why automated verification ?

- to somehow be able to verify complexer & larger systems
- to liberate human from the labor-intensive verification tasks
 - to set free the creativity of human
- to avoid the huge cost of fixing early bugs in late cycles.
- to compete with the core verification technology of the future.

Specification & Verification ?

- Specification → Complete & sound.
- Verification
 - Reducing bugs in a system.
 - Making sure there are very few bugs.

Very difficult!

Competitiveness of high-tech industry!

A way to survive for the students!

A way to survive for Taiwan!



\$4 billion development effort
> 50% system integration & validation cost
2,500,000+1,500,000 lines of codes (most in Ada)

400 horses
100 microprocessors



Bugs in complex software

- They take effects only with special event sequences.
 - the number of event sequences is factorial and super astronomical!
- It is impossible to check all traces with test/simulation.

Three technologies in verification

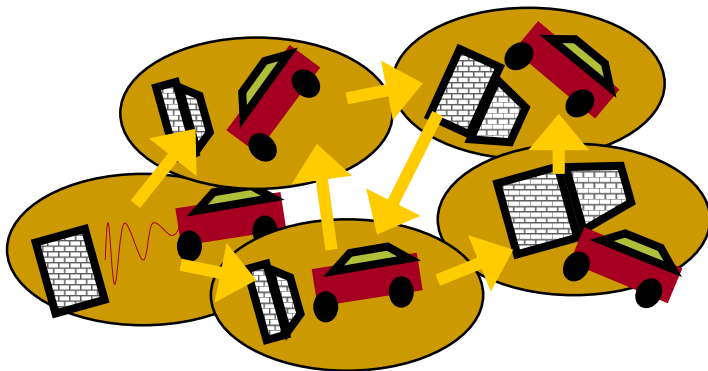


- Testing (real wall for real cars)
 - Expensive
 - Low coverage
 - Late in development cycles



Simulation (virtual wall for virtual cars)

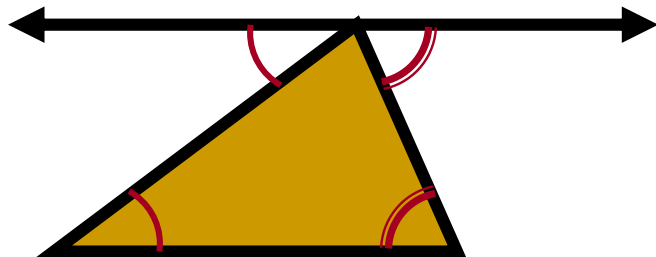
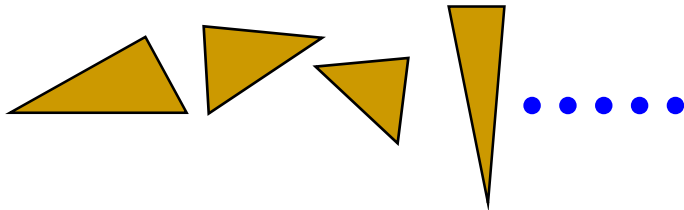
- Economic
- Low coverage
- Don't know what you haven't seen.



- Formal Verification (virtual car checked)

- Expensive
- Functional completeness
 - ◆ 100% coverage
- Automated!
 - ◆ With algorithms and proofs.

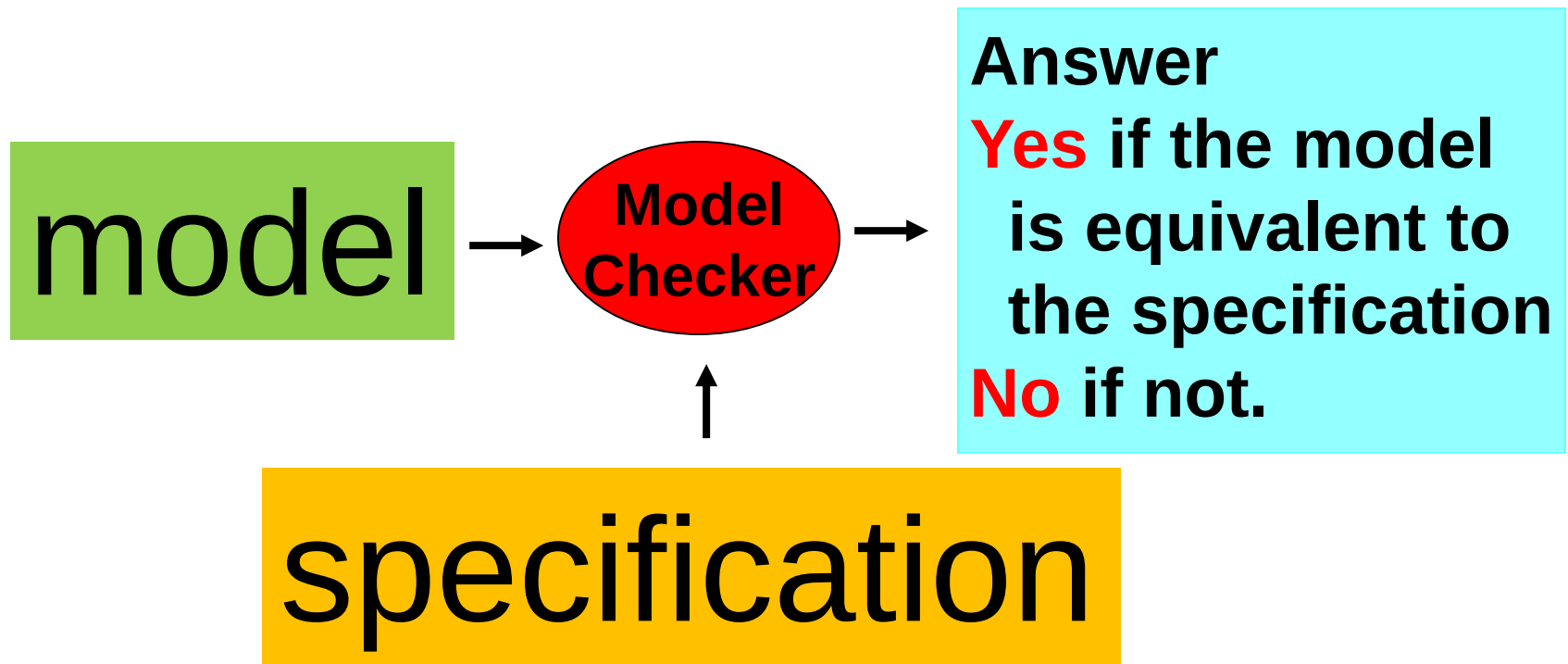
Sum of the 3 angles = 180 ?



- Testing (check all Δ s you see)
 - Expensive
 - Low coverage
 - Late in development cycles
- Simulation (check all Δ s you draw)
 - Economic
 - Low coverage
 - Don't know what you haven't seen.
- Formal Verification (we prove it.)
 - Expensive
 - Functional completeness
 - ◆ 100% coverage
 - Automated!
 - ◆ With algorithms and proofs.

Model-checking

- a general framework for verification of hw/sw systems



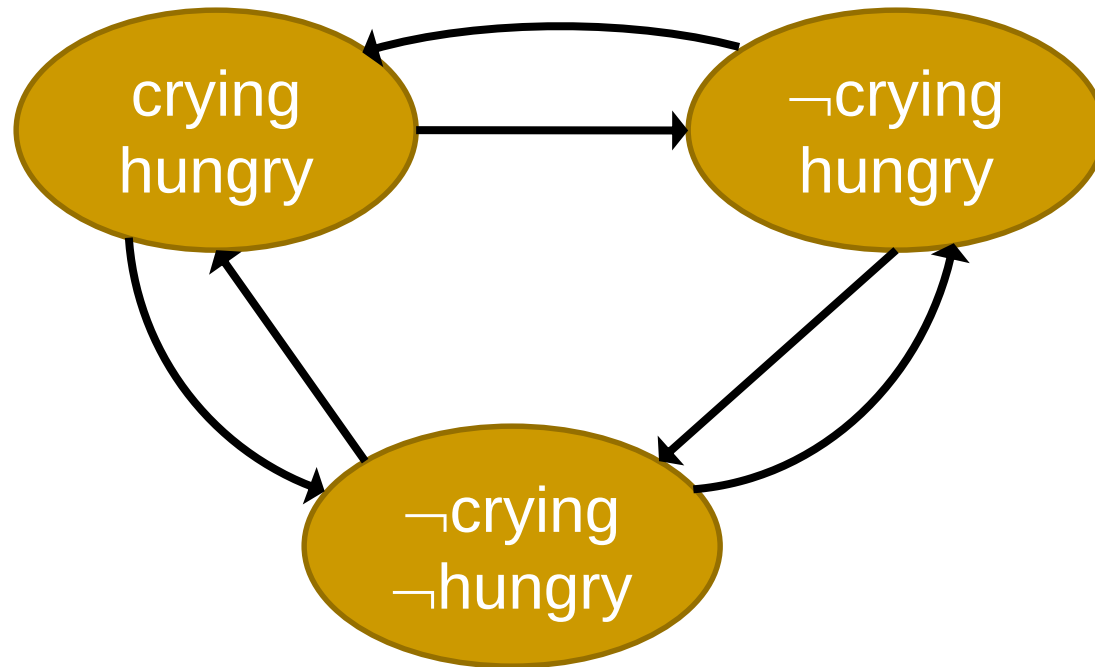
Models & Specifications

- formalism

Whenever a baby cries, it is hungry.

■ Logics: $(\text{crying} \rightarrow \text{hungry})$

■ Graphs:



Models & Specifications

- fairness assumptions

Some properties are almost impossible to verify without assumptions.

Example: (start $\rightarrow$ $\diamond$ finish)

To verify that a program halts, we assume

- CPU does not burn out.
- OS gives the program a *fair* share of CPU time.
- All the drivers do not stuck.
-

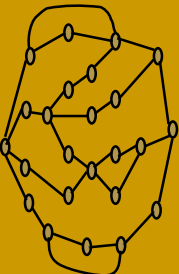
Model-checking

- frameworks in our lecture

F: set of fairness assumptions.

✓: known;

☑: discussed in the lecture

Spec Model							Logics			
			traces		Trees		Linear		Branching	
			F=∅	F≠∅	F=∅	F≠∅	F=∅	F≠∅	F=∅	F≠∅
	traces	F=∅	✓	✓			✓	✓		
		F≠∅	✓	✓			✓	✓		
	Trees	F=∅			☑	✓			☑	✓
		F≠∅			✓	✓			✓	✓
Logics	Linear	F=∅					☑	☑		
		F≠∅					☑	☑		
	Branching	F=∅							✓	✓
		F≠∅							✓	✓

Kripke Structure

- A state-transition system that captures
 - What is true of a state
 - What can be viewed as an **atomic move**
 - The succession of states
- Static representation that can be unrolled to a tree of execution traces, on which temporal properties are verified

Kripke str

I'm honored by your proposal, by my mum says I have to finish high-school first.

Saul Kripke

Born

Pr

Di

- wrote his first essay on Kripke structure at 16
- invited to teach at Princeton
- taught a graduate logic course at MIT since sophomore year at Harvard.



y of languages
tgenstein

Kripke structure

- syntax

$$A = (S, S_0, R, L)$$

■ S

- a set of all states of the system

■ $S_0 \subseteq S$

- a set of initial states

■ $R \subseteq S \times S$

- a transition relation

■ $L : S \rightarrow 2^P$

- a function that associates each state with set of propositions true in that state

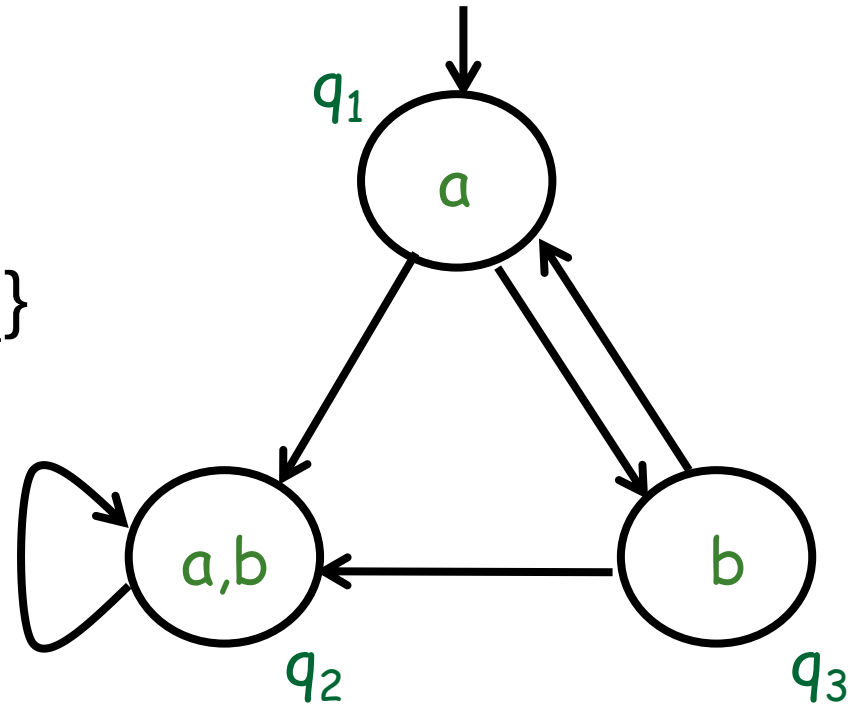
To extend to integer programs,
 $L : S \times P \rightarrow \{0, 1\}$

- L allows us to describe the truth/falsehood of a proposition in the various states of a system.
- The propositions refer to valuations of the state variables.

Kripke Model

- syntax

- Set of states $S=\{q_1, q_2, q_3\}$
- Set of initial states $S_0=\{q_1\}$
- $R = \{(q_1, q_2), (q_2, q_2), (q_1, q_3), (q_3, q_1), (q_3, q_2)\}$
- Set of atomic propositions $AP=\{a, b\}$
- $L(q_1)=\{a\}, L(q_2)=\{a, b\}, L(q_3)=\{b\}$



Kripke structure

- semantics

Given a Kripke structure $A = (S, S_0, R, L)$,
a **run** is a finite or infinite sequence

$$s_0 s_1 s_2 \dots s_k \dots$$

such that

- $s_0 \in S_0$
- for each $k \in \mathbb{N}$, if s_{k+1} exists,
 - $s_{k+1} \in S$ and
 - $R(s_k, s_{k+1})$ is true.

Control and data variables

- State = valuation of control and data vars.
 - In our example
 - *pc0, pc1* are control variables.
 - *turn* is a shared data variable.
- state examples: (pc0=1,pc1=2,turn=0),
(pc0=2,pc1=2,turn=1),
- To generate a finite state transition system
 - Data variables must have finite types, and
 - Finitely many control locations

Program → Kripke structure

- Data variables

Data variables often do not have finite types

- integer, ...
- Usually **abstracted** into a finite type.
- An integer variable can be abstracted to $\{-, 0, +\}$
- Just store the information about the sign of the variable. (coming up with these abstractions is a whole new problem).

Program → Kripke structure

- Control Locations

Isn't the control locations of a program always finite ?

- NO, because your program may be a concurrent program with unboundedly many processes or threads (**parameterized system**).
- Can employ control abstractions (such as symmetry reduction)

Program → Kripke structure

- States and Transitions

- Each component makes a move at every step.
- Digital circuits are most often *synchronous*.
 - Common clock driving the system.
 - Contents of flip-flops define the states.
 - On every clock pulse, the content of every flip-flop (potentially) changes.
- This change is captured by the transition relation.

Program $\rightarrow$ Kripke structure

- States and Transitions

- Define $V=\{v_1,\dots,v_n\}$, boolean variables representing state of flip-flops in the circuit.
- Set of states represented by boolean formula over $v_1,\dots,v_n$.
- To define transitions, define a fresh set of variables $V'=\{v'_1,\dots,v'_n\}$. These are the next state variables.
- The transitions are now represented by a relation $R(V,V')\subseteq V\times V'$

Kripke structure

- Transition Relation

- $(s, s') \in R(V, V')$ implies $s \rightarrow s'$
- Now, $R(V, V') = \bigcup_{i \in \{1, \dots, n\}} R_i(V, V')$, where captures the changes in state variable v_i
- Define $R_i(V, V') = (v'_i \Leftrightarrow f_i(V))$ where $f_i(V)$ is a boolean function defining the value of flip-flop i in next state.
- Given a synchronous circuit, we then need to define $f_i(V)$ for each i .

Transition relation

- A synchronous mod 8 counter

- $V = \{v_2, v_1, v_0\}$, where v_0 is the least significant bit.

- The transitions can be enumerated as:

$000 \rightarrow 001 \rightarrow 010 \rightarrow \dots$

- Alternatively define how each of the three bits are changed on every clock cycle

- $v'_0 = \neg v_0$ (the least significant bit)

- $v'_1 = v_0 \oplus v_1$

- $v'_2 = (v_0 \wedge v_1) \oplus v_2$ (the most significant bit)

Kripke Structure

- example

Suppose there is a program

```
initially x==1 && y==1;  
while (true)  
  x = (x+y) % 2;
```

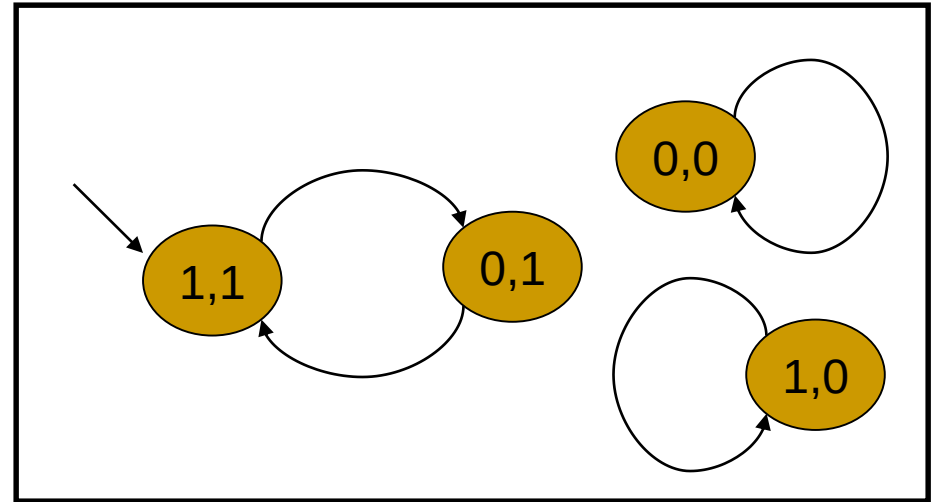
where x and y range over $D=\{0,1\}$

Kripke Structure

- example

Suppose there is a program

```
initially x==1 && y==1;  
while (true)  
  x = (x+y) % 2;
```



where x and y range over $D=\{0,1\}$

Kripke Structure

- example

Suppose there is a program

```
initially x==1 && y==1;  
while (true)  
  x = (x+y) % 2;
```

$S = D \times D = \{(0,0), (0,1), (1,0), (1,1)\}$

$S_0 = \{(1,1)\}$

$R = \{((1,1), (0,1)), ((0,1), (1,1)),$
 $((1,0), (1,0)), ((0,0), (0,0))\}$

$L((1,1)) = \{x=1, y=1\},$

$L((0,1)) = \{x=0, y=1\},$

$L((1,0)) = \{x=1, y=0\},$

$L((0,0)) = \{x=0, y=0\}$

where x and y range over $D = \{0,1\}$

Kripke Structure

- example

Suppose there is a program

```
initially x==1 && y==1;  
while (true)  
    x = (x+y) % 2;
```

$S = D \times D = \{a, b, c, d\}$

$S_0 = \{a\}$

$R = \{(a, b), (b, a),$
 $(c, c), (d, d)\}$

$L(a) = \{x=1, y=1\},$

$L(b) = \{x=0, y=1\},$

$L(c) = \{x=1, y=0\},$

$L(d) = \{x=0, y=0\}$

where x and y range over $D = \{0, 1\}$

Workout

- Kripke Structure

Suppose there is a program

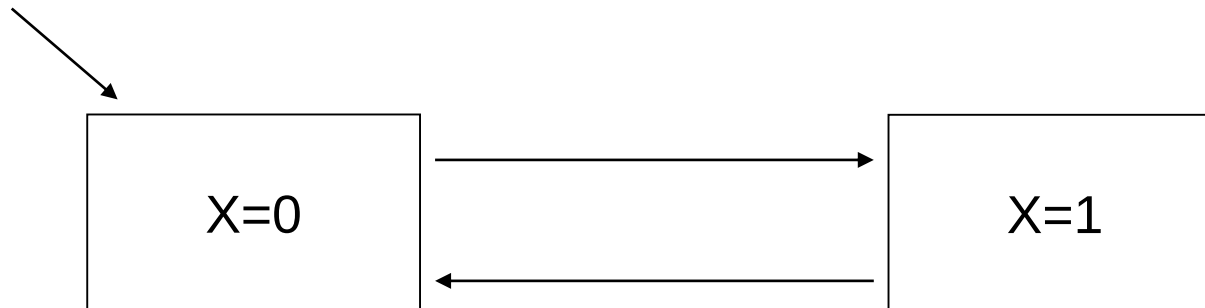
```
initially x==1 && y==1;  
while (true)  
  x = (x+y) % 3;
```

where x and y range over $D=[0,2]$

Kripke Structure

- an example

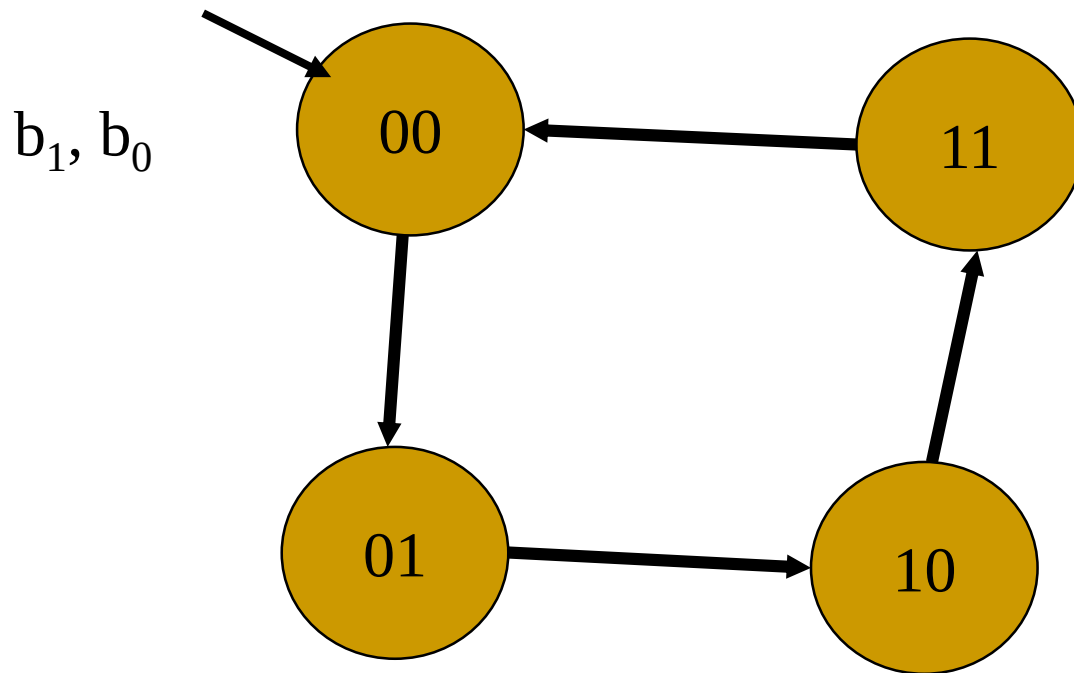
Initially $x=0$
While (true)
 $x:=1-x$;



Kripke Structure

- example

A 2-bit counter operates at bit-level.



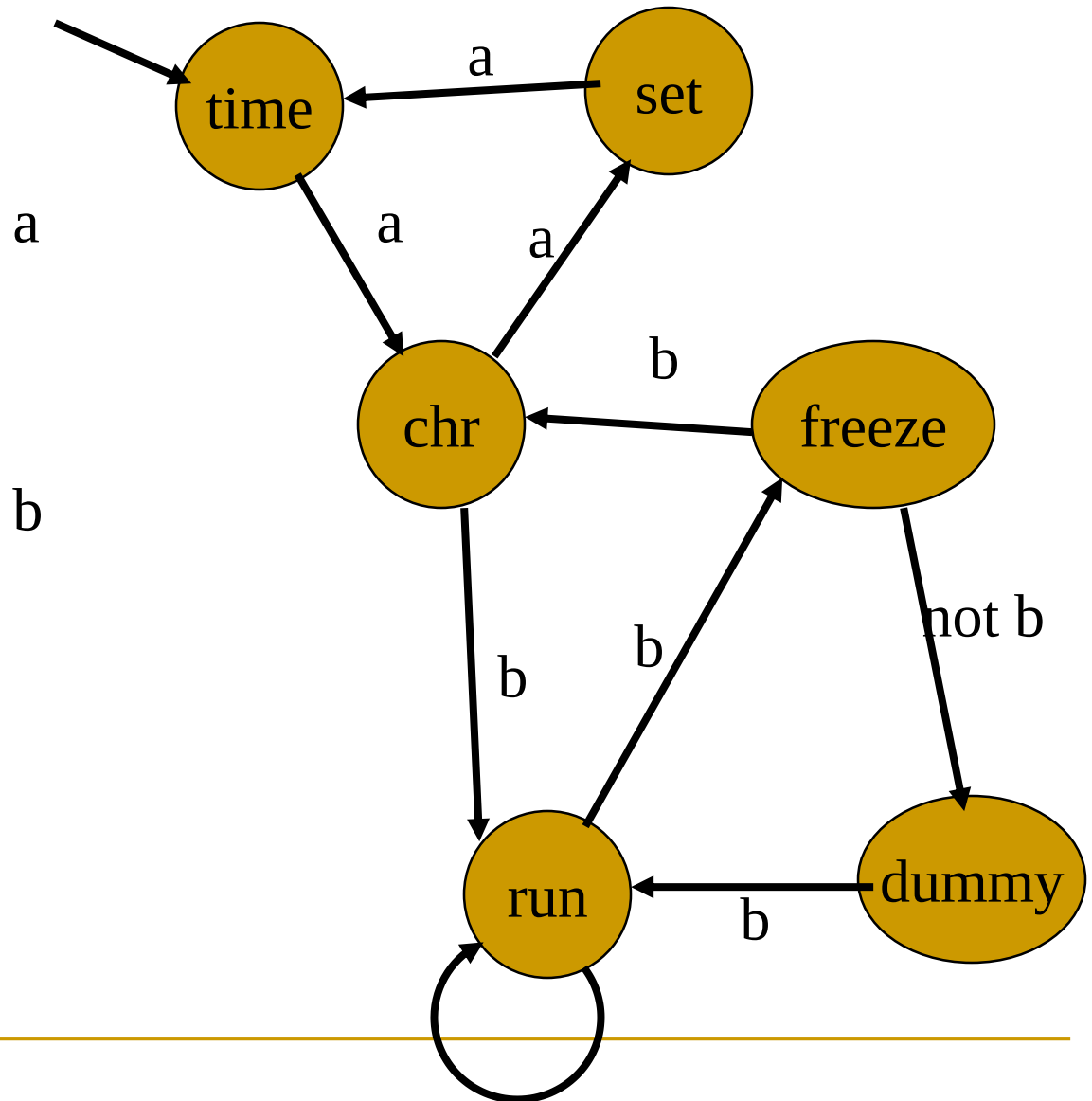
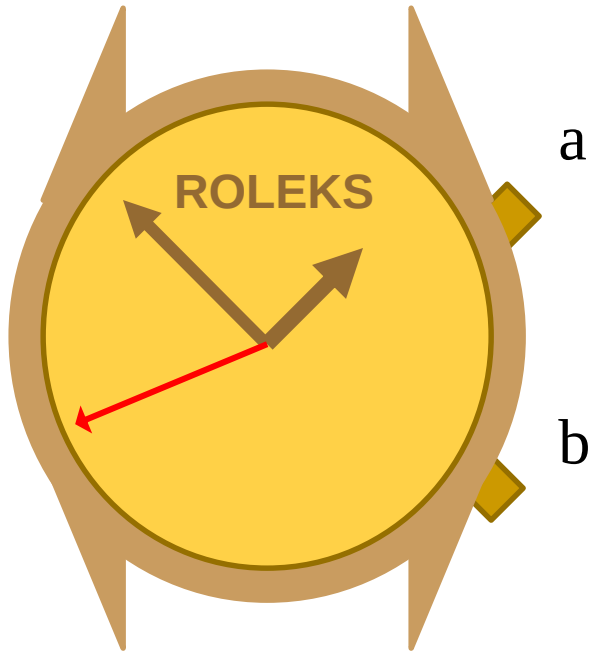
Kripke Structure

- workout

Write a simple program for the Kripke structures in the last page.



Automata & Kripke structure



Concurrent programs

- A set programs running independently, communicating from time to time, thereby performing a common task.
- *Flavors of Concurrency*
 - Synchronous execution
 - Asynchronous / interleaved execution
 - Communication via shared variables
 - Message passing communication

Kripke Structure

- for a concurrent system

- Programs (as opposed to circuits) are typically considered asynchronous.
- An asynchronous concurrent system is a collection of sequential programs $P_1 \dots P_k$ running in parallel with only one pgm. making a move at every time step.
 - How do the sequential programs communicate ?
 - What are the behaviors of the concurrent system ?

Kripke Structure

- for a concurrent system

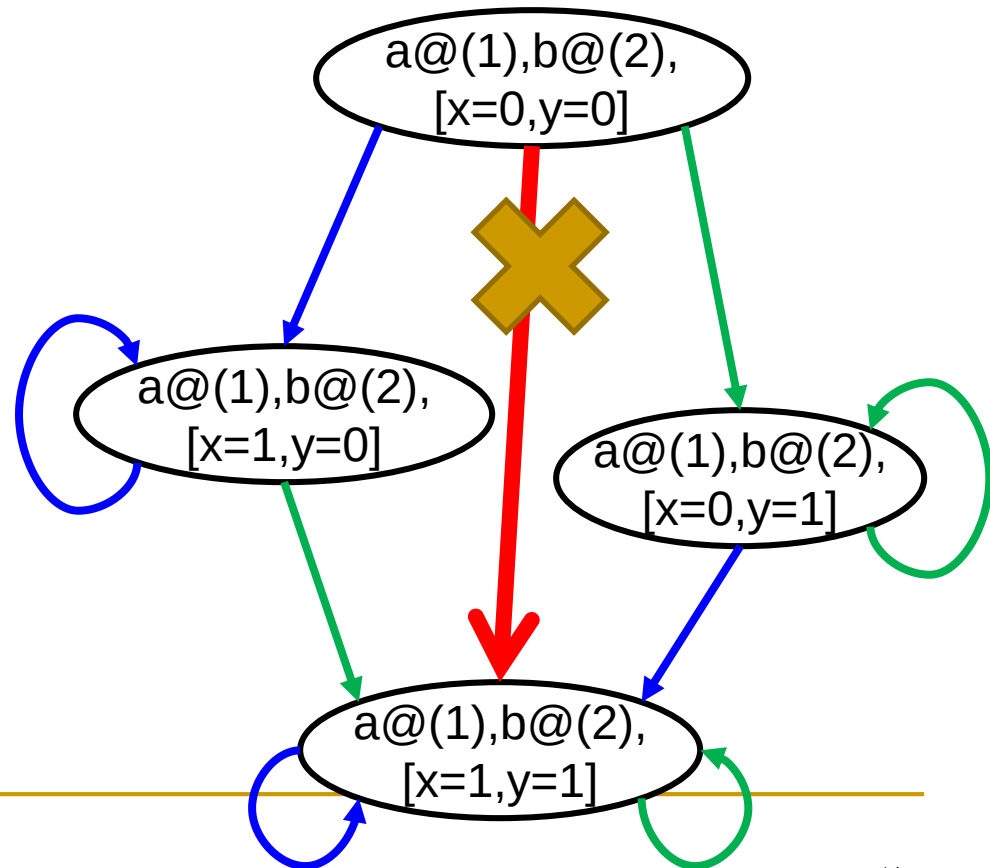
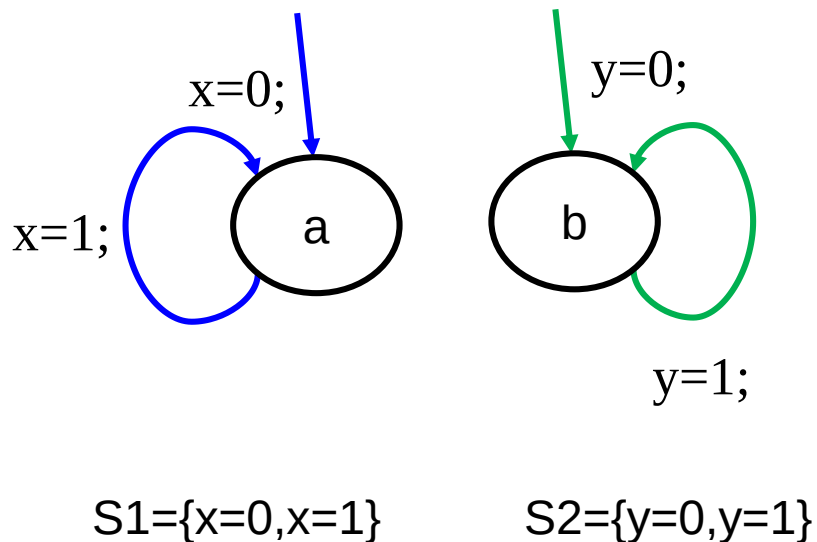
- Behaviors of each sequential program P_i captured by its operational semantic.
- The programs P_i need not be terminating.
- Behaviors (Traces) of $P_1 \dots P_k$ formed by **interleaving** the transitions of the programs.
- Consider two non-communicating programs.

Concurrent systems

- Interleaving semantics

Semantics as Kripke structure

state-transition graphs



Kripke Structures

- composition for a concurrent system

Given $A_i = \langle S_i, S_{i,0}, R_i, L_i \rangle$, $1 \leq i \leq n$

Cartesian Product of $A_1, A_2, \dots, A_n$,

$$A = \langle S, S_0, R, L \rangle$$

$$S : S_1 \times S_2 \times \dots \times S_n$$

$$S_0 : S_{1,0} \times S_{2,0} \times \dots \times S_{n,0}$$

$$R([s_1, \dots, s_{j-1}, s_j, s_{j+1}, \dots, s_n], [s_1, \dots, s_{j-1}, s'_j, s_{j+1}, \dots, s_n])$$

$$\square (s_j, s'_j) \in R_j$$

$\square$ *According to the interleaving semantics, one process transition at a moment*

$$L([s_1, s_2, \dots, s_n]) = L_1(s_1) \cup L_2(s_2) \cup \dots \cup L_n(s_n)$$

Kripke Structures

- Cartesian product method

1. Construct all the vectors of component process states
 2. Eliminate all those inconsistent vectors according to invariance condition
 3. Draw arcs from vectors to vectors according to process transitions
- *Very often creates many unreachable states*

Kripke structure

- Practical algorithm for construction

Given $A = \langle S, S_0, R, L \rangle$

- Usually only S_0, R, L are given.
- We may want to construct S .
- Usually S is too big to construct.

Kripke Structures

- on-the-fly method

1. Starting from the initial states (or goal states in backward analysis)
 2. Step by step, add states that is reachable from those already reached, until no more new reachable states are generated.
- *Tedious but may result in much smaller reachable state-space representation.*

History of Temporal Logic

- Designed by philosophers to study the way that time is used in natural language arguments
- Reviewed by Prior [PR57, PR67]
- Brought to Computer Science by Pnueli [PN77]
- Has proved to be useful for specification of concurrent systems

Framework

- Temporal Logic is a class of **Modal Logic**
- Allows qualitatively describing and reasoning about changes of the truth values over time
- Usually implicit time representation
- Provides variety of **temporal operators** (*sometimes, always*)
- Different views of time (branching vs. linear, discrete vs. continuous, past vs. future, etc.)

Outline

- Linear
 - LPTL (Linear time Propositional Temporal Logics)
- Branching
 - CTL (Computation Tree Logics)
 - CTL* (the full branching temporal logics)

BNF, syntax definitions

Note!

Be sure how to read BNF !

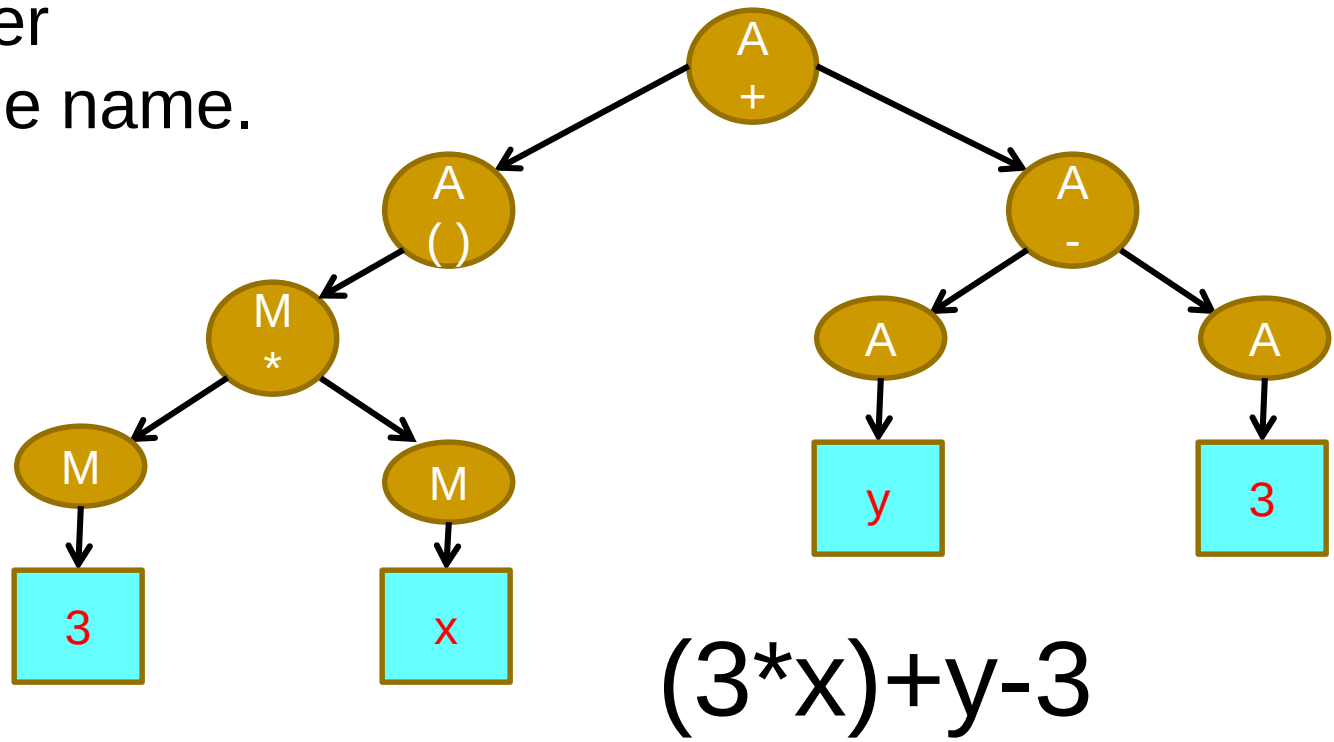
- used for define syntax of context-free language
- important for the definition of
 - automata predicates and
 - temporal logics
- Used throughout the lectures!
- In exam: violate the syntax rules → **no credit.**

$$A ::= c \mid x \mid (M) \mid A_1 + A_2 \mid A_1 - A_2$$
$$M ::= c \mid x \mid (A) \mid M_1 * M_2 \mid M_1 / M_2$$

c is an integer
x is a variable name.

BNF, syntax definitions

$A ::= c \mid x \mid (M) \mid A_1 + A_2 \mid A_1 - A_2$
 $M ::= c \mid x \mid (A) \mid M_1 * M_2 \mid M_1 / M_2$
 c is an integer
 x is a variable name.



BNF, syntax definitions

- derivation trees (from top down)

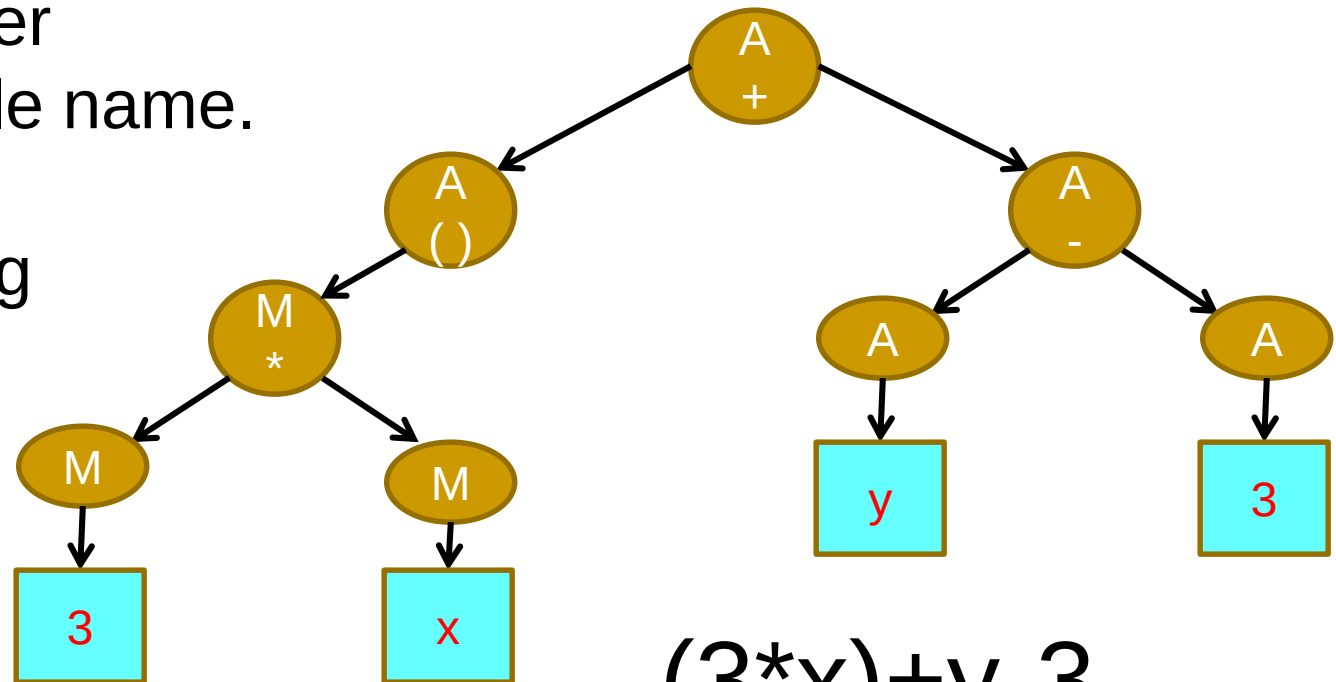
$A ::= c \mid x \mid (M) \mid A_1 + A_2 \mid A_1 - A_2$

$M ::= c \mid x \mid (A) \mid M_1 * M_2 \mid M_1 / M_2$

c is an integer

x is a variable name.

used in string
generation.



$(3 * x) + y - 3$

BNF, syntax definitions

- parsing trees (from bottom up)

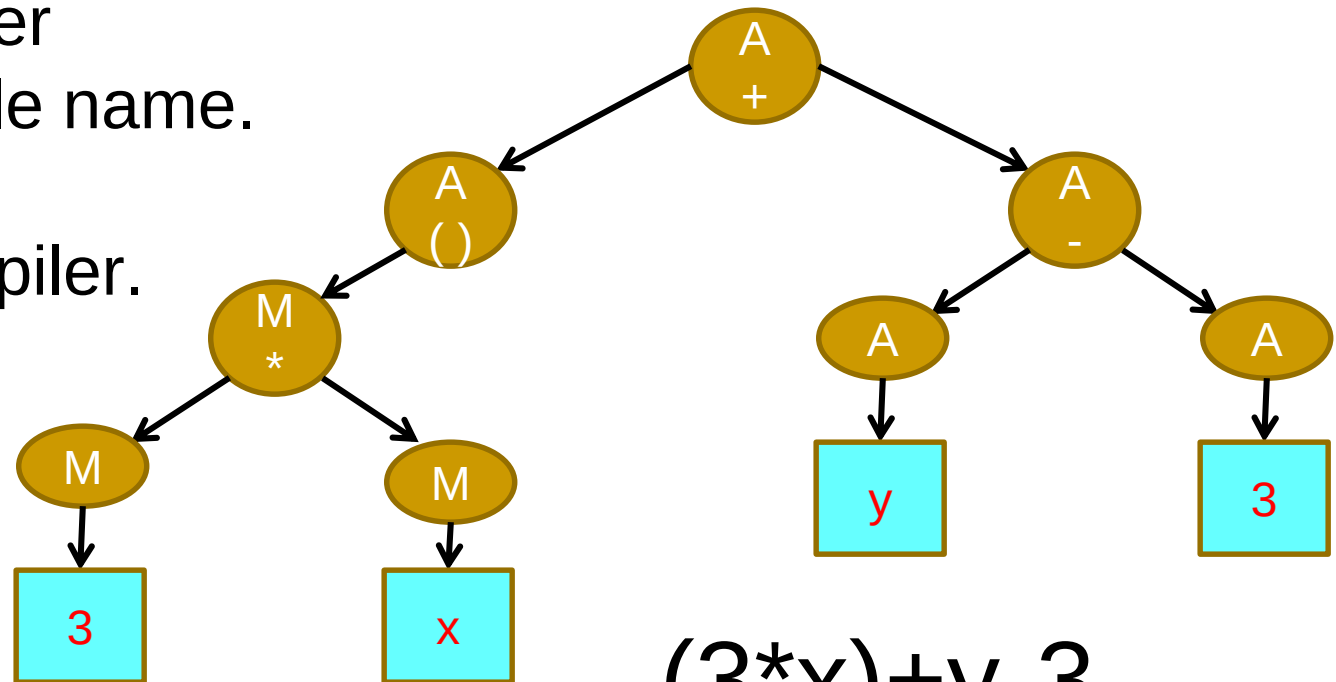
$A ::= c \mid x \mid (M) \mid A_1 + A_2 \mid A_1 - A_2$

$M ::= c \mid x \mid (A) \mid M_1 * M_2 \mid M_1 / M_2$

c is an integer

x is a variable name.

used in compiler.



$(3 * x) + y - 3$

Temporal Logics : Catalog

propositional	$\leftrightarrow$	first-order
global	$\leftrightarrow$	compositional
branching	$\leftrightarrow$	linear-time
points	$\leftrightarrow$	intervals
discrete	$\leftrightarrow$	continuous
past	$\leftrightarrow$	future

Temporal Logics

■ Linear

- LPTL (Linear time Propositional Temporal Logics)
 - LTL, PTL, PLTL

■ Branching

- CTL (Computation Tree Logics)
- CTL* (the full branching temporal logics)

Amir Pnueli

1941

- Professor, Weizmann Institute
- Professor, NYU
- Turing Award, 1996

Presentation of a gift at
ATVA /FORTE 2005,
Taipei



LPTL (PTL, LTL)

Linear-Time Propositional Temporal Logic

Conventional notation :

- propositions : $\mathbf{p, q, r, \dots}$
- sets : $\mathbf{A, B, C, D, \dots}$
- states : $\mathbf{s}$
- state sequences : $\mathbf{S}$
- formulas : $\mathbf{\varphi, \psi}$
- Set of natural number : $\mathbf{N = \{0, 1, 2, 3, \dots\}}$
- Set of real number : $\mathbf{R}$

LPTL

Given **P** : a set of propositions,
a Linear-time structure : *state sequence*

$$\mathbf{S} = \mathbf{s}_0 \mathbf{s}_1 \mathbf{s}_2 \mathbf{s}_3 \mathbf{s}_4 \dots \mathbf{s}_k \dots$$

$\mathbf{s}_k$ is a function of P where **$P \mapsto \{true, false\}$**

or $\mathbf{s}_k \in 2^P$

example: $P = \{a, b\}$

$\{a\}\{a, b\}\{a\}\{a\}\{b\}\dots$

Syntax definitions

Note!

Be sure how to read BNF !

- used for define syntax of context-free language
- important for the definition of
 - automata predicates and
 - temporal logics
- Used throughout the lectures!
- In exam: violate the syntax rules → **no credit.**

$$A ::= (M) \mid A1 + A2 \mid A1 - A2$$
$$M ::= (A) \mid M1 * M2 \mid M1 / M2$$

LPTL

- syntax

syntax definition
in BNF

$\psi ::= \text{true} \mid p \mid \neg\psi \mid \psi_1 \vee \psi_2 \mid O\psi \mid \psi_1 \cup \psi_2$

abbreviation

$\text{false} \equiv \neg \text{true}$

$\psi_1 \wedge \psi_2 \equiv \neg ((\neg\psi_1) \vee (\neg\psi_2))$

$\psi_1 \rightarrow \psi_2 \equiv (\neg\psi_1) \vee \psi_2$

$\Diamond\psi \equiv \text{true} \cup \psi$

$\Box\psi \equiv \neg \Diamond \neg \psi$

LPTL

- syntax

Exam.	Symbol in CMU
-------	------------------

Op	Xp	<i>p</i> is true on next state
$p \cup q$	$p \cup q$	From now on, <i>p</i> is always true until <i>q</i> is true
$\Diamond p$	Fp	From now on, there will be a state where <i>p</i> is eventually (sometimes) true
$\Box p$	Gp	From now on, <i>p</i> is always true

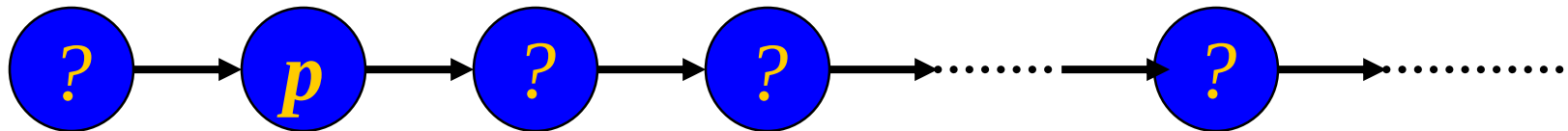
LPTL

- syntax

Op

Xp

p is true on **next** state



? : don't care

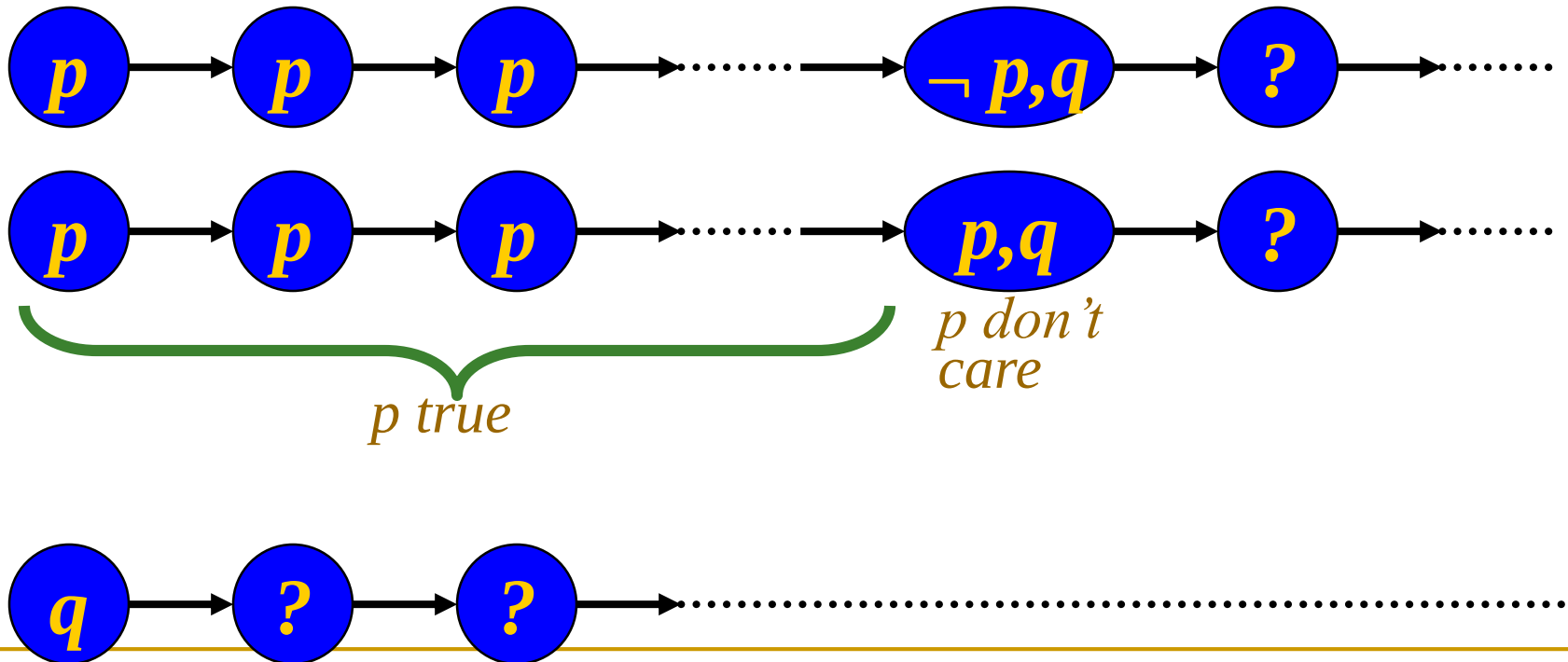
LPTL

- syntax

$p \cup q$

$p \cup q$

From now on, p is always true **until** q is true



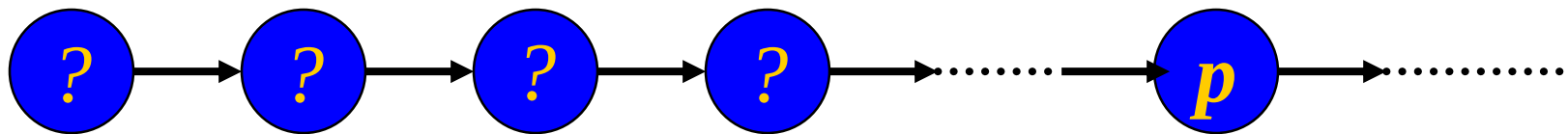
LPTL

- syntax

$\Diamond p$

Fp

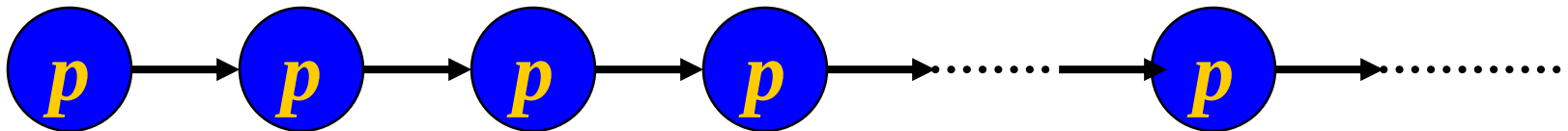
From now on, there will be a state where p is **eventually (sometimes)** true



$\Box p$

Gp

From now on, p is **always** true



LPTL

- syntax

Two operators for Fairness

- $\Diamond^\infty p \equiv \Box \Diamond p$; **p will happen infinitely many times**
infinitely often
- $\Box^\infty p \equiv \Diamond \Box p$; **p will be always true after some time in the future**
almost everywhere

2011/06/30 stopped here.

LPTL

- semantics

suffix path :

$$S = s_0 s_1 s_2 s_3 s_4 s_5 \dots\dots\dots$$

$$S^{(0)} = s_0 s_1 s_2 s_3 s_4 s_5 \dots\dots\dots$$

$$S^{(1)} = s_1 s_2 s_3 s_4 s_5 s_6 \dots\dots\dots$$

$$S^{(2)} = s_2 s_3 s_4 s_5 s_6 \dots\dots\dots$$

$$S^{(3)} = s_3 s_4 s_5 s_6 \dots\dots\dots$$

$$S^{(k)} = s_k s_{k+1} s_{k+2} s_{k+3} \dots\dots\dots$$

LPTL

- semantics

Given a state sequence

$$S = s_0 s_1 s_2 s_3 s_4 \dots s_k \dots$$

We define $S \models \psi$ (S satisfies ψ) inductively as :

- $S \models \text{true}$
- $S \models p \Leftrightarrow s_0(p) = \text{true}$, or equivalently $p \in s_0$
- $S \models \neg\psi \Leftrightarrow S \not\models \psi$ is false
- $S \models \psi_1 \vee \psi_2 \Leftrightarrow S \models \psi_1$ or $S \models \psi_2$
- $S \models O\psi \Leftrightarrow S^{(1)} \models \psi$
- $S \models \psi_1 U \psi_2 \Leftrightarrow \exists k \geq 0 (S^{(k)} \models \psi_2 \wedge \forall 0 \leq j < k (S^{(j)} \models \psi_1))$

LTL

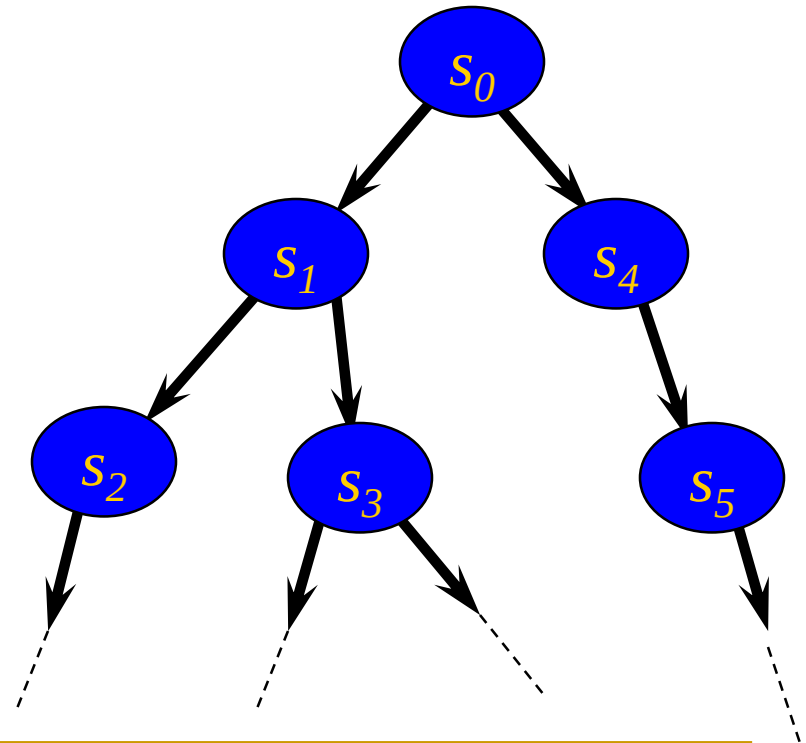
- examples

- $(\text{start} \rightarrow \Diamond \text{finish})$ 人要永遠有始有終
- $\Diamond \text{comet-hit-earth}$
- $(\text{earth}) \rightarrow \Diamond \text{comet-hit-earth}$
- $(\Diamond \text{buy-lottery-ticket}) \rightarrow \Diamond \text{win-lottery}$
- $(\text{power-on} \rightarrow \Diamond \text{boot-success})$
- $(\text{power-on}) \rightarrow \Diamond \text{boot-success}$
- $((\text{power-on}) \rightarrow \Diamond \text{boot-success})$
- $(((\text{power-on}) \&\& \Diamond \text{boot-CPU})$
 $\rightarrow \Diamond \text{boot-success})$

Branching Temporal Logics

Basic assumption of tree-like structure

- Every **node** is a function of $P \rightarrow \{\text{true}, \text{false}\}$
- Every state may have many **successors**



Branching Temporal Logics

Basic assumption of tree-like structure

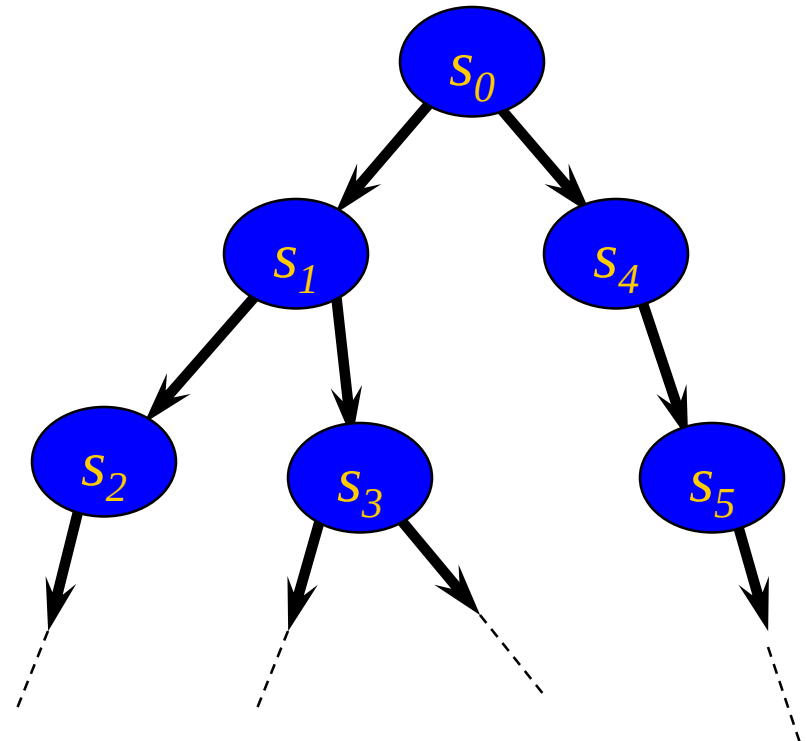
- Every **path** is isomorphic as N
 - Correspond to a **state sequence**

Path : $s_0 s_1 s_3 \dots$

$s_0 s_1 s_2 \dots$

$s_1 s_3 \dots$

$s_4 s_5 \dots$



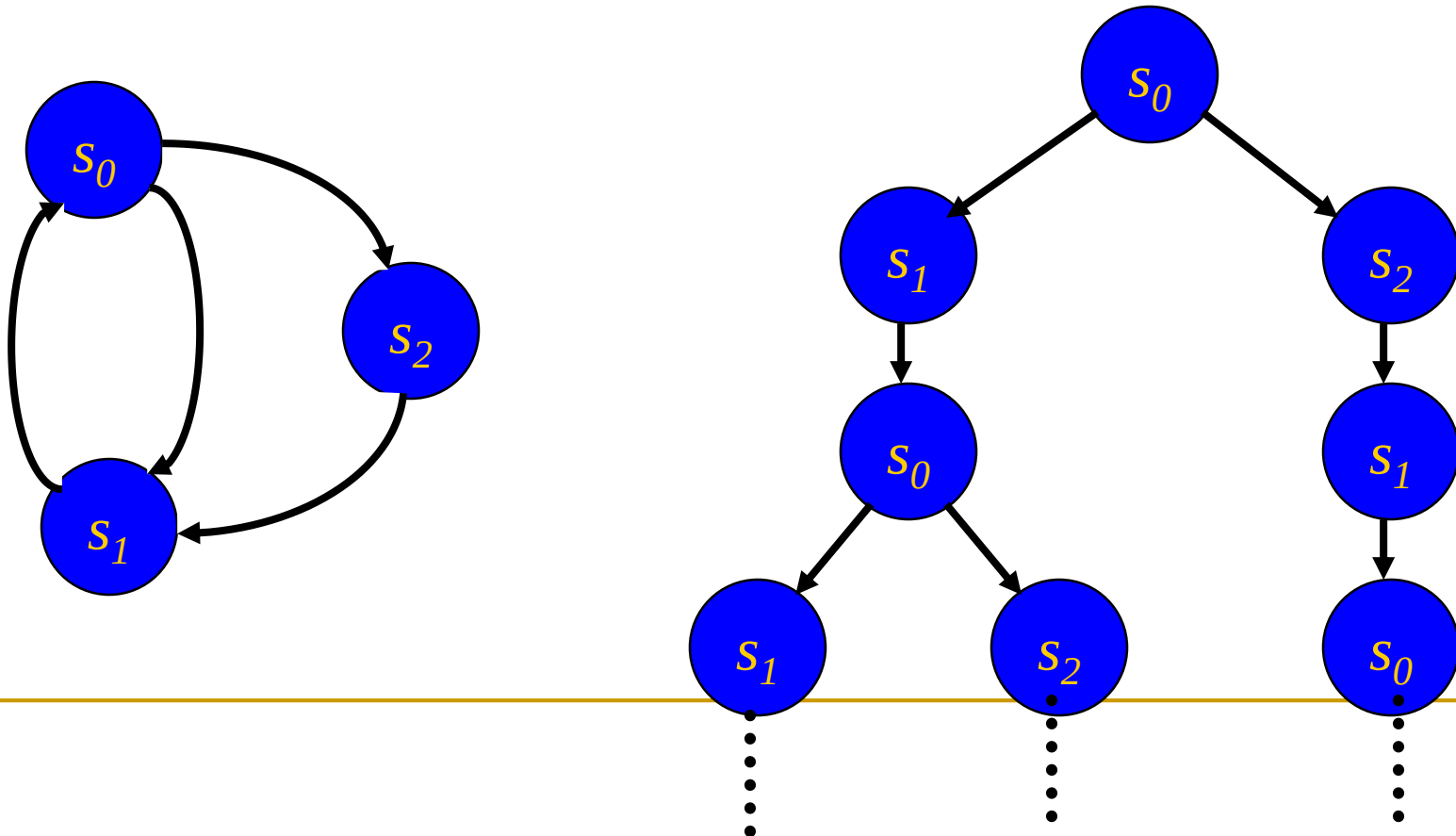
Branching Temporal Logic

It can accommodate infinite and dense state successors

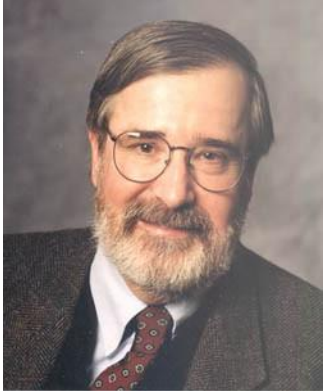
- In CTL and CTL*, it can't tell
 - Finite and infinite
 - Is there infinite transitions ?
 - Dense and discrete
 - Is there countable (ω) transitions ?

Branching Temporal Logic

Get by flattening a finite state machine

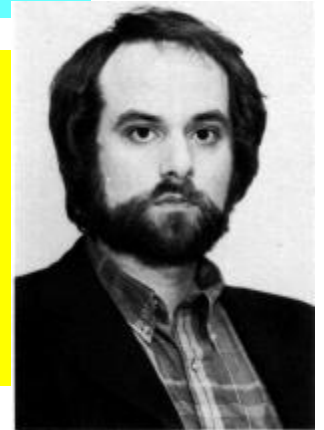


CTL(Computation Tree Logic)



Edmund M. Clarke
Professor, CS & ECE
Carnegie Mellon University

E. Allen Emerson
Professor, CS
The University of Texas at Austin



Chin-Laung Lei
Professor, EE
National Taiwan University

CTL(Computation Tree Logic)

- syntax

$\varphi ::= \text{true} \mid p \mid \neg\varphi \mid \varphi_1 \vee \varphi_2 \mid \exists O\varphi \mid \forall O\varphi$
 $\mid \exists\varphi_1 U\varphi_2 \mid \forall\varphi_1 U\varphi_2$

abbreviation :

false	$\equiv$	$\neg \text{true}$
$\varphi_1 \wedge \varphi_2$	$\equiv$	$\neg ((\neg\varphi_1) \vee (\neg\varphi_2))$
$\varphi_1 \rightarrow \varphi_2$	$\equiv$	$(\neg\varphi_1) \vee \varphi_2$
$\exists \Diamond \varphi$	$\equiv$	$\exists \text{true} U \varphi$
$\forall \Box \varphi$	$\equiv$	$\neg \exists \Diamond \neg \varphi$
$\forall \Diamond \varphi$	$\equiv$	$\forall \text{true} U \varphi$
$\exists \Box \varphi$	$\equiv$	$\neg \forall \Diamond \neg \varphi$

CTL

- semantics

example	symbol in CMU
---------	------------------

$\exists Op$	EXp	there exists a path where p is true on next state
$\exists pUq$	$pEUq$	from now on, there is a path where p is always true until q is true
$\forall Op$	AXp	for all path where p is true on next state
$\forall pUq$	$pAUq$	from now on, for all path where p is always true until q is true

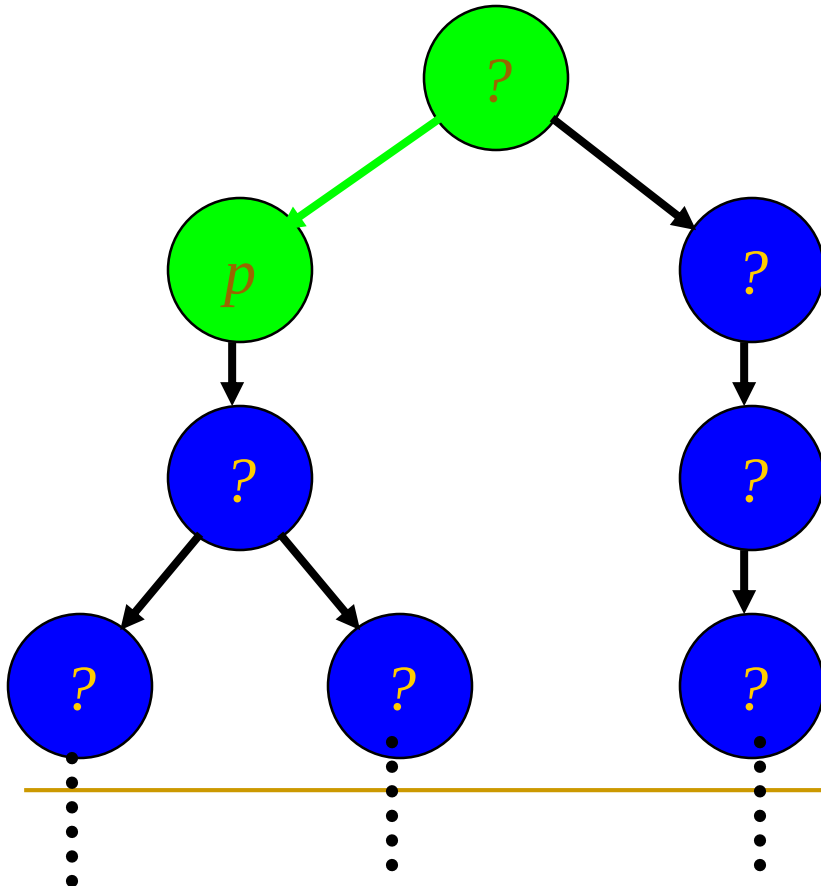
CTL

- semantics

$\exists Op$

EXp

there exists a path where p is true on next state



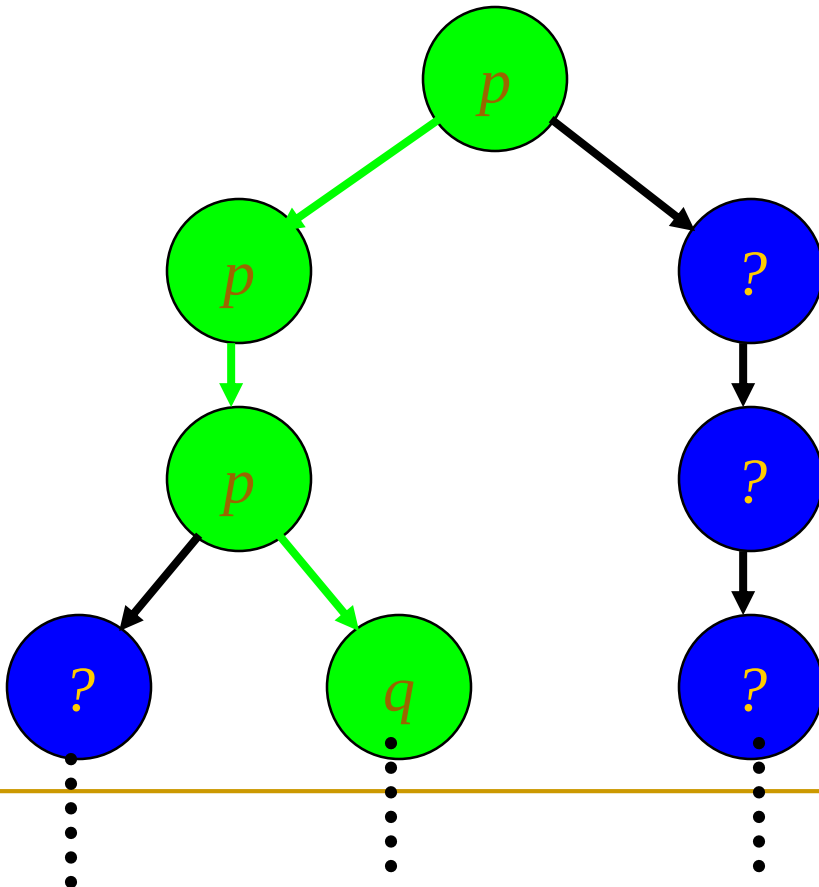
CTL

- semantics

$\exists p \cup q$

$p \text{EU} q$

from now on, there is a path where p is always true until q is true



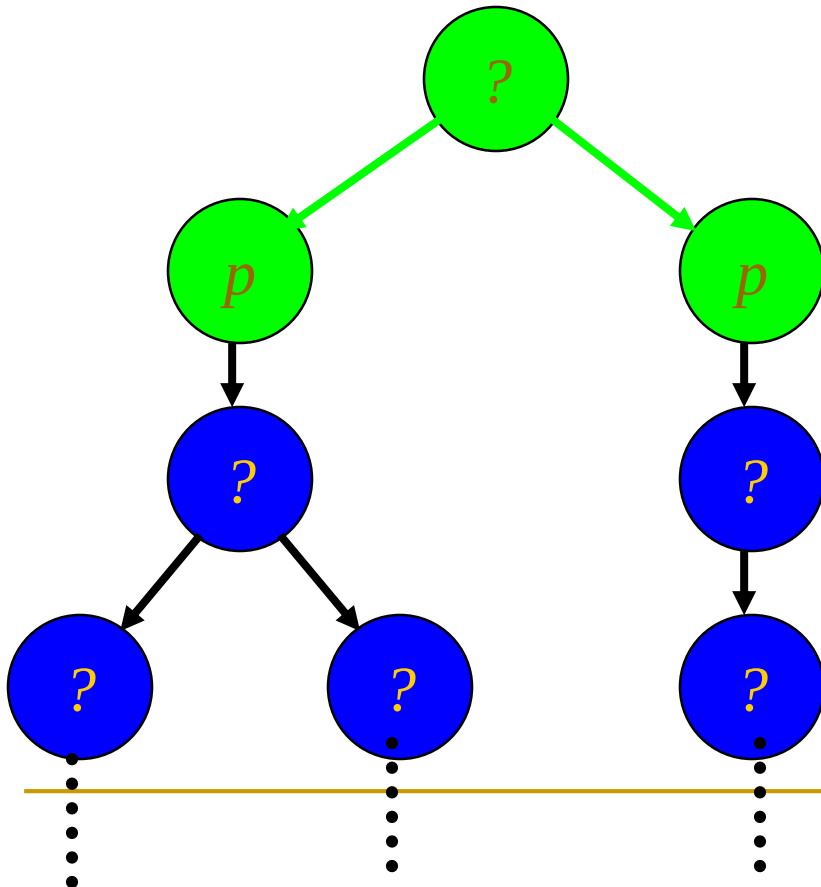
CTL

- semantics

$\forall \bigcirc p$

AXp

for all path where p is true on next state



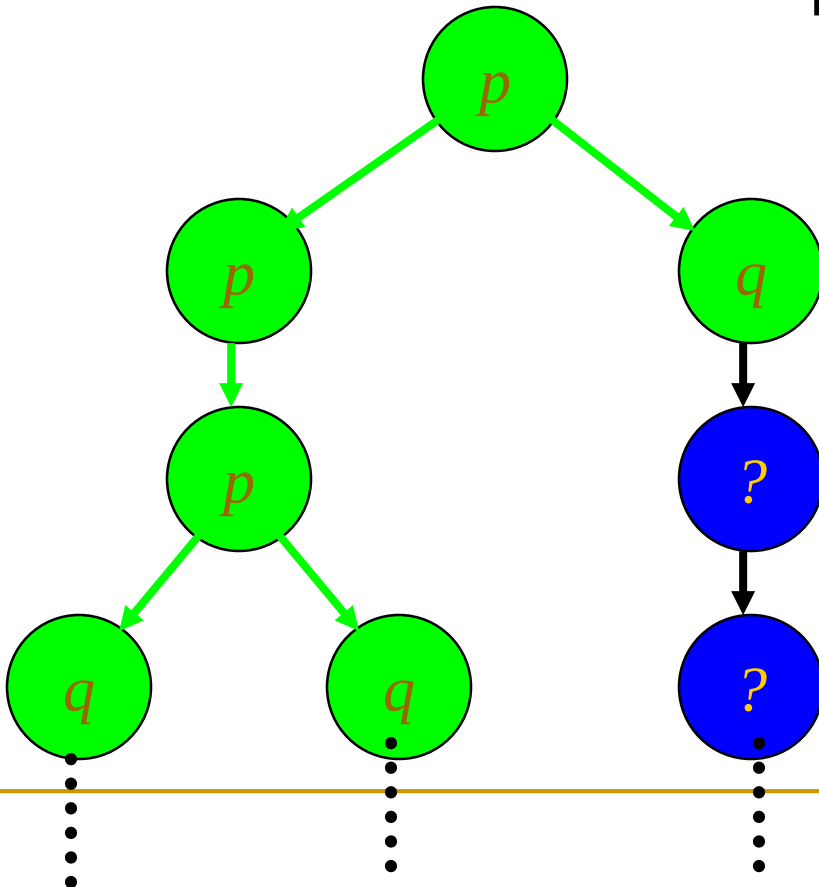
CTL

- semantics

$\forall p \cup q$

$p \text{AU} q$

from now on, for all path
where p is always true until q
is true



CTL

- semantic

Assume there are

- a tree structure M ,
- one state s in M , and
- a CTL formula φ

$M, s \models \varphi$ means s in M satisfy φ

CTL

- semantics

s-path : a path in M
which starts from s

s_0 -path:

$s_0 s_1 s_2 s_3 s_5 \dots$

$s_0 s_1 s_6 s_7 s_8 \dots$

s_1 -path:

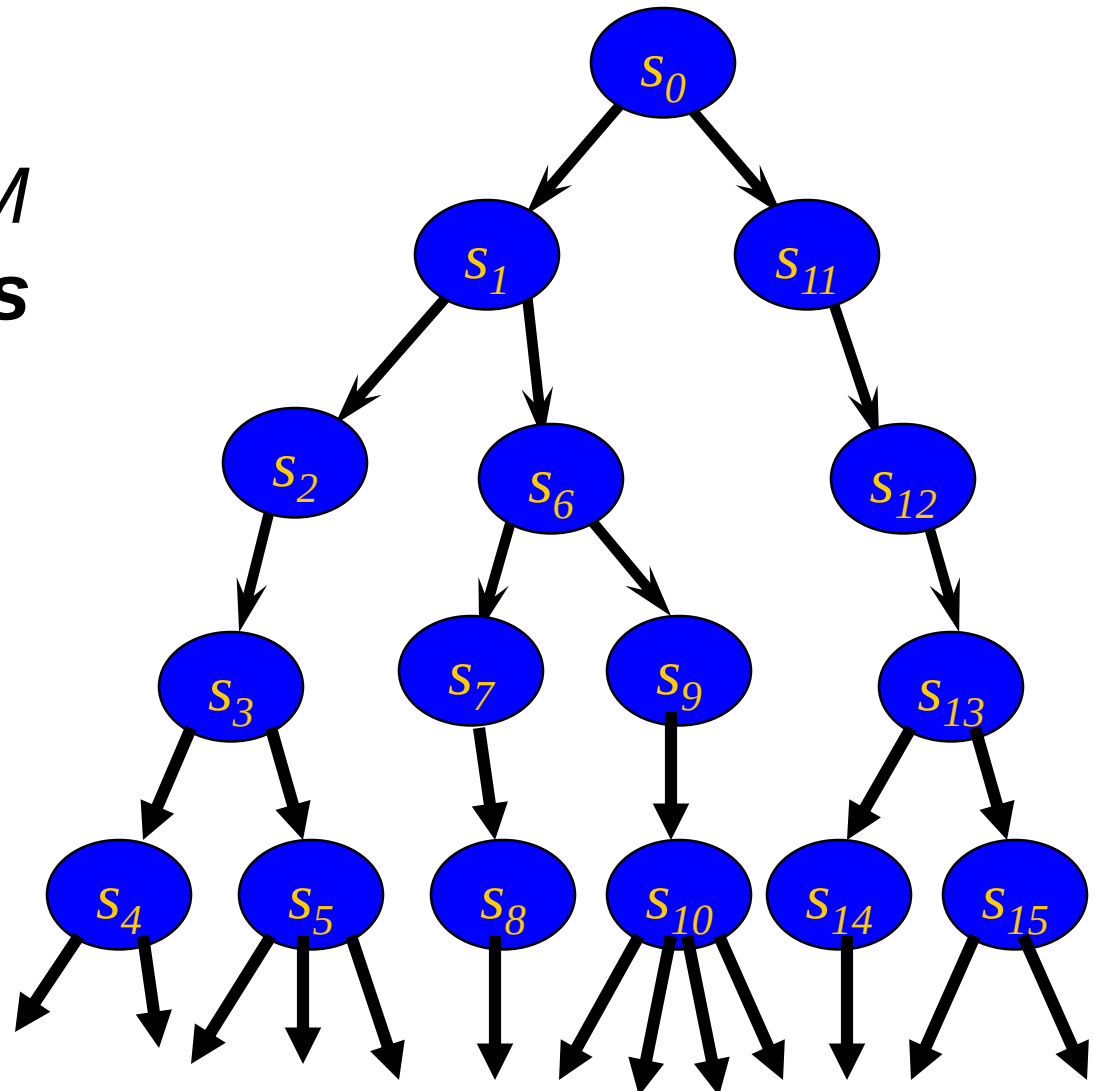
$s_1 s_2 s_3 s_5 \dots$

s_2 -path:

$s_2 s_3 s_5 \dots$

s_{13} -path:

$s_{13} s_{15} \dots$



CTL

- semantics

- $M, s \quad \text{true}$
- $M, s \quad p \Leftrightarrow p \in s$
- $M, s \quad \neg\varphi \Leftrightarrow \text{it is false that } M, s \quad \varphi$
- $M, s \quad \varphi_1 \vee \varphi_2 \Leftrightarrow M, s \quad \varphi_1 \text{ or } M, s \quad \varphi_2$
- $M, s \quad \exists O\varphi \Leftrightarrow \exists \text{ s-path } = s_0 s_1 \dots (M, s_1 \quad \varphi)$
- $M, s \quad \forall O\varphi \Leftrightarrow \forall \text{ s-path } = s_0 s_1 \dots (M, s_1 \quad \varphi)$
- $M, s \quad \exists \varphi_1 U \varphi_2 \Leftrightarrow \exists \text{ s-path } = s_0 s_1 \dots, \exists k \geq 0$
 $(M, s_k \quad \varphi_2 \wedge \forall 0 \leq j < k (M, s_j \quad \varphi_1))$
- $M, s \quad \forall \varphi_1 U \varphi_2 \Leftrightarrow \forall \text{ s-path } = s_0 s_1 \dots, \exists k \geq 0$
 $(M, s_k \quad \varphi_2 \wedge \forall 0 \leq j < k (M, s_j \quad \varphi_1))$

CTL

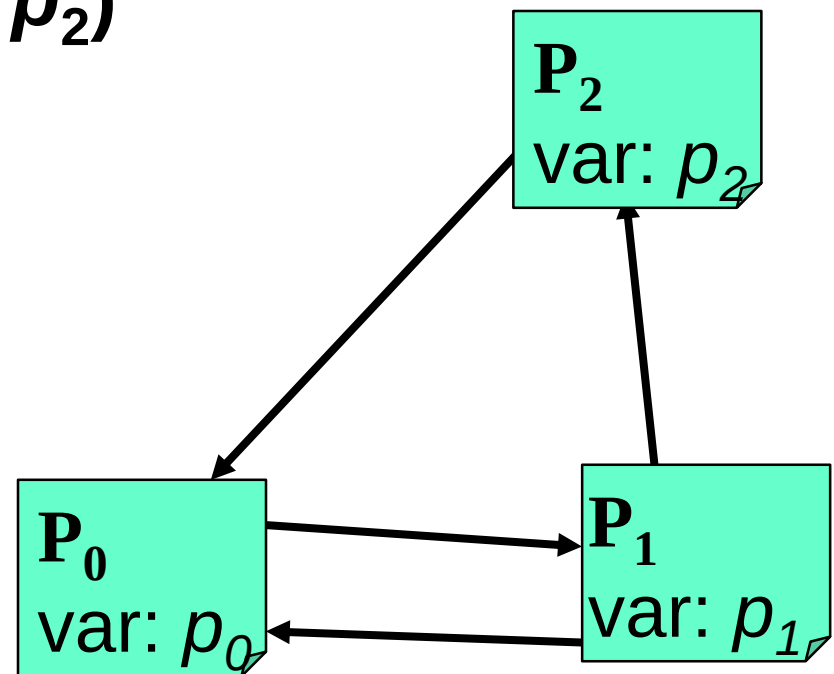
- examples (I)

$P_0: (p_0 := 0 \mid p_0 := p_0 \vee p_1 \vee p_2)$

$P_1: (p_1 := 0 \mid p_1 := p_0 \vee p_1)$

$P_2: (p_2 := 0 \mid p_2 := p_1 \vee p_2)$

If P_0 is true, it is possible that P_2 can be true after the next two cycles.



$\forall \square (p_0 \rightarrow \exists \bigcirc \exists \bigcirc p_2)$

CTL

- examples (II)

1. If there are dark clouds, it will rain.

$$\forall \Box (\text{dark-clouds} \rightarrow \forall \Diamond \text{rain})$$

2. if a butterfly flaps its wings, the New York stock could plunder.

$$\forall \Box (\text{butterfly-flap-wings} \rightarrow \exists \Diamond \text{NY-stock-plunder})$$

3. if I win the lottery, I will be happy forever.

$$\forall \Box (\text{win-lottery} \rightarrow \forall \Box \text{happy})$$

4. In an execution state, if an interrupt occurs in the next cycle, the interrupt handler will execute at the 2nd next cycle.

$$\forall \Box (\text{exec} \rightarrow \forall \bigcirc (\text{intrpt} \rightarrow \forall \bigcirc (\text{intrpt-handler})))$$

CTL

- examples (III)

In an execution state, if an interrupt occurs in the next cycle, the interrupt handler will execute at the 2nd next cycle.

$$\forall \Box (\text{exec} \rightarrow \forall \bigcirc (\text{intrpt} \rightarrow \forall \bigcirc (\text{intrpt-handler})))$$

Some possible mistakes:

$$\forall \Box (\text{exec} \rightarrow ((\forall \bigcirc \text{intrpt}) \rightarrow \forall \bigcirc \text{intrpt-handler}))$$

$$\forall \Box (\text{exec} \rightarrow ((\forall \bigcirc \text{intrpt}) \rightarrow \forall \bigcirc \forall \bigcirc \text{intrpt-handler}))$$

CTL

- examples (IIIa)

Please draw a Kripke structure that tells

$$\forall \bigcirc (\text{intrpt} \rightarrow \forall \bigcirc (\text{intrpt-handler}))$$

from

$$(\forall \bigcirc \text{intrpt}) \rightarrow \forall \bigcirc \text{intrpt-handler}$$

and

$$(\forall \bigcirc \text{intrpt}) \rightarrow \forall \bigcirc \forall \bigcirc \text{intrpt-handler}$$

CTL

- important classes

■ $\forall \eta$: safety properties

- η is always true in all computations from now.

■ $\exists \Diamond \eta$: reachability properties

- η is eventually true in some computation from now.

- $\forall \eta \equiv \neg \exists \Diamond \neg \eta$

■ $\forall \Diamond \eta$: inevitabilities

- η is eventually true in all computations from now.

■ $\exists \eta$

- $\forall \Diamond \eta \equiv \neg \exists \neg \eta$

CTL*

- syntax

- CTL* formula (state-formula)

$$\varphi ::= \text{true} \mid p \mid \neg \varphi_1 \mid \varphi_1 \vee \varphi_2 \mid \exists \psi \mid \forall \psi$$

- path-formula

$$\psi ::= \varphi \mid \neg \psi_1 \mid \psi_1 \vee \psi_2 \mid O\psi_1 \mid \psi_1 U \psi_2$$

CTL* is the set of all state-formulas!

CTL*

- examples (1 / 4)

In a fair concurrent environment, jobs will eventually finish.

$$\forall(((\Box\Diamond\textit{execute}_1) \wedge (\Box\Diamond\textit{execute}_2)) \rightarrow \Diamond\textit{finish})$$

or

$$\forall(((\Diamond^\infty\textit{execute}_1) \wedge (\Diamond^\infty\textit{execute}_2)) \rightarrow \Diamond\textit{finish})$$

CTL*

- semantics

suffix path :

$S = s_0 s_1 s_2 s_3 s_5 \dots$

$S^{(0)} = s_0 s_1 s_2 s_3 s_5 \dots$

$S^{(1)} = s_1 s_2 s_3 s_5 \dots$

$S^{(2)} = s_2 s_3 s_5 \dots$

$S^{(3)} = s_3 s_5 \dots$

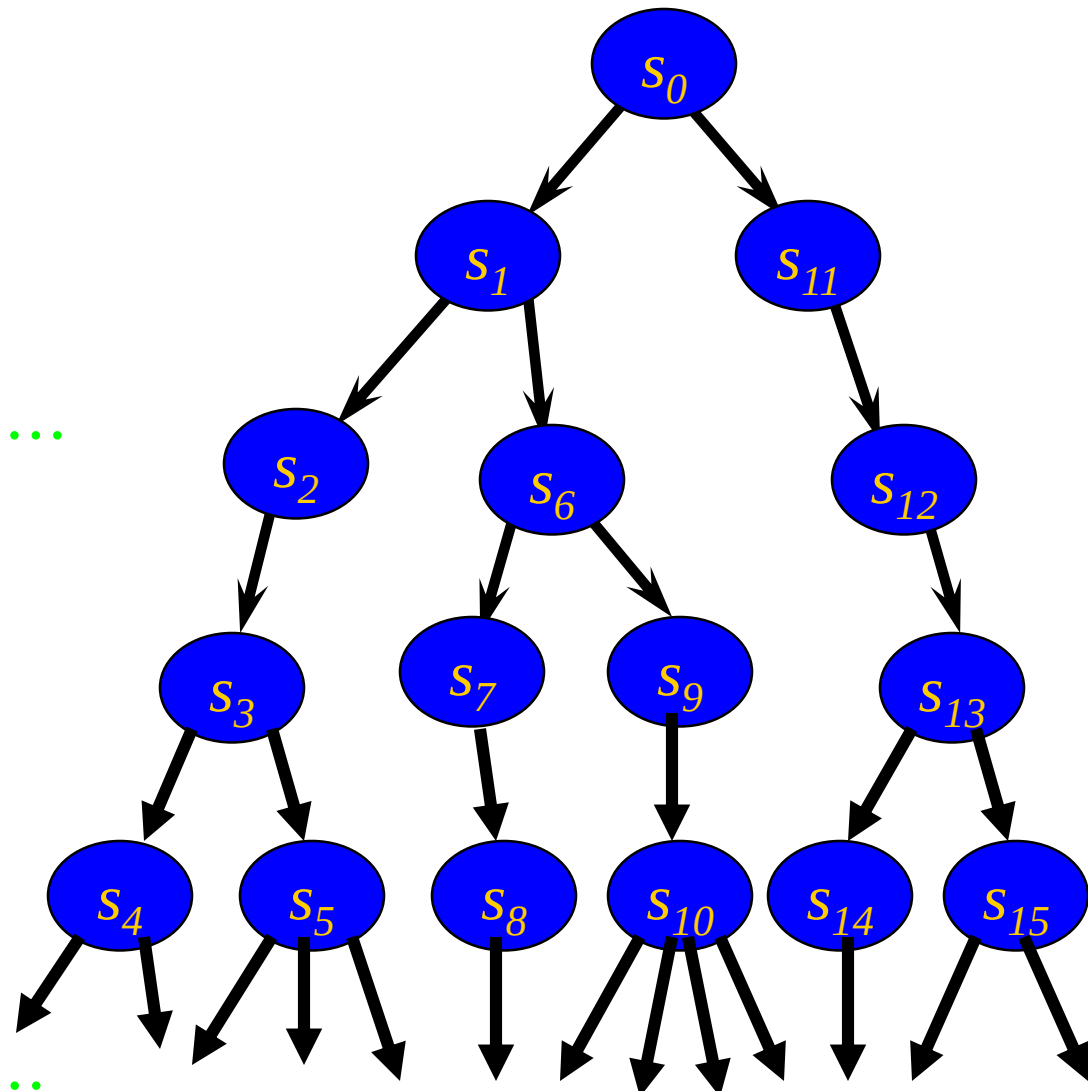
$S^{(4)} = s_5 \dots$

$S = s_0 s_1 s_6 s_7 s_8 \dots$

$S^{(2)} = s_6 s_7 s_8 \dots$

$S = s_0 s_{11} s_{12} s_{13} s_{15} \dots$

$S^{(3)} = s_{13} s_{15} \dots$



CTL*

- semantics

state-formula

$\varphi ::= \text{true} \mid p \mid \neg\varphi_1 \mid \varphi_1 \vee \varphi_2 \mid \exists\psi \mid \forall\psi$

- $M, s \quad \text{true}$
- $M, s \quad p \Leftrightarrow p \in s$
- $M, s \quad \neg\varphi \Leftrightarrow M, s \quad \varphi \text{ 是false}$
- $M, s \quad \varphi_1 \vee \varphi_2 \Leftrightarrow M, s \quad \varphi_1 \text{ or } M, s \quad \varphi_2$
- $M, s \quad \exists\psi \Leftrightarrow \exists \text{ s-path} = S (S \quad \psi)$
- $M, s \quad \forall\psi \Leftrightarrow \forall \text{ s-path} = S (S \quad \psi)$

CTL*

- semantics

path-formula

$$\psi ::= \varphi \mid \neg \psi_1 \mid \psi_1 \vee \psi_2 \mid O\psi \mid \psi_1 U \psi_2$$

- If $S = s_0 s_1 s_2 s_3 s_4 \dots$, $\varphi \Leftrightarrow M, s_0 \quad \varphi$
- $S \quad \neg \psi_1 \Leftrightarrow S \quad \psi_1$ 是 false
- $S \quad \psi_1 \vee \psi_2 \Leftrightarrow S \quad \psi_1$ or $S \quad \psi_2$
- $S \quad O\psi \Leftrightarrow S^{(1)} \quad \psi$
- $S \quad \psi_1 U \psi_2 \Leftrightarrow \exists k \geq 0 (S^{(k)} \quad \psi_2 \wedge \forall 0 \leq j < k (S^{(j)} \quad \psi_1))$

Expressiveness

Given a language L ,

- what model sets L can express ?
- what model sets L cannot ?

model set: a set of behaviors

A formula = a set of models (behaviors)

- for any $\varphi \in L$, $[\varphi] \stackrel{\text{def}}{=} \{M \mid M \models \varphi\}$

A language = a set of formulas.

Expressiveness: Given a model set F ,

F is **expressible** in L iff $\exists \varphi \in L ([\varphi] = F)$

Expressiveness

Comparison in expressiveness:

Given two languages L_1 and L_2

Definition: L_1 is **more expressive than** L_2 ($L_2 < L_1$)
iff $\forall \varphi \in L_2$ ($[\varphi]$ is expressible in L_1)

Definition: L_1 and L_2 **are expressively equivalent**
($L_1 \equiv L_2$) iff $(L_2 < L_1) \wedge (L_1 < L_2)$

Definition: L_1 & L_2 are **expressively incomparable** iff
 $\neg((L_2 < L_1) \vee (L_1 < L_2))$

Expressiveness

- branching-time logics

What to compare with ?

- finite-state automata on infinite trees.
- 2nd-order logics with monadic predicate and many successors (SnS)
- 2nd-order logics with monadic and partial-order

Very little known at the moment,

the fine difference in semantics of branching-structures

Expressiveness

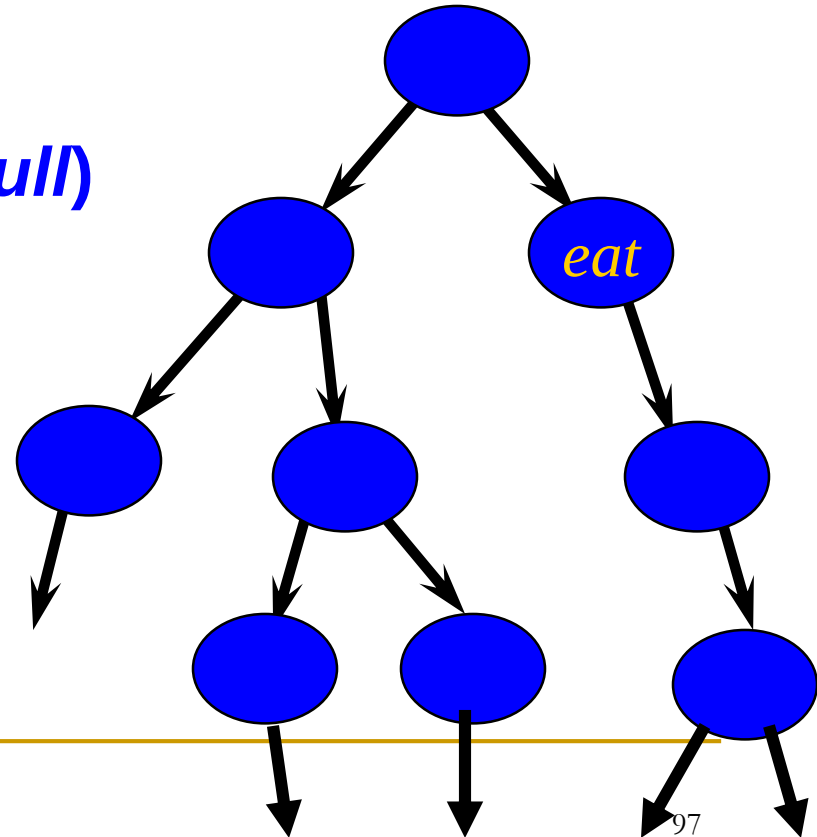
- CTL*, example (I)

A tree that distinguishes the following two formulas.

■ $\forall((\Diamond eat) \rightarrow \Diamond full)$

□ Negation: $\exists((\Diamond eat) \wedge \Box \neg full)$

■ $(\forall \Diamond eat) \rightarrow (\forall \Diamond full)$

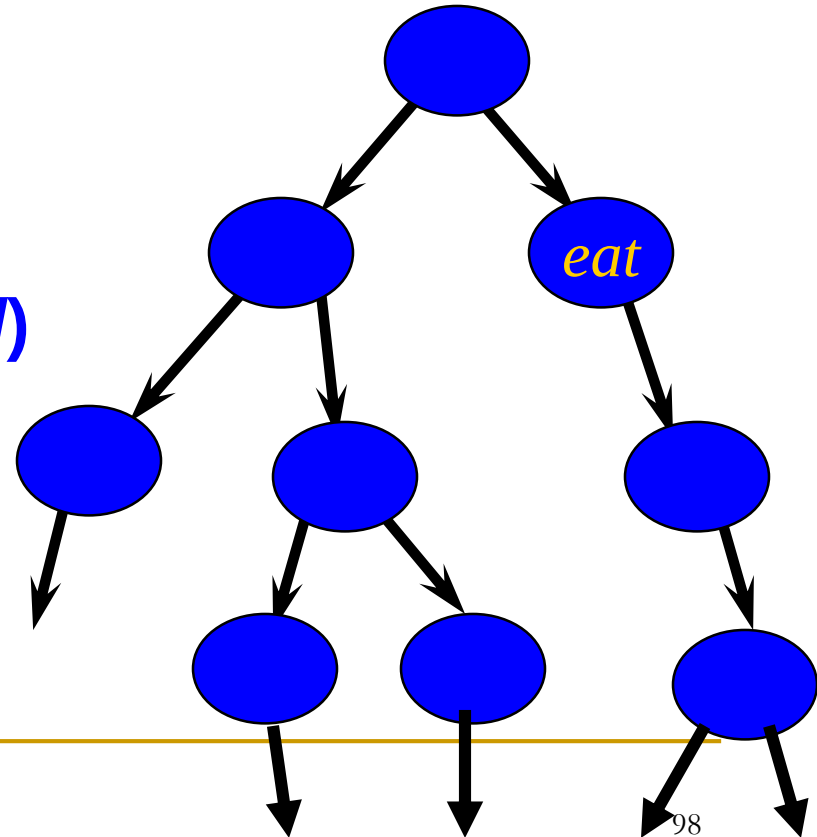


Expressiveness

- CTL*, example (II)

A tree that distinguishes the following two formulas.

- $\forall((\Box eat) \rightarrow \Diamond full)$
- $\forall\Box (eat \rightarrow \forall\Diamond full)$
- **Negation:** $\exists\Diamond (eat \wedge \exists\Diamond \neg full)$



CTL*

- examples (2/4)

No matter what, infinitely many comets will hit earth.

$\forall \square \bigcirc \diamond \text{comet-hit-earth}$

Why not CTL?

■ $\forall \square \forall \square \bigcirc \forall \square \diamond \text{comet-hit-earth}$

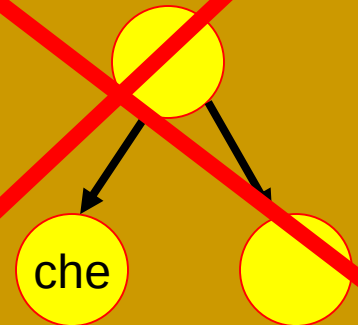
■ $\forall \square \forall \square \bigcirc \exists \square \diamond \text{comet-hit-earth}$

Exercise, please construct a
model that tells the last
from the first

Difference ?



Difference ?



CTL*

- examples (2/4)

No matter what, infinitely many comets will hit earth.

$\forall \square \diamond \text{comet-hit-earth}$

Or

$\forall \diamond^\infty \text{comet-hit-earth}$

Why not CTL?

■ $\forall \square \forall \diamond \text{comet-hit-earth}$

■ $\forall \square \exists \diamond \text{comet-hit-earth}$

What is the difference ?
weak
next !

true

CTL*

- Workout

The same
according to
lemma

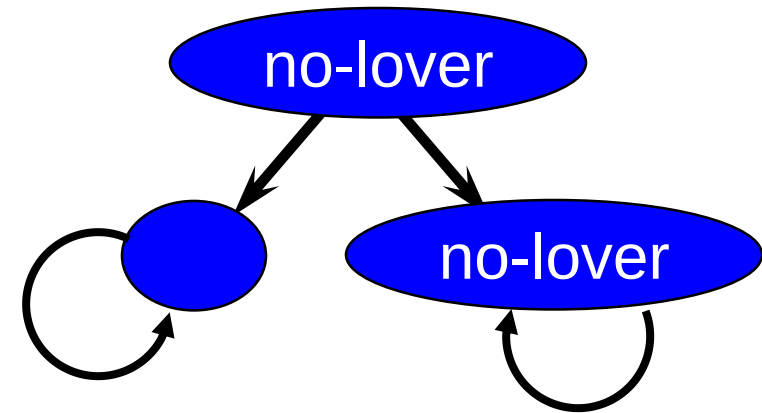
- **(1)** $\forall \square \diamond \text{comet-hit-earth}$
- **(2)** $\forall \square \forall \diamond \text{comet-hit-earth}$
- **(3)** $\forall \square \exists \diamond \text{comet-hit-earth}$

Please draw Kripke structures that tell

- (1) from (2) and (3)
- (2) from (1) and (3)
- (3) from (1) and (2)

CTL*

- examples (3/4)



If you never have a lover, I will marry you.

$\forall((\Box \text{you-have-no-lover}) \rightarrow \Diamond \text{marry-you})$

Why not CTL ?

- $(\forall \Box \text{you-have-no-lover}) \rightarrow \forall \Diamond \text{marry-you}$
- $(\forall \Box \text{you-have-no-lover}) \rightarrow \exists \Diamond \text{marry-you}$
- $(\exists \Box \text{you-have-no-lover}) \rightarrow \forall \Diamond \text{marry-you}$

CTL*

- Workout

- (1) $\forall((\Box \text{you-have-no-lover}) \rightarrow \Diamond \text{marry-you})$
- (2) $(\forall \Box \text{you-have-no-lover}) \rightarrow \forall \Diamond \text{marry-you}$
- (3) $(\forall \Box \text{you-have-no-lover}) \rightarrow \exists \Diamond \text{marry-you}$
- (4) $(\exists \Box \text{you-have-no-lover}) \rightarrow \forall \Diamond \text{marry-you}$

Please draw trees that tell

- (1) *from* (2)
- (2) *from* (3)
- (3) *from* (4)
- (4) *from* (1)

CTL*

- examples (4/4)

If I buy lottery tickets infinitely many times,
eventually I will win the lottery.

$$\forall ((\Box \Diamond \text{buy-lottery}) \rightarrow \Diamond \text{win-lottery})$$

or

$$\forall ((\Diamond^\infty \text{buy-lottery}) \rightarrow \Diamond \text{win-lottery})$$

Expressiveness

- CTL*

With the abundant semantics in CTL*, we can compare the subclasses of CTL*.

With restrictions on the modal operations after $\exists, \forall$, we have many CTL* subclasses.

Example:

$B(\neg, \vee, \bigcirc, U)$: only $\neg, \vee, \bigcirc, U$ after $\exists, \forall$

$B(\neg, \vee, \bigcirc, \Diamond^\infty)$: only $\neg, \vee, \bigcirc, \Diamond^\infty$ after $\exists, \forall$

$B(\bigcirc, \Diamond)$: only $\bigcirc, \Diamond$ after $\exists, \forall$

Expressiveness

- CTL*

CTL* subclass expressiveness hierarchy

CTL*	>	$B(\neg, \vee, \bigcirc, \Diamond, U, \Diamond^\infty)$
	>	$B(\bigcirc, \Diamond, U, \Diamond^\infty)$
	>	$B(\neg, \vee, \bigcirc, \Diamond, U)$
	=	$B(\bigcirc, \Diamond, U)$
	>	$B(\neg, \vee, \bigcirc, \Diamond)$
	>	$B(\bigcirc, \Diamond)$
	>	$B(\Diamond)$

Expressiveness

- CTL*

Some theorems :

■ $B(\neg, \vee, \bigcirc, \Diamond, U) \equiv B(\bigcirc, \Diamond, U)$

■ $\exists \Diamond^\infty p$ is inexpressible in $B(\bigcirc, \Diamond, U)$.

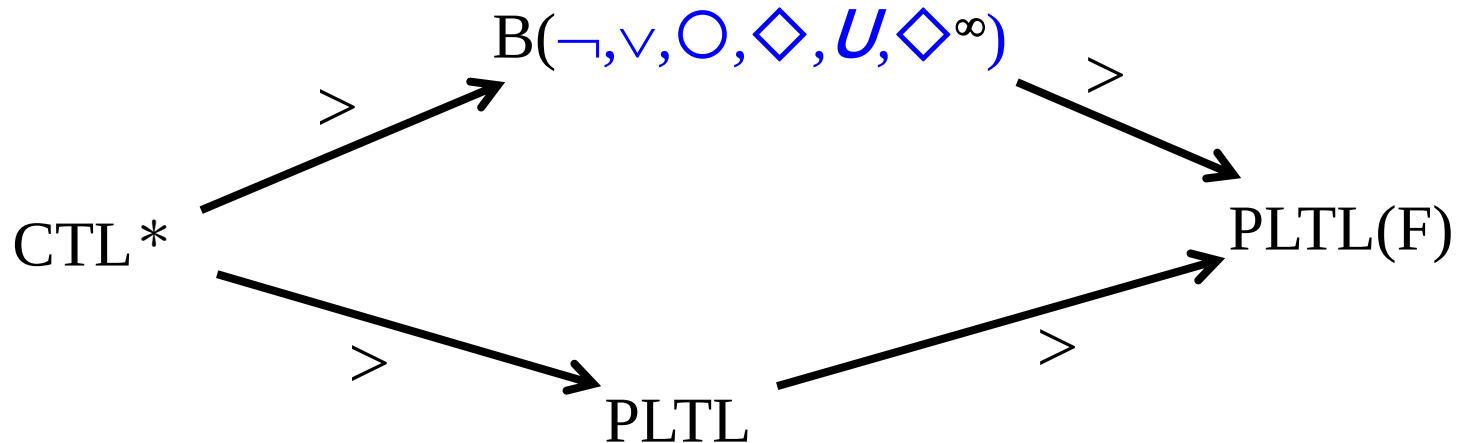
Expressiveness

- CTL*

Comparing PLTL with CTL*

assumption, all $\phi \in \text{PLTL}$ are interpreted as $\forall \phi$

Intuition: PLTL is used to specify all runs of a system.



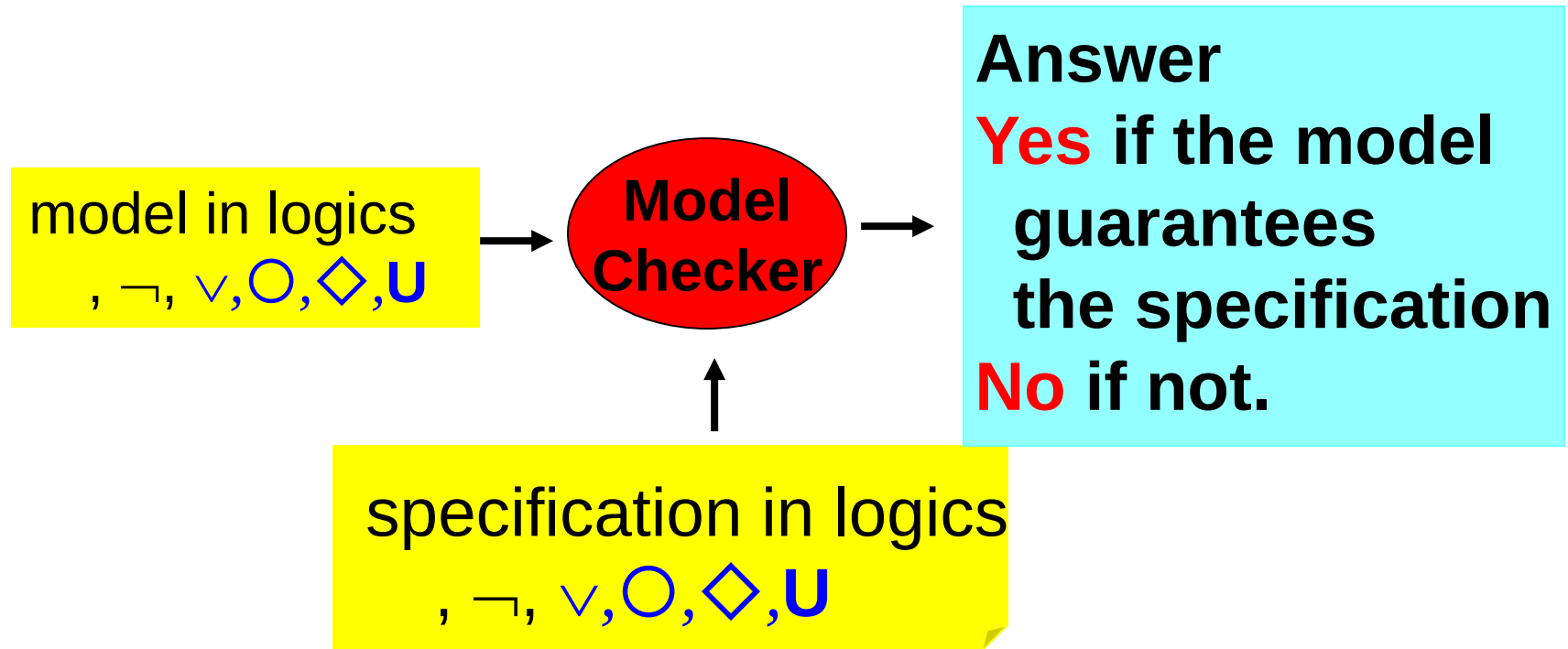
Verification

model (system)
formula

specification
formula

- LPTL, validity checking
 - instead, check the satisfiability of $\phi \wedge \neg \psi$
 - construct a tabelau for $\phi \wedge \neg \psi$
- model-checking M
 - LPTL: M: a Büchi automata, ϕ : an LPTL formula
 - CTL: M: a finite-state automata, ϕ : a CTL formula
- simulation & bisimulation checking M M'

Satisfiability-checking framework



LPTL

- tableau for satisfiability checking

Given $\varphi_m \rightarrow \varphi_s$,

- we in fact check the validity of $\varphi_m \rightarrow \varphi_s$,
 - i.e., check $\varphi_m \rightarrow \varphi_s$ is always true.
- in practice, we check whether
$$\neg(\varphi_m \rightarrow \varphi_s) \equiv \varphi_m \wedge \neg \varphi_s$$
- is satisfiable
 - i.e., the satisfiability of $\varphi_m \wedge \neg \varphi_s$

LPTL

- tableau for satisfiability checking

Tableau for φ

- a finite Kripke structure that fully describes the behaviors of φ
- **exponential** number of states
- An algorithm can explore a fulfilling path in the tableau to answer the satisfiability.
 - nondeterministic
 - without construction of the tableau
 - PSPACE.

LPTL

- tableau for satisfiability checking

Tableau construction

a preprocessing step: push all negations to the literals.

- $\neg (\psi_1 \wedge \psi_2) \equiv (\neg \psi_1) \vee (\neg \psi_2)$
- $\neg (\psi_1 \vee \psi_2) \equiv (\neg \psi_1) \wedge (\neg \psi_2)$
- $\neg \neg \psi \equiv \psi$
- $\neg \neg \psi \equiv \psi$
- $\neg (\psi_1 \mathbf{U} \psi_2) \equiv (\Box \neg \psi_2) \vee ((\neg \psi_2) \mathbf{U} ((\neg \psi_1) \wedge (\neg \psi_2)))$
- $\neg \Box \psi \equiv \Diamond \neg \psi$
- $\neg \Diamond \psi \equiv \Box \neg \psi$

LPTL

- tableau for satisfiability checking

Tableau construction

$\text{CL}(\varphi)$ (**closure**) is the smallest set of formulas containing φ with the following consistency requirement.

- $\varphi \in \text{CL}(\varphi)$
- $\neg \psi \in \text{CL}(\varphi)$ iff $\psi \in \text{CL}(\varphi)$
- If $\psi_1 \vee \psi_2, \psi_1 \wedge \psi_2 \in \text{CL}(\varphi)$, then $\psi_1, \psi_2 \in \text{CL}(\varphi)$
- If $\psi \in \text{CL}(\varphi)$, then $\psi \in \text{CL}(\varphi)$
- If $\psi_1 \mathbf{U} \psi_2 \in \text{CL}(\varphi)$, then $\psi_1, \psi_2, (\psi_1 \mathbf{U} \psi_2) \in \text{CL}(\varphi)$
- If $\Box \psi \in \text{CL}(\varphi)$, then $\psi, \Box \psi \in \text{CL}(\varphi)$

LPTL

- tableau for satisfiability checking

Tableau (V, E) , *node consistency condition*:

A tableau node $v \in V$ is a set $v \subseteq CL(f)$ such that

- $p \in v$ iff $\neg p \notin v$
- If $\psi_1 \vee \psi_2 \in v$, then $\psi_1 \in v$ or $\psi_2 \in v$
- If $\psi_1 \wedge \psi_2 \in v$, then $\psi_1 \in v$ and $\psi_2 \in v$
- if $\Box \psi \in v$, then $\psi \in v$ and $\Box \psi \in v$
- if $\Diamond \psi \in v$, then $\psi \in v$ or $\Diamond \psi \in v$
- if $\psi_1 \mathbf{U} \psi_2 \in v$, then $\psi_2 \in v$ or $(\psi_1 \in v \text{ and } (\psi_1 \mathbf{U} \psi_2) \in v)$

LPTL

- tableau for satisfiability checking

Tableau (V, E) , *arc consistency condition*:

Given an arc $(v, v') \in E$, if $\psi \in v$, then $\psi \in v'$

- A node v in (V, E) is initial for φ if $\varphi \in v$.

LPTL

- tableau for satisfiability checking

$CL(p \cup q) = \{p \cup q, \quad p \cup q, p, \neg p, q, \neg q\}$

Example: $(p \cup q)$

tableau (V, E)

V:	$\{p, q, p \cup q, \quad p \cup q\}$	$\{p, q, \quad p \cup q\}$	$\{p, q\}$
	$\{p, q, p \cup q\}$		
	$\{p, \neg q, p \cup q, \quad p \cup q\}$	$\{p, \neg q, \quad p \cup q\}$	$\{p, \neg q\}$
	$\{\neg p, q, p \cup q, \quad p \cup q\}$	$\{\neg p, q, p \cup q\}$	$\{\neg p, q\}$
	$\{\neg p, q, \quad p \cup q\}$		
	$\{\neg p, \neg q, \quad p \cup q\}$	$\{\neg p, \neg q\}$	

E: ?

LPTL

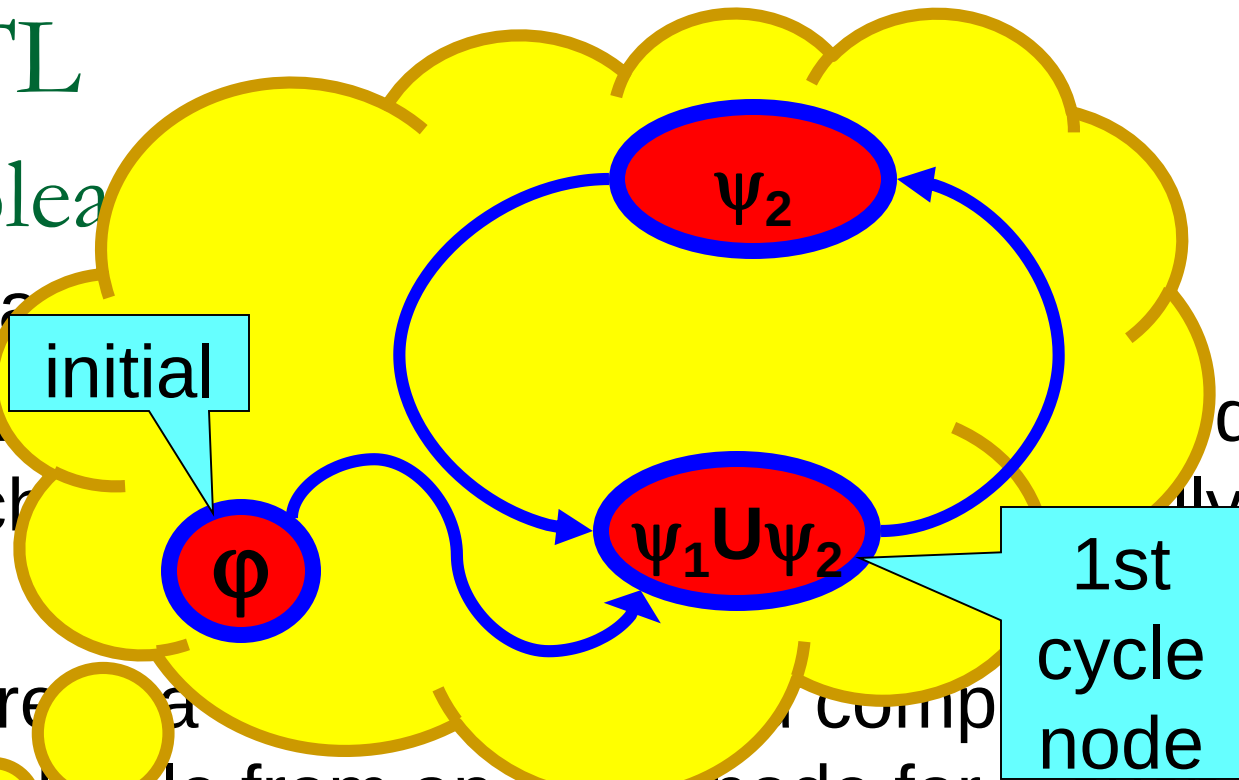
- tableau

φ is satisfied

■ there is a node for φ such that φ is satisfied; or

■ there is a cycle reachable from an initial node for φ such that for all until formula $\psi_1 \mathbf{U} \psi_2$ in a node in the SCC, there is also a node in the SCC containing ψ_2 ; or

■ there is a cycle reachable from an initial node for φ such that the for all until formulas $\psi_1 \mathbf{U} \psi_2$ in **the first cycle node**, there is also a node in the cycle containing ψ_2 .



LPTL

- tableau for satisfiability checking

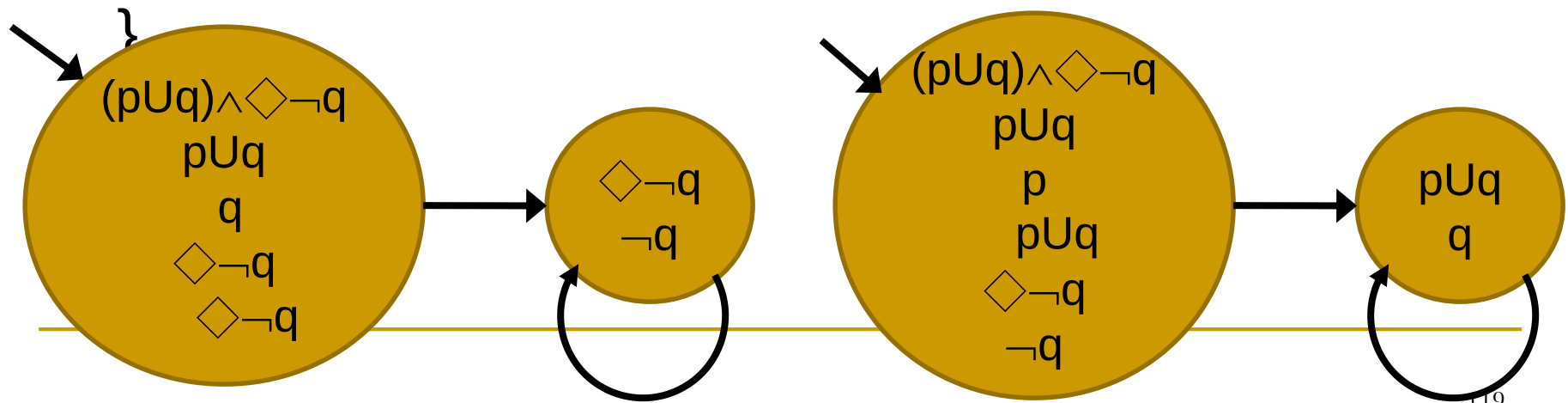
Please use tableau method to show that

$p \cup q \quad \Box q$ is false.

1) Convert to negation: $(p \cup q) \wedge \Diamond \neg q$

2) $CL((p \cup q) \wedge \Diamond \neg q)$

$= \{(p \cup q) \wedge \Diamond \neg q, p \cup q, \quad p \cup q, p, q, \Diamond \neg q, \quad \Diamond \neg q$



LPTL

- tableau for satisfiability checking

Please use tableau method to show that

$p \cup q \quad \Diamond q$ is true.

1) Convert to negation: $(p \cup q) \wedge \Box \neg q$

2) CL($(p \cup q) \wedge \Box \neg q$)

$$= \{(p \cup q) \wedge \Box \neg q, p \cup q, \quad p \cup q, p, q, \Box \neg q, \quad \Box \neg q\}$$

Pf: In each path that is a model of $(p \cup q) \wedge \Box \neg q$, q must always be satisfied. Thus, $p \cup q$ is never fulfilled in the model.

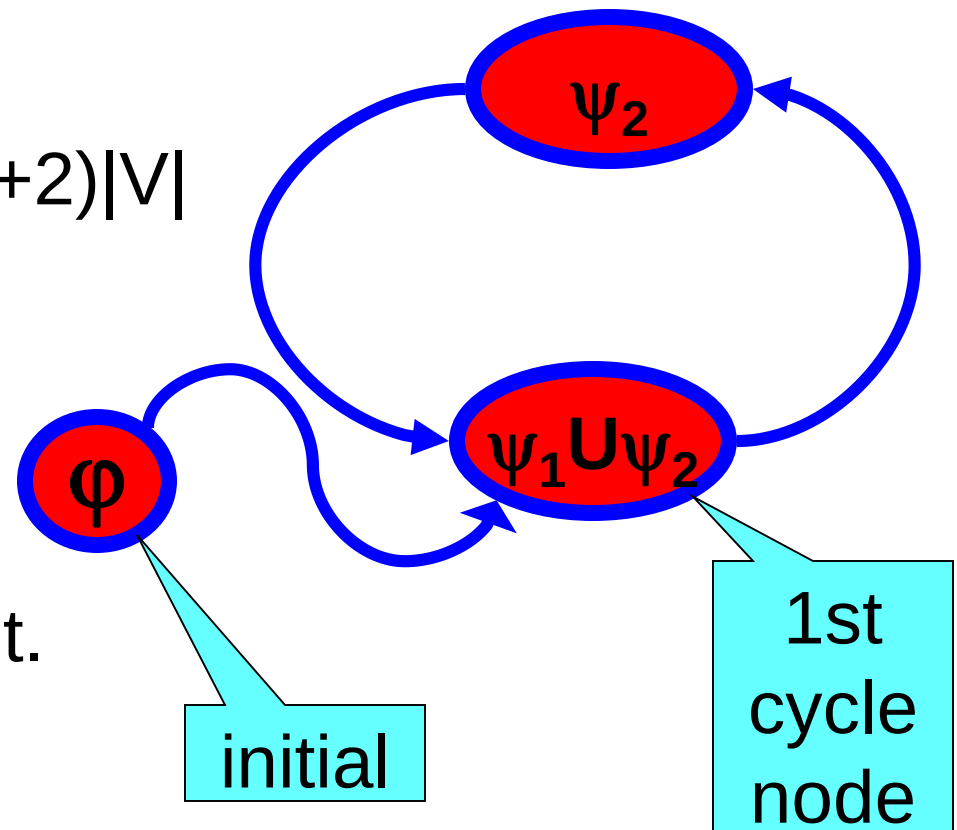
QED

LPTL

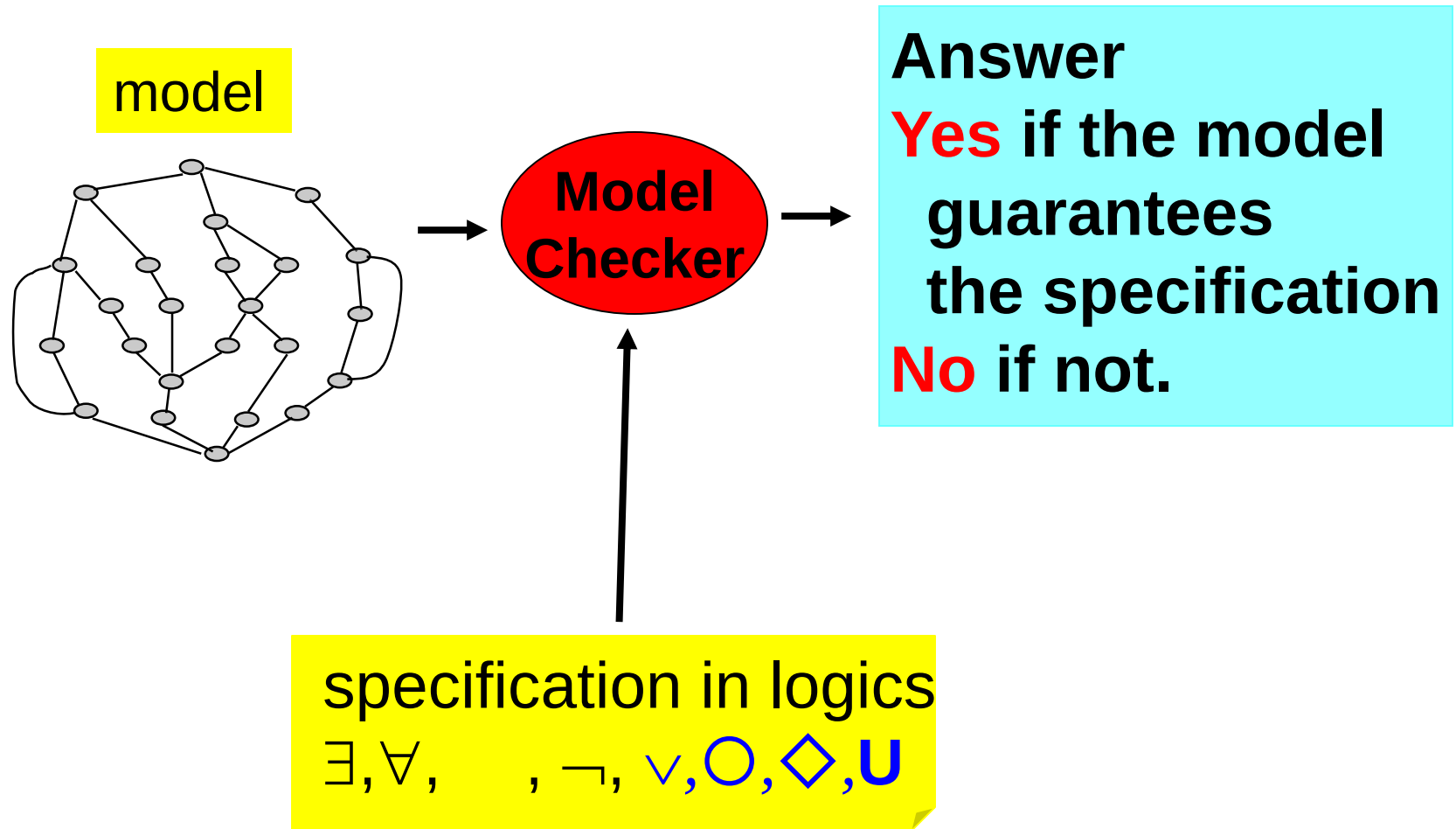
- tableau for satisfiability checking
 φ is satisfiable iff in (V, E) ,

there exists ...

- $\text{path} + \text{cycle} \leq (|\text{CL}(\varphi)| + 2)|V|$
- $|\text{CL}(\varphi)|$ flags to check the until-formulas from the first cycle node.
- nondeterministic PSPACE can solve it.
- PSPACE-complete.



CTL model-checking framework



CTL

- model-checking

Given a finite Kripke structure M and a CTL formula φ , is M a model of φ ?

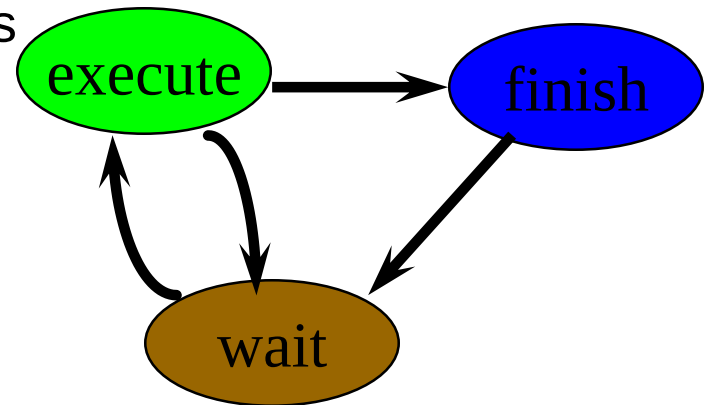
- usually, M is a finite-state automata.
- PTIME algorithm.
- When M is generated from a program with variables, its size is easily exponential.

CTL

- model-checking algorithm

techniques

- state-space exploration
 - state-spaces represented as finite Kripke structure
 - directed graph
 - nodes: states or possible worlds
 - arcs: state transitions
 - regular behaviors
- Usually the state count is astronomical.



Kripke structure

- *Least fixpoint in modal logics*

逐步擴充法、連坐法

Dark-night murder, strategy I:

A suspect will be in the 2nd round iff

- *He/she lied to the police in the 1st round; or*
- *Some one in the 2nd round is loyal to him/her*

What is the minimal solution to *2nd[]* ?

$$Liar[i] \vee \exists j \neq i (2nd[j] \wedge Loyal-to[j,i]) \rightarrow 2nd[i]$$

Kripke structure

- *Least fixpoint in modal logics*

In a dark night, there was a cruel murder.

- n suspects, numbered 0 through $n-1$.
- *Liar*[i] iff suspect i has lied to the police in the 1st round investigation.
- *Loyal-to*[i,j] iff suspect i is loyal to suspect j in the same criminal gang.
- *2nd*[i] iff suspect i to be in 2nd round investigation.

What is the minimal solution to *2nd*[] ?

Kripke structure

- *Greatest fixpoint in modal logics*

逐步消去法

In a dark night, there was a cruel murder.

- n suspects, numbered 0 through $n-1$.
- $\neg \text{Liar}[i]$ iff the police cannot prove suspect i has lied to the police in the 1st round investigation.
- $\text{Loyal-to}[i,j]$ iff suspect i is loyal to j .
- $\text{2nd}[i]$ iff suspect i to be in 2nd round investigation.

What is the maximal solution to $\neg \text{2nd}[]$?

Kripke structure

- *Greatest fixpoint in modal logics*

Dark-night murder, strategy II

A suspect will not be in the 2nd round iff

- *We cannot prove he/she has lied to the police; and*
- *He/she is loyal to someone not in the 2nd round.*

What is the maximal solution to $\neg 2nd[]$?

$$\neg 2nd[i] \rightarrow \neg Liar[i] \wedge \exists j \neq i (\neg 2nd[j] \wedge Loyal-to[i,j])$$

In comparison:

$$\neg 2nd[i] \equiv \neg Liar[i] \wedge \forall j \neq i (\neg 2nd[j] \wedge Loyal-to[i,j])$$

~~$$\neg 2nd[i] \equiv \neg Liar[i] \wedge \forall j \neq i (\neg 2nd[j] \rightarrow Loyal-to[i,j])$$~~

$$\neg 2nd[i] \equiv \neg Liar[i] \wedge \forall j \neq i (Loyal-to[i,j] \rightarrow \neg 2nd[j])$$

Safety analysis

Given M and p (safety predicate), do all states reachable from initial states in M satisfy p ?

- In model-checking:

Is M a model of $\forall \Box p$?

- Or in **risk analysis**: Is there a state reachable from initial states in M satisfy p ?

$$\forall \Box p \equiv \neg \exists \Diamond \neg p \equiv \neg \exists \text{true} \cup \neg p$$

Reachability analysis: $\exists \Diamond \eta$

Is there a state s reachable from another state s' ?

- Encode risk analysis
- Encode the complement of safety analysis
- Most used in real applications

What space S characterizes $\exists \Diamond \eta$?

$$(M, s \models \eta \vee \exists s' (R(s, s') \wedge s' \in \exists \Diamond \eta)) \rightarrow s \in \exists \Diamond \eta$$

Is that all ? No, $\exists \Diamond \eta = \text{Universe}$ is a solution.

We need the smallest $\exists \Diamond \eta$.

$\exists \Diamond \eta$ characterizes a *least fixpoint*.

Kripke structure

- safety analysis

Reachability algorithm in graph theory

Given

- a Kripke structure $A = (S, S_0, R, L)$
- a safety predicate η ,

find a path from a state in S_0 to a state in $[\neg\eta]$.

Solutions in graph theory

- Shortest distance algorithms
- spanning tree algorithms

Kripke structure

- safety analysis

/* Given $A = (S, S_0, R, L)$ */

safety_analysis(η) /* using least fixpoint algorithm */ {

for all s , if $\neg\eta \in L(s)$, $L(s) = L(s) \cup \{\exists \Diamond \neg\eta\}$;

repeat {

for all s , if $\exists (s, s') (\exists \Diamond \neg\eta \in L(s'))$,

$L(s) = L(s) \cup \{\exists \Diamond \neg\eta\}$;

} until no more changes to $L(s)$ for any s .

if there is an $s_0 \in S_0$ with $\exists \Diamond \neg\eta \in L(s_0)$, return '*unsafe*,'

else return '*safe*.'

}

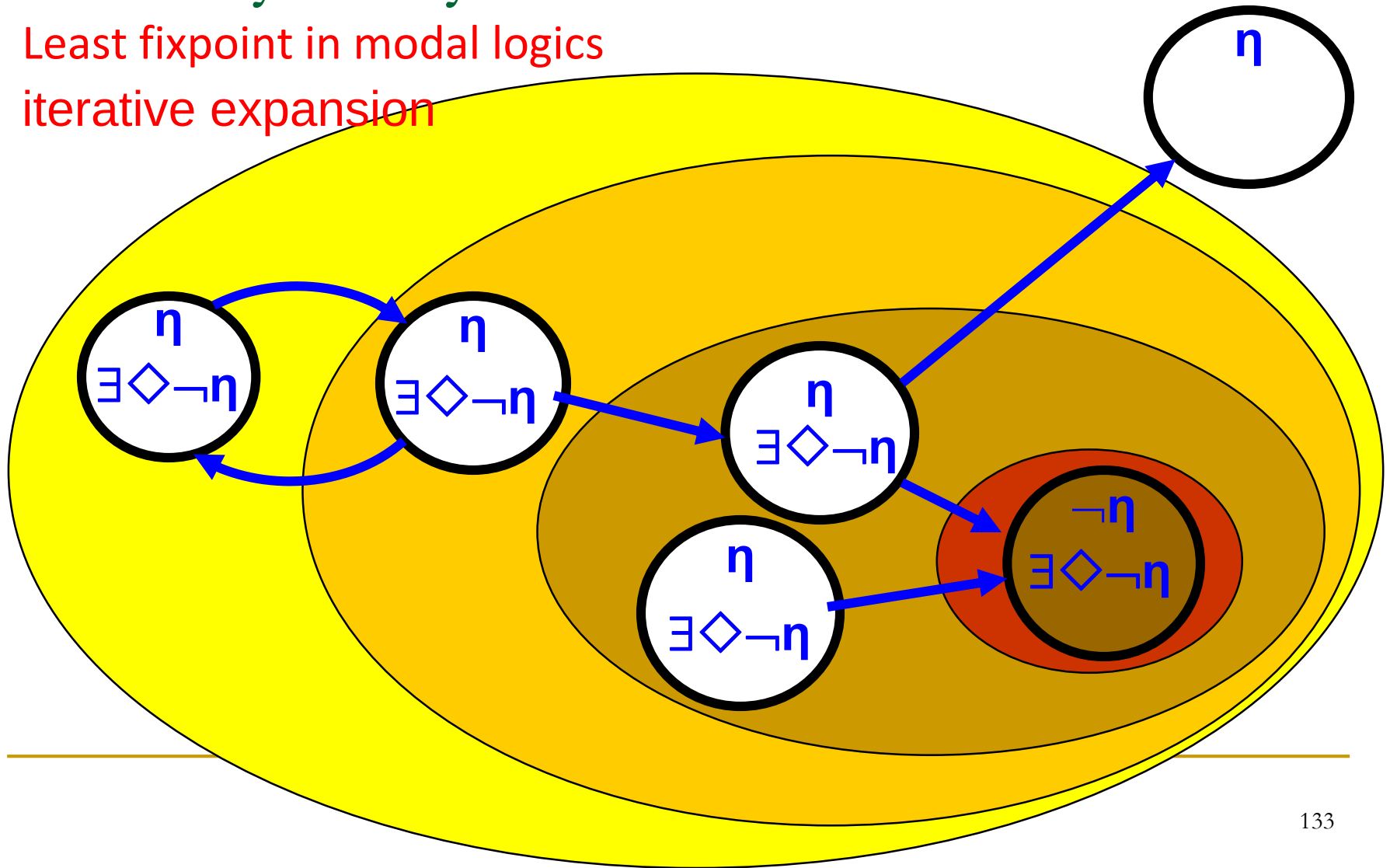
A notation for the possibility of $\neg\eta$

The procedure terminates since S is finite in the Kripke structure.

Kripke structure

- safety analysis

Least fixpoint in modal logics
iterative expansion



Inevitability analysis

Given M and p (inevitability predicate), do all computations from initial states in M reach p ?

- In model-checking:

Is M a model of $\forall \Diamond p$?

- Or in **avoidability analysis**: Is there a computation from initial states in M that maintains p ?

$$\forall \Diamond p \equiv \neg \exists \Box \neg p$$

Inevitability analysis: $\exists \Box \neg \eta$

Is there a computation that maintains $\neg \eta$?

- Encode avoidability analysis
- Encode the complement of inevitability analysis
- Most used in real applications

What space S characterizes $\exists \Box \neg \eta$?

$$s \in S \rightarrow (s \models \neg \eta \wedge \exists s' (R(s, s') \wedge s' \in S))$$

Is that all ?

No, $S = \emptyset$ is a solution. We need the largest S .

So, $\exists \Box \neg \eta$ characterizes a *greatest fixpoint*.

Kripke structure

- inevitability analysis : $\forall \Diamond \eta$

Given

- a Kripke structure $A = (S, S_0, R, L)$
- an inevitability predicate η ,

can η be true eventually ?

Example:

Can the computer be started successfully ?

Will the alarm sound in case of fire ?

Kripke structure

- inevitability analysis

Strongly connected component algorithm in graph theory

Given

- a Kripke structure $A = (S, S_0, R, L)$

- an inevitability predicate η ,

find a cycle such that

- all states in the cycle are $\neg\eta$

- there is a $\neg\eta$ path from a state in S_0 to the cycle.

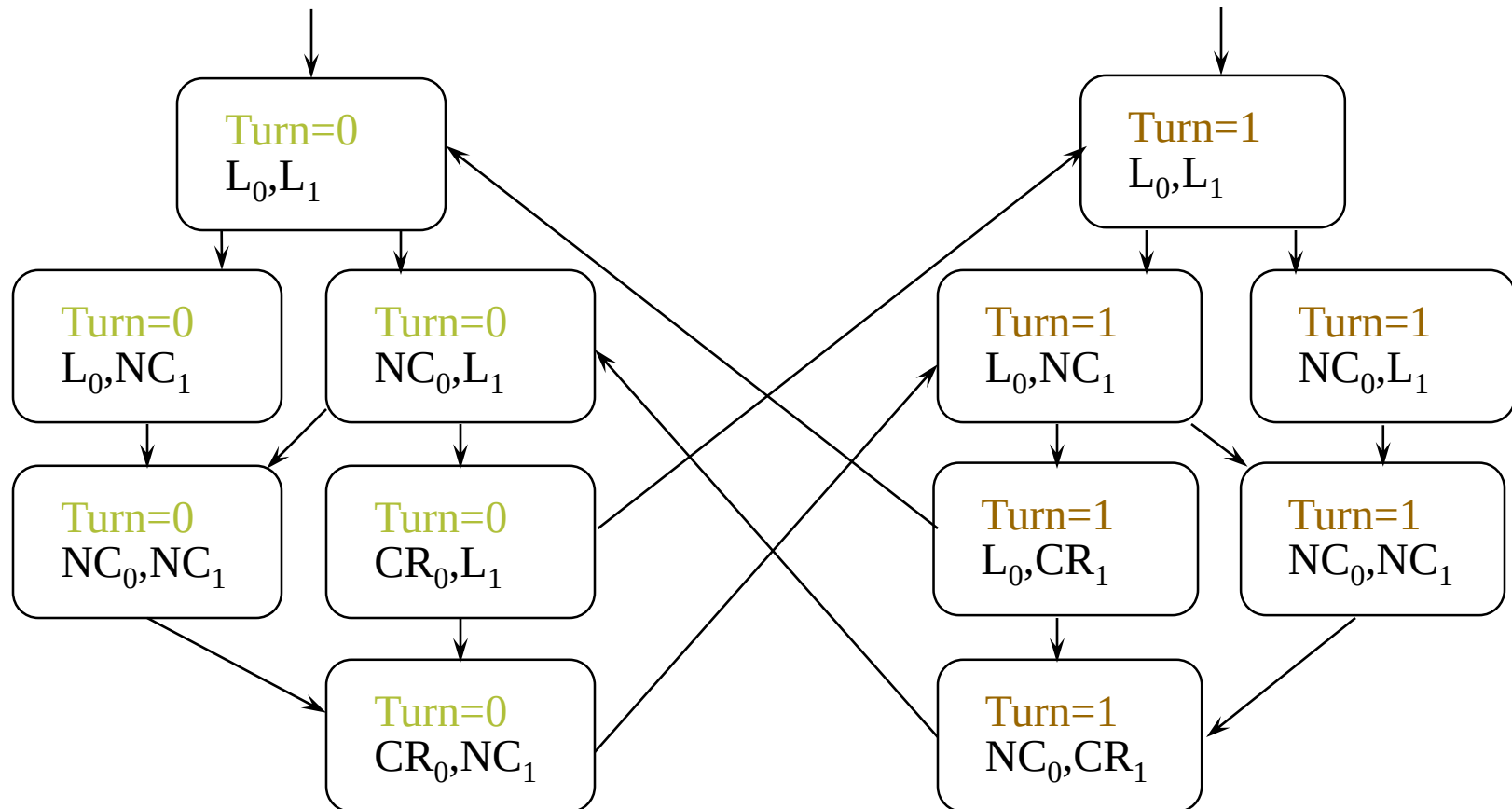
Solutions in graph theory

- strongly connected components (SCC)

Kripke structure

- inevitability analysis

$$\forall \Box (\text{Turn}=0 \rightarrow \forall \Diamond \text{Turn}=1)$$



Kripke structure

- inevitability analysis

```
inevitability( $\eta$ ) /* using greatest fixpoint algorithm */ {  
  for all  $s$ , if  $\neg\eta \in L(s)$ ,  $L(s) = L(s) \cup \{\exists\Box\neg\eta\}$ ;  
  repeat {  
    for all  $s$ , if  $\exists\Box\neg\eta \in L(s)$  and  $\forall(s,s')(\exists\Box\neg\eta \rightarrow L(s'))$ ,  
       $L(s) = L(s) - \{\exists\Box\neg\eta\}$ ;  
  } until no more changes to  $L(s)$  for any  $s$ .  
  if there is an  $s_0 \in S_0$  with  $\exists\Box\neg\eta \in L(s_0)$ ,  
    return 'inevitability not true,'  
    else return 'inevitability true.'  
}
```

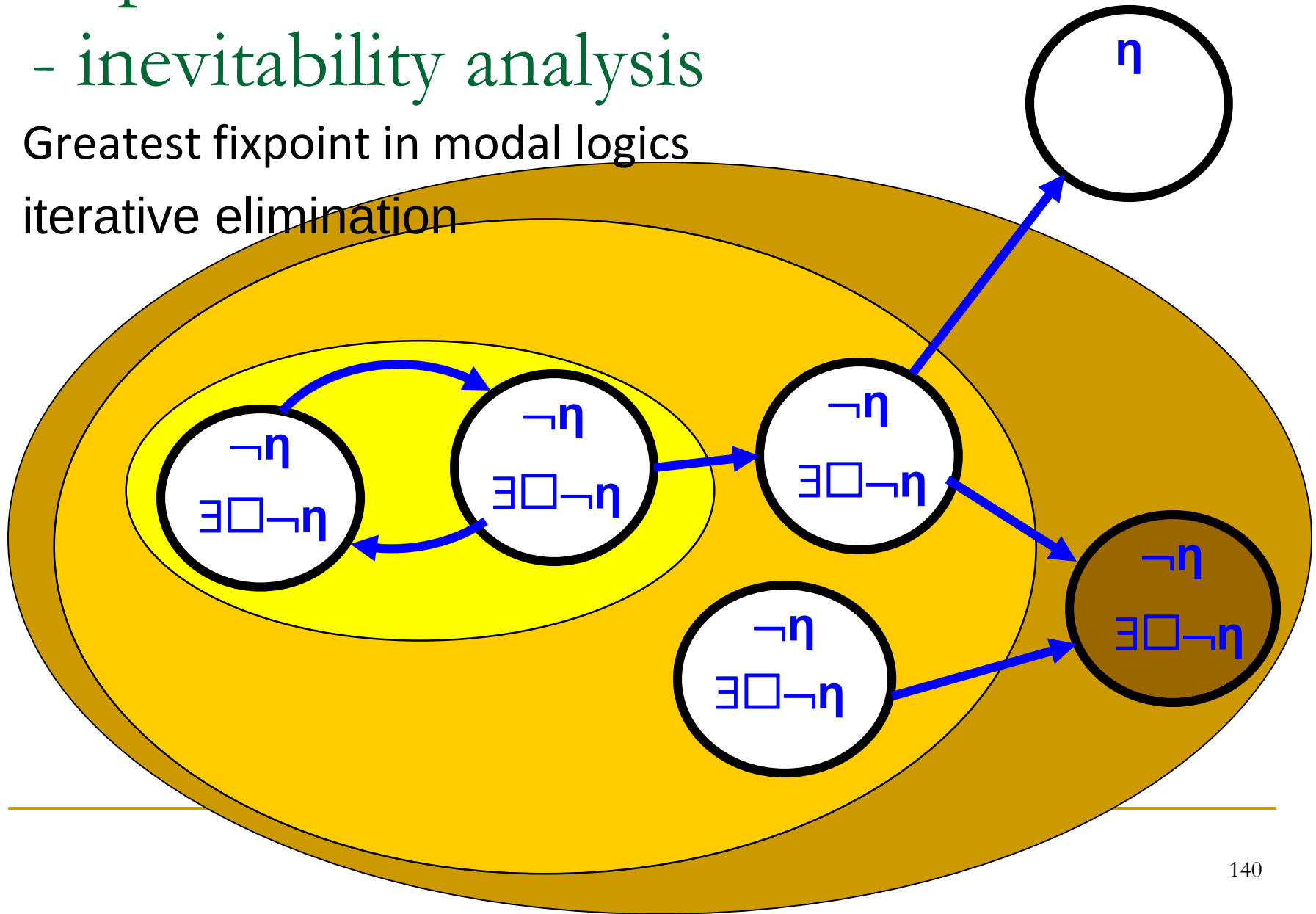
The procedure terminates since S is finite in the Kripke structure.

Kripke structure

- inevitability analysis

Greatest fixpoint in modal logics

iterative elimination



CTL model-checking

The NORMAL form needed in CTL model-checking:

1. only modal operators

$$\exists \bigcirc \varphi, \exists \psi_1 \mathbf{U} \psi_2, \exists \square \varphi$$

2. No modal operators

$$\forall \bigcirc \varphi, \forall \psi_1 \mathbf{U} \psi_2, \forall \square \varphi, \forall \diamond \varphi, \exists \diamond \varphi$$

3. No double negation: $\neg \neg \varphi$

4. No implication: $\psi_1 \Rightarrow \psi_2$

CTL

- model-checking algorithm (1 / 6)

Given M and φ ,

1. Convert φ to NORMAL form.
2. list the elements in $Cl(\varphi)$ according to their sizes

$$\varphi_0 \varphi_1 \varphi_2 \dots \varphi_n$$

for all $0 \leq i < j \leq n$, φ_j is not a subformula of φ_i

2. for $i=0$ to n ,

label (φ_i)

3. for some initial states s_0 of M , if $\varphi \models L(s_0)$, return 'No!'

4. return 'Yes!'

See
next
page!

CTL

- model-checking algorithm (2/6)

```
label( $\varphi$ ) {  
  case  $p$ , return;  
  case  $\neg\varphi$ , for all  $s$ , if  $\varphi \notin L(s)$ ,  $L(s) = L(s) \cup \{\neg\varphi\}$   
  case  $\varphi \vee \psi$ , for all  $s$ , if  $\varphi \in L(s)$  or  $\psi \in L(s)$ ,  
     $L(s) = L(s) \cup \{\varphi \vee \psi\}$   
  case  $\exists O\varphi$ , for all  $s$ , if  $\exists (s, s')$  with  $\varphi \in L(s')$ ,  
     $L(s) = L(s) \cup \{\exists O\varphi\}$   
  case  $\exists \psi_1 \mathbf{U} \psi_2$ , lfp( $\psi_1, \psi_2$ );  
  case  $\exists \Box\varphi$ , gfp( $\varphi$ );  
}
```

CTL

- model-checking algorithm (3/6)

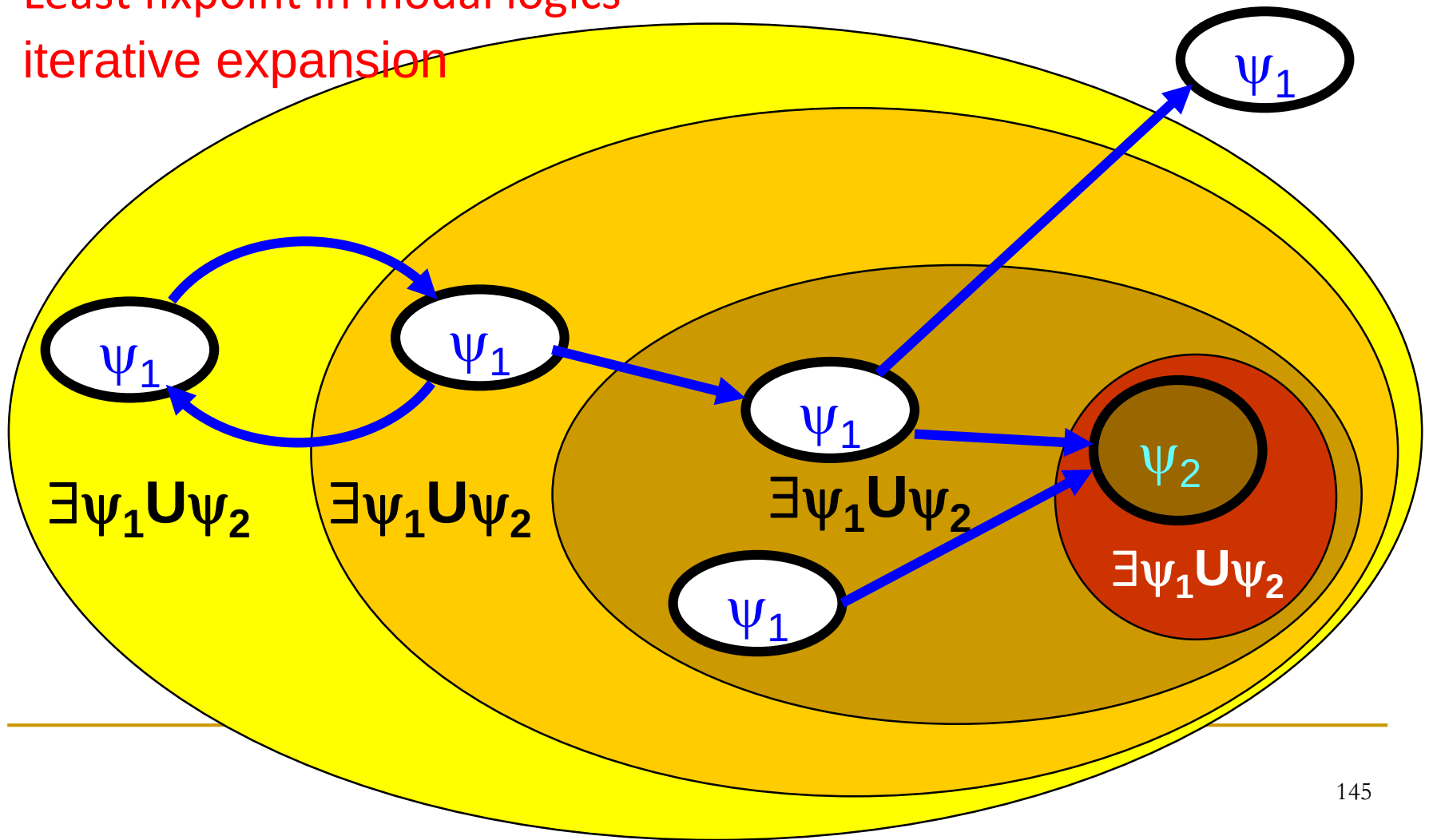
```
lfp( $\psi_1$ ,  $\psi_2$ ) /* least fixpoint algorithm */ {  
  for all s, if  $\psi_2 \in L(s)$ ,  $L(s) = L(s) \cup \{\exists \psi_1 \mathbf{U} \psi_2\}$ ;  
  repeat {  
    for all s, if  $\psi_1 \in L(s)$  and  $\exists (s, s') (\exists \psi_1 \mathbf{U} \psi_2 \in L(s'))$ ,  
       $L(s) = L(s) \cup \{\exists \psi_1 \mathbf{U} \psi_2\}$ ;  
  } until no more changes to  $L(s)$  for any s.  
}
```

The procedure terminates since S is finite in the Kripke structure.

CTL

- model-checking algorithm (4/6)

Least fixpoint in modal logics
iterative expansion



CTL

- model-checking algorithm (5/6)

```
gfp( $\psi$ ) /* greatest fixpoint algorithm */ {  
  for all s, if  $\psi \in L(s)$ ,  $L(s) = L(s) \cup \{\exists \Box \psi\}$ ;  
  repeat {  
    for all s, if  $\exists \Box \psi \in L(s)$  and  $\forall (s, s') (\exists \Box \psi \rightarrow L(s'))$ ,  
       $L(s) = L(s) - \{\exists \Box \psi\}$ ;  
  } until no more changes to  $L(s)$  for any s.  
}
```

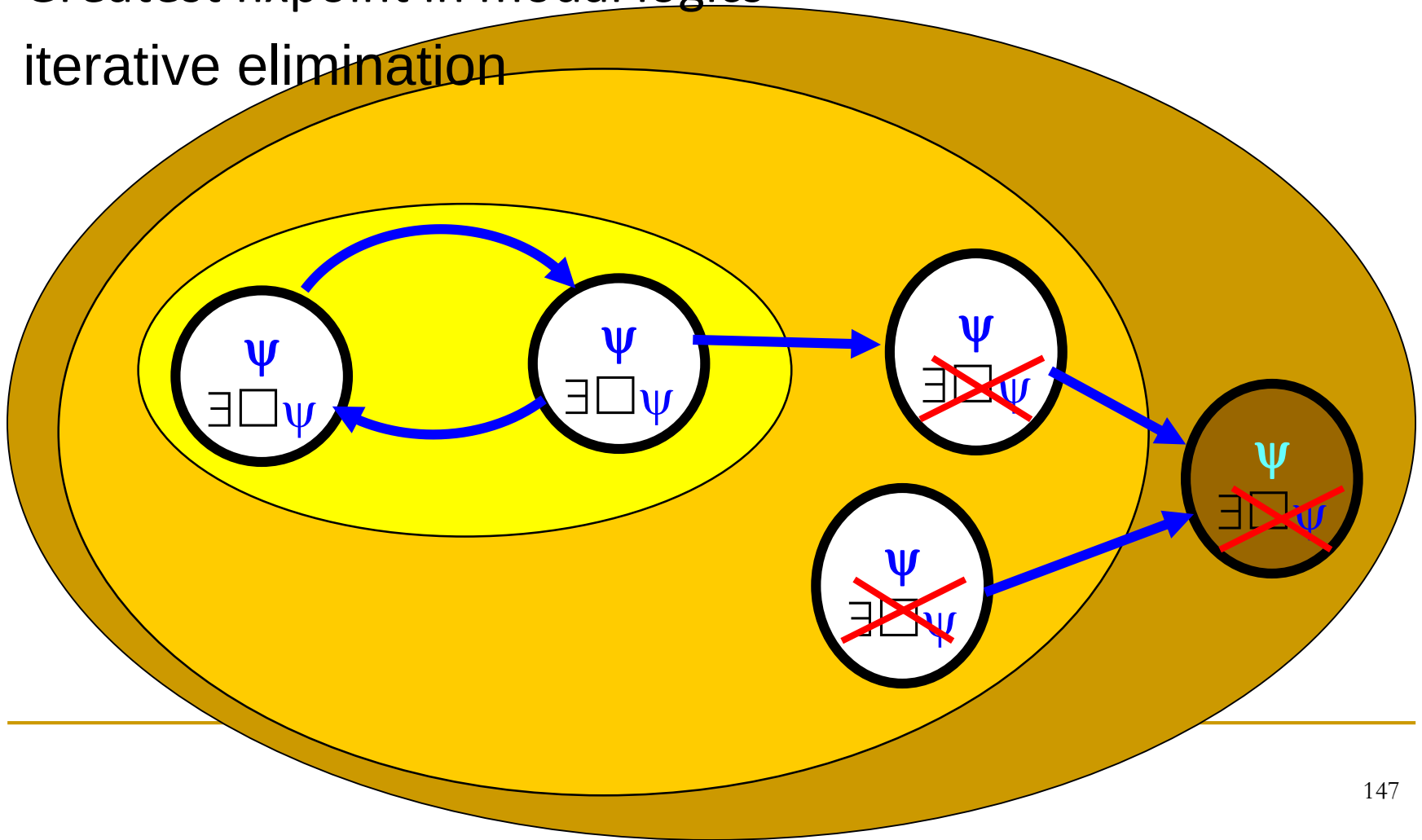
The procedure terminates since S is finite in the Kripke structure.

CTL

- model-checking algorithm (6/6)

Greatest fixpoint in modal logics

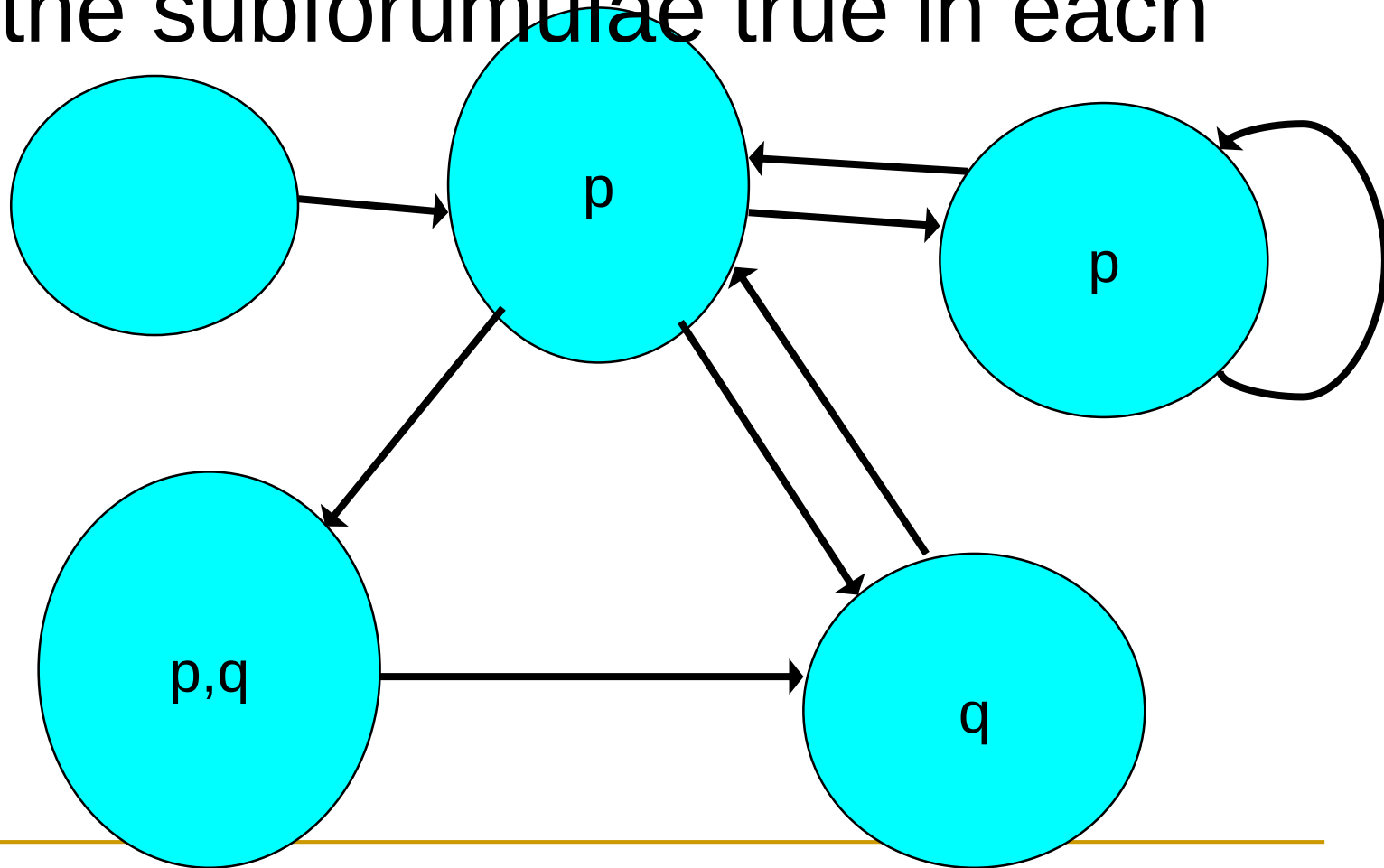
iterative elimination



$$(\exists \bigcirc \exists p U q) \wedge \exists \Box p$$

Labeling function:

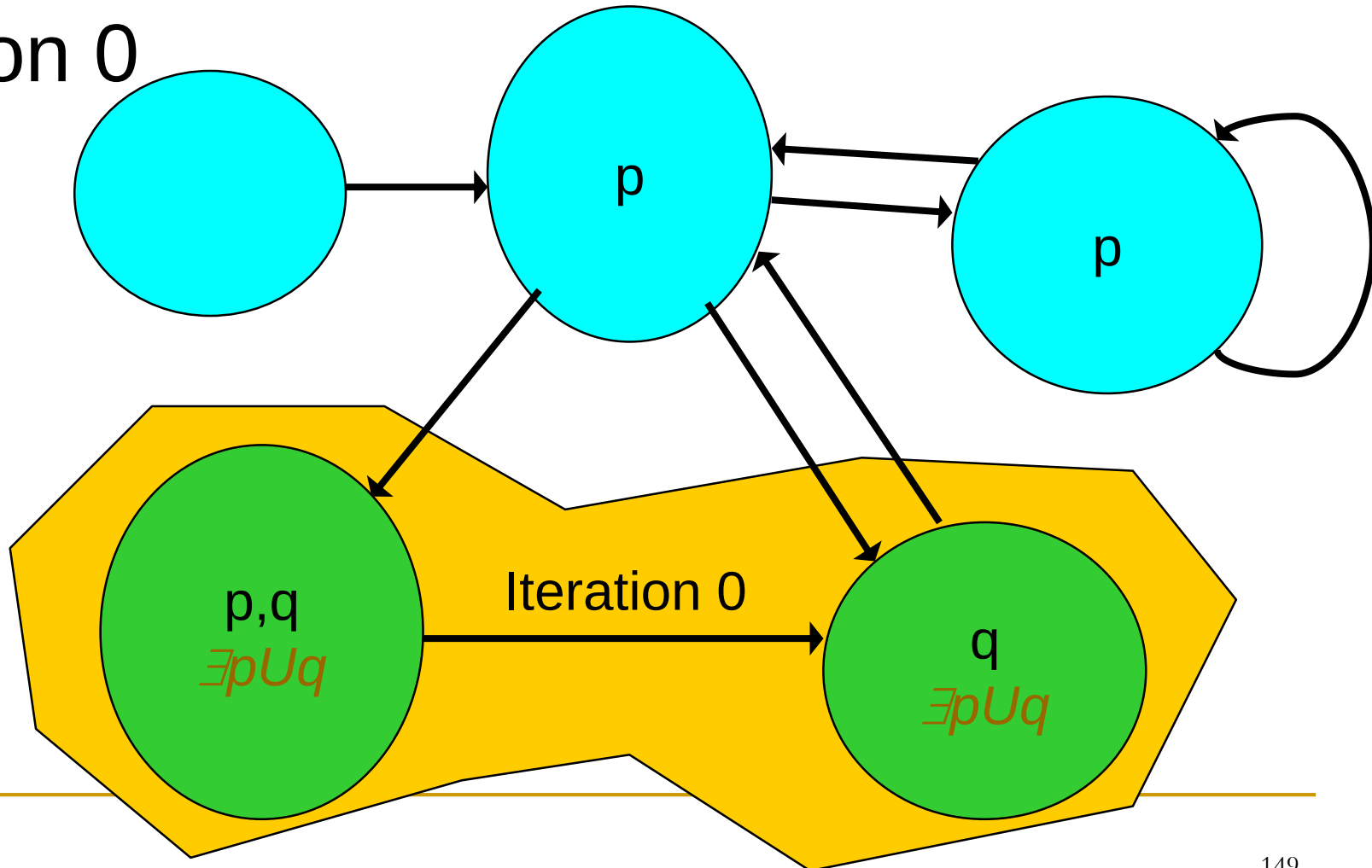
label the subformulae true in each state.



$$(\exists \bigcirc \exists p U q) \wedge \exists \square p$$

Evaluating $\exists p U q$ using least fixpoint

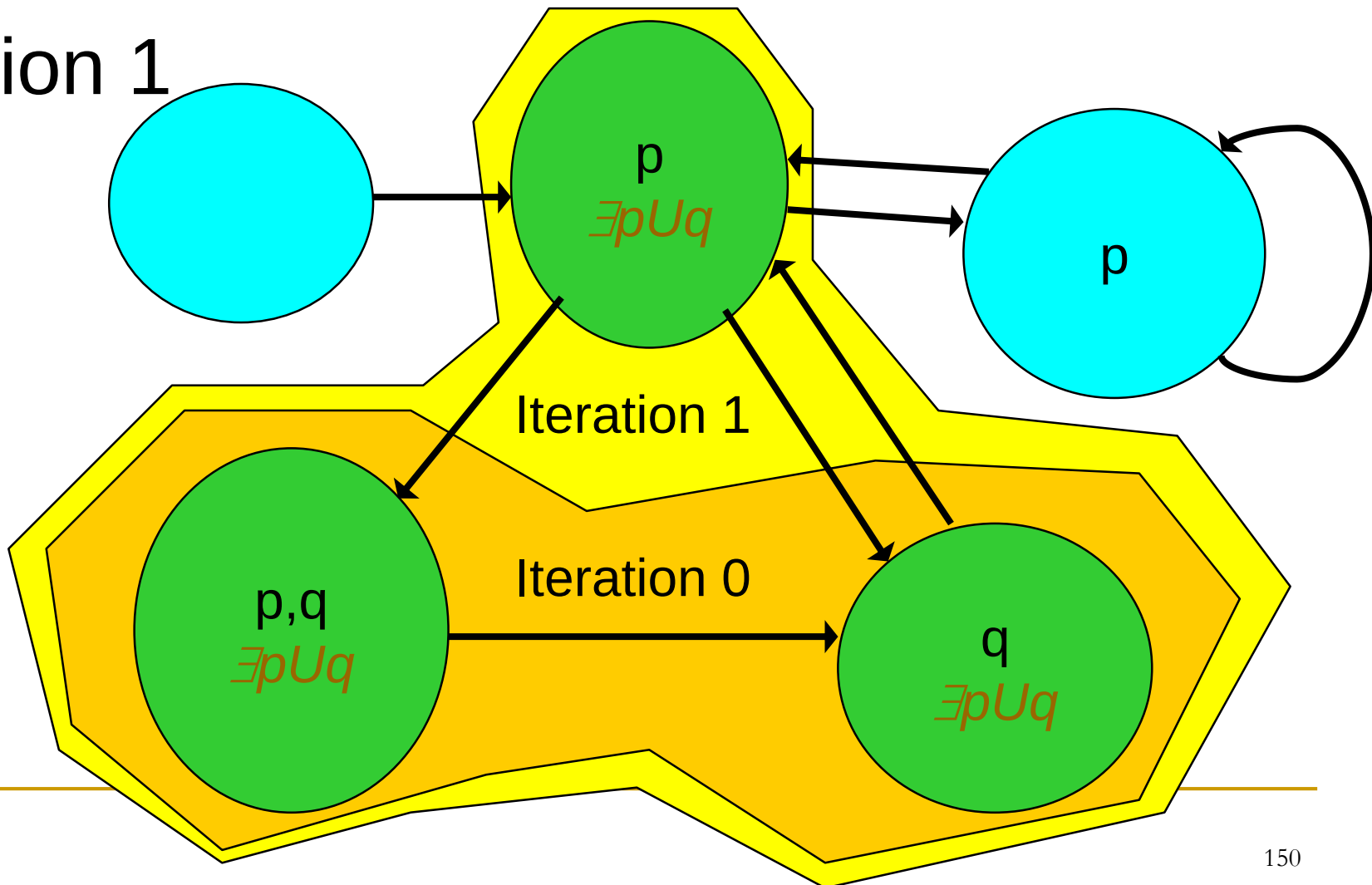
Iteration 0



$$(\exists \bigcirc \exists p U q) \wedge \exists \square p$$

Evaluating $\exists p U q$ using least fixpoint

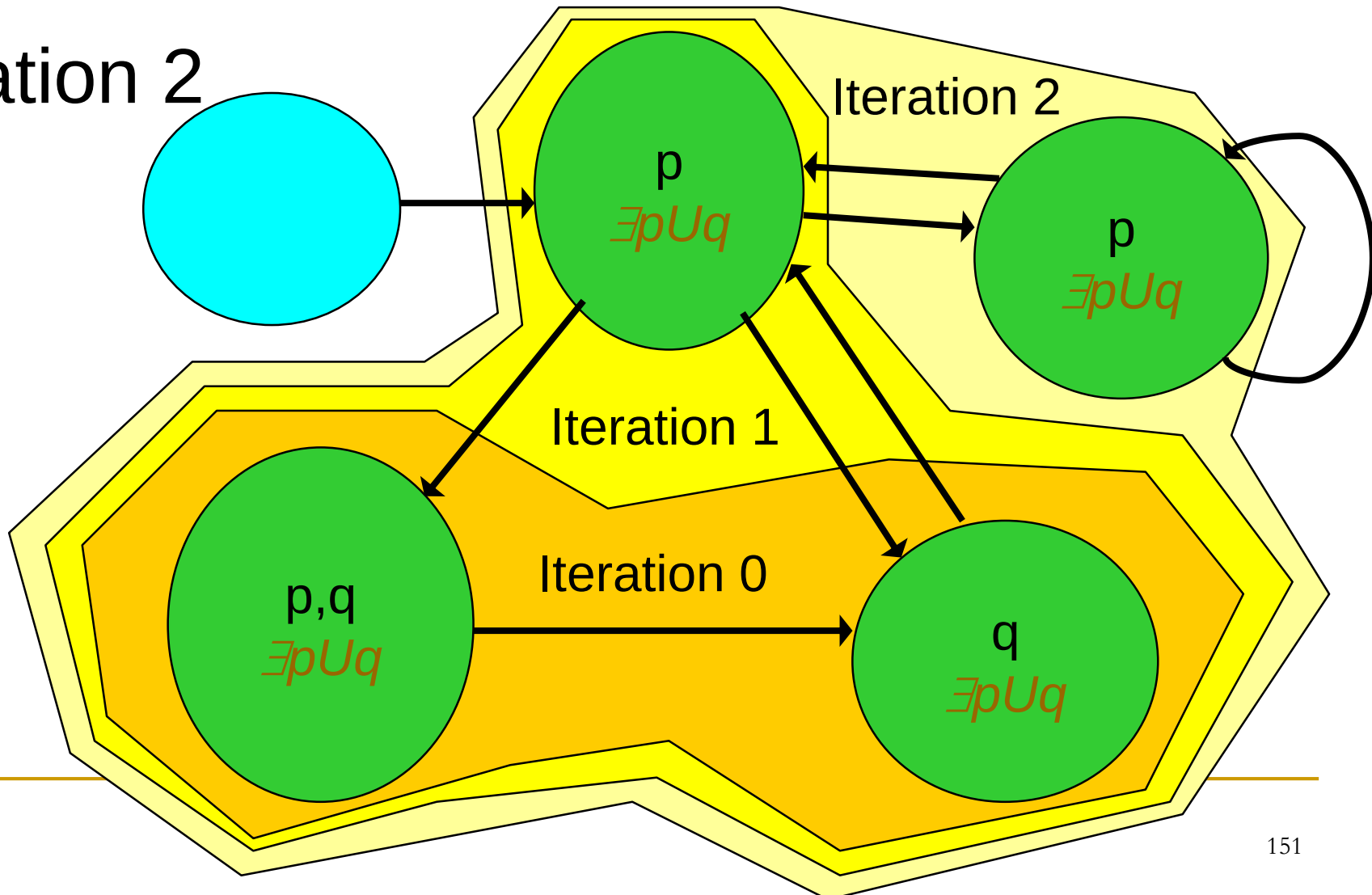
Iteration 1



$$(\exists \bigcirc \exists p U q) \wedge \exists \square p$$

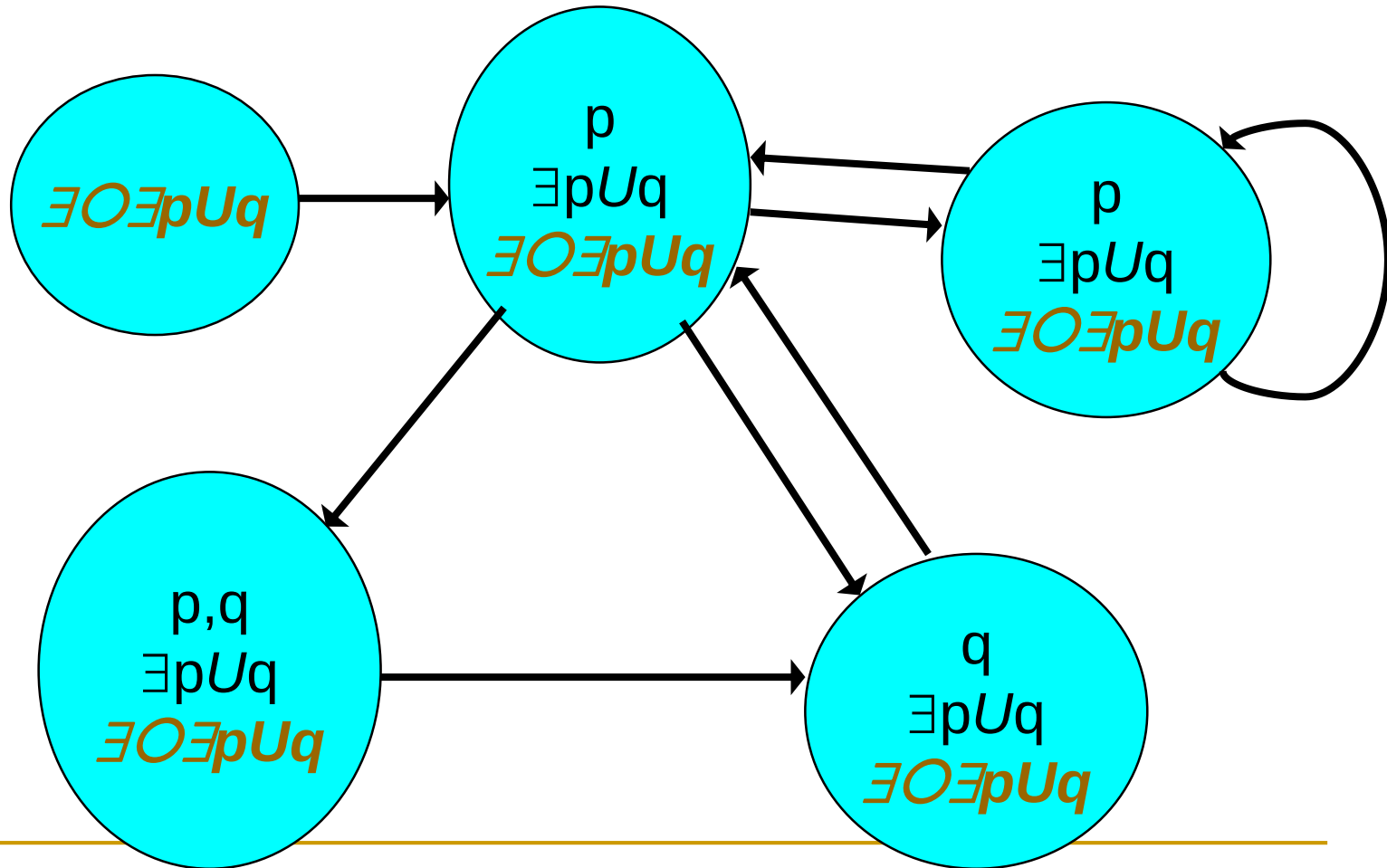
Evaluating $\exists p U q$ using least fixpoint

Iteration 2



$$(\exists O \exists p U q) \wedge \exists \Box p$$

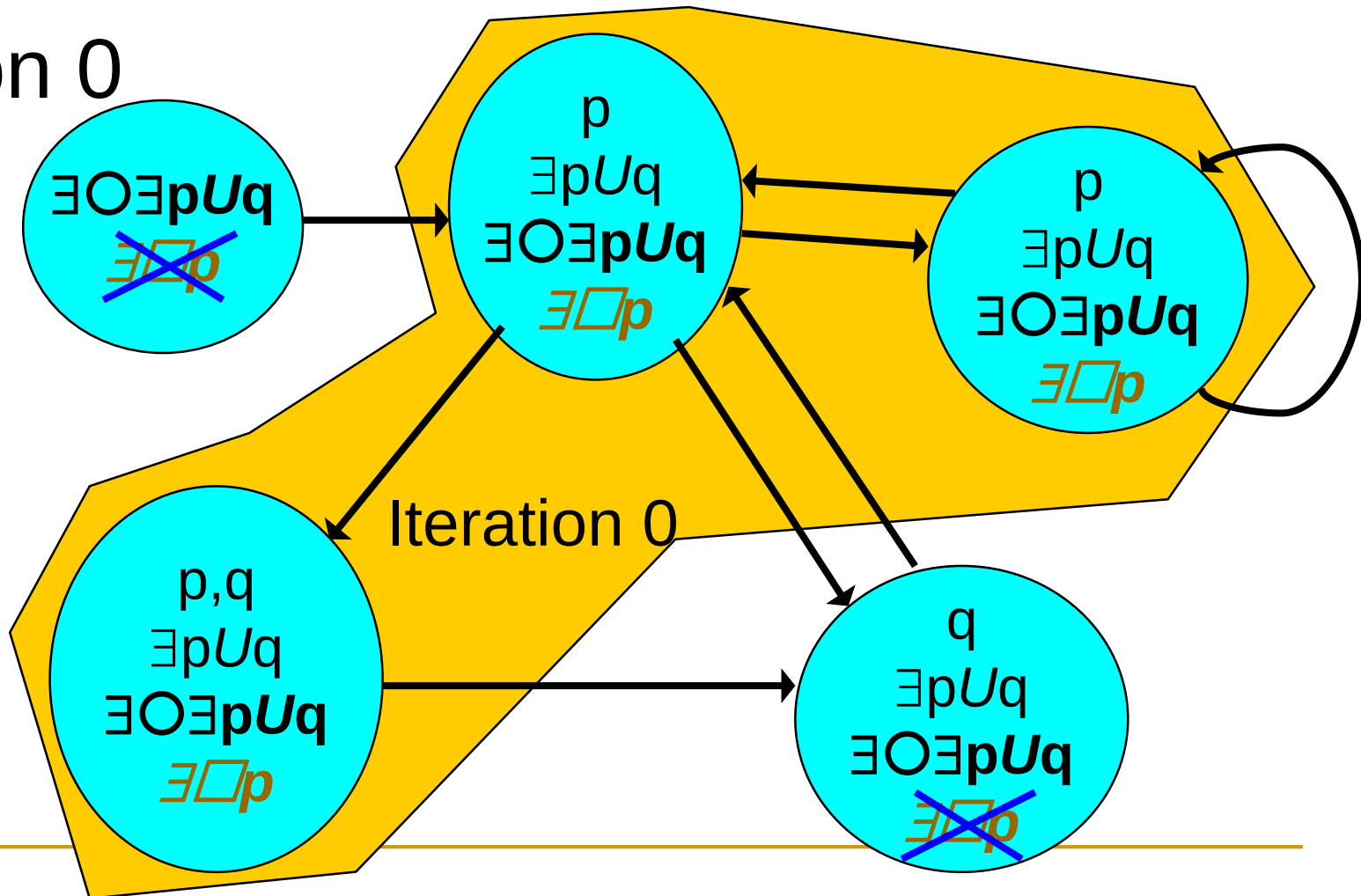
Evaluating $\exists O \exists p U q$



$$(\exists \bigcirc \exists p U q) \wedge \exists \Box p$$

Evaluating $\exists \Box p$ using greatest fixpoint

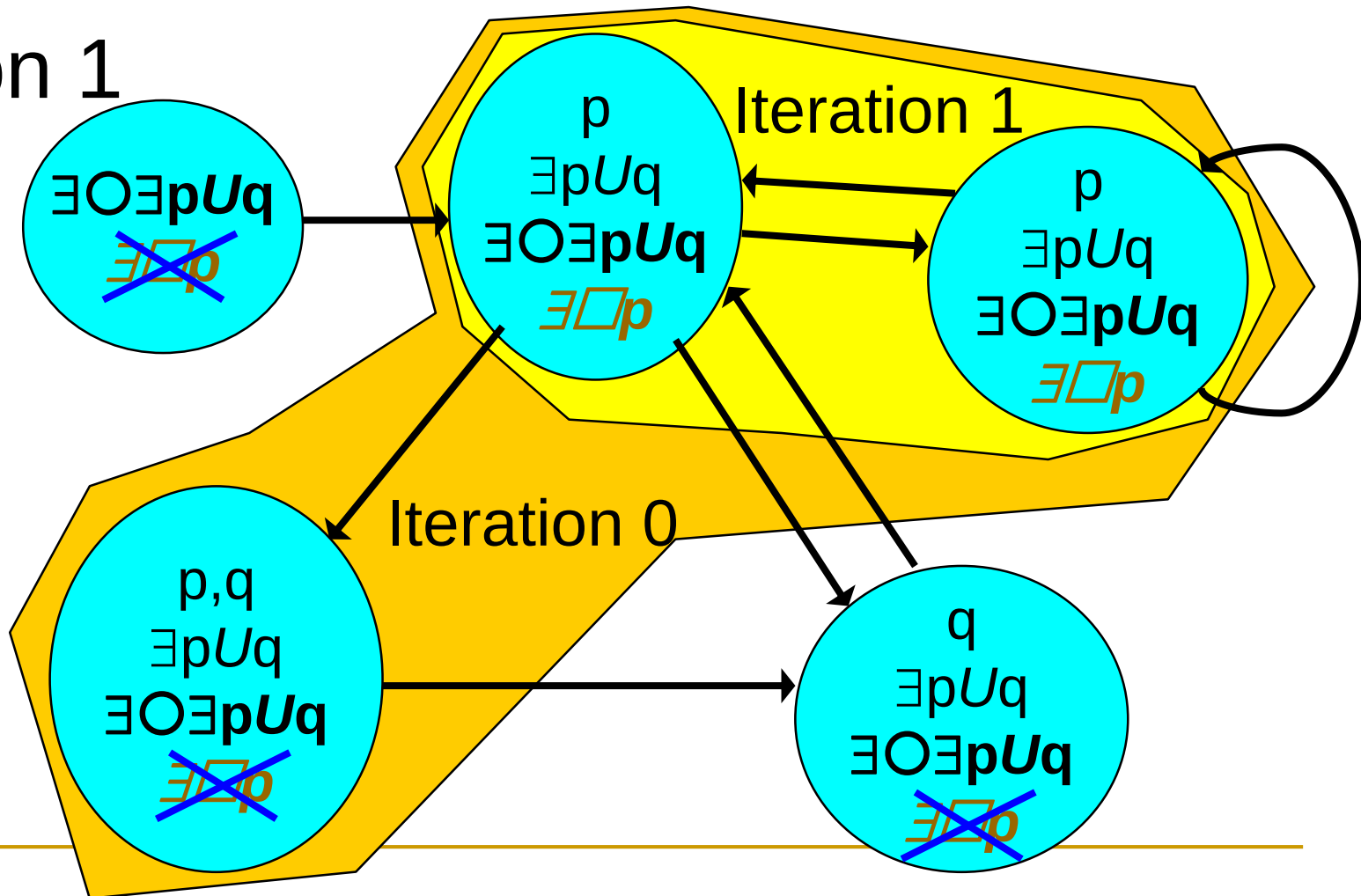
Iteration 0



$$(\exists \bigcirc \exists p U q) \wedge \exists \Box p$$

Evaluating $\exists \Box p$ using greatest fixpoint

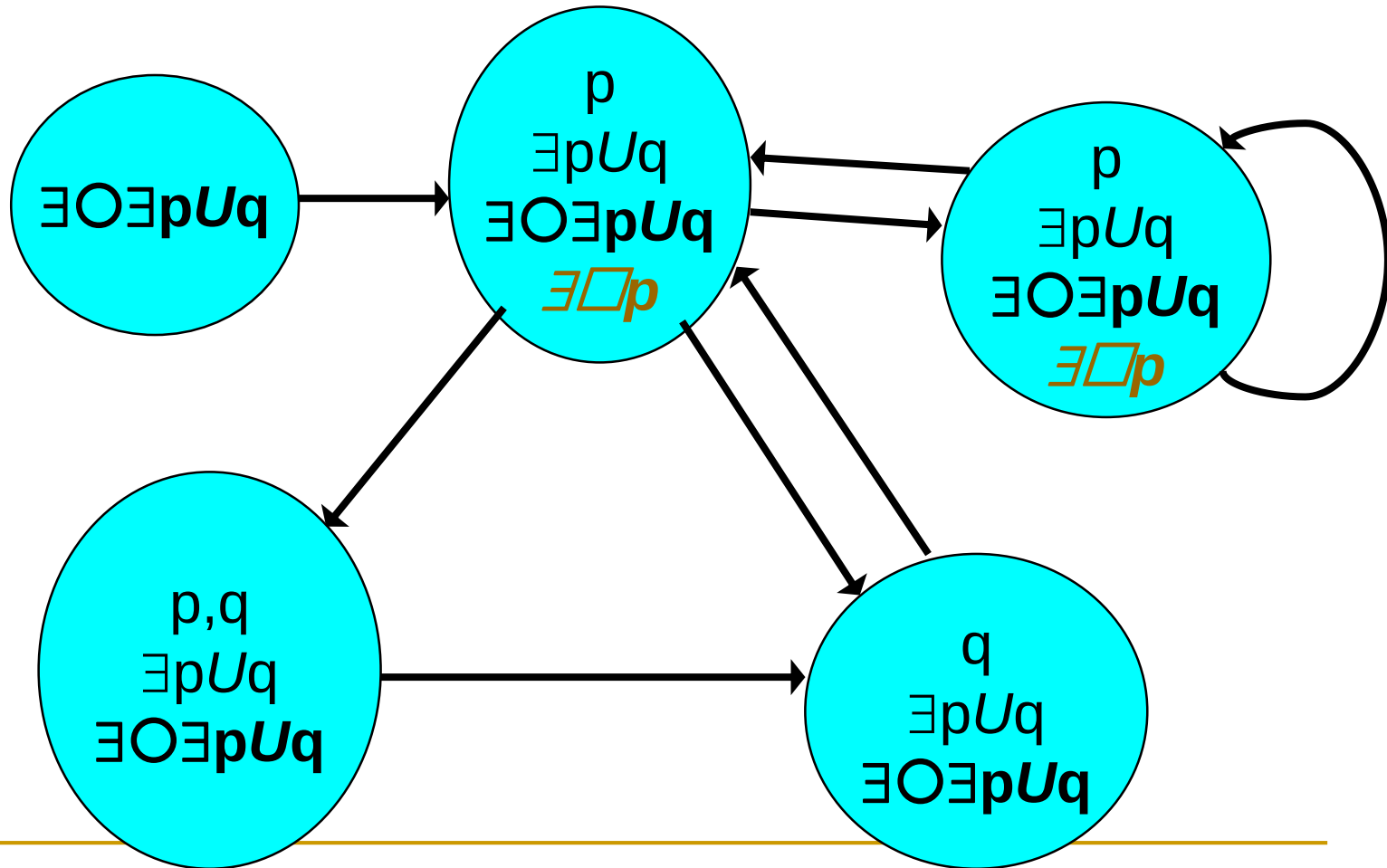
Iteration 1



$$(\exists \bigcirc \exists p U q) \wedge \exists \Box p$$

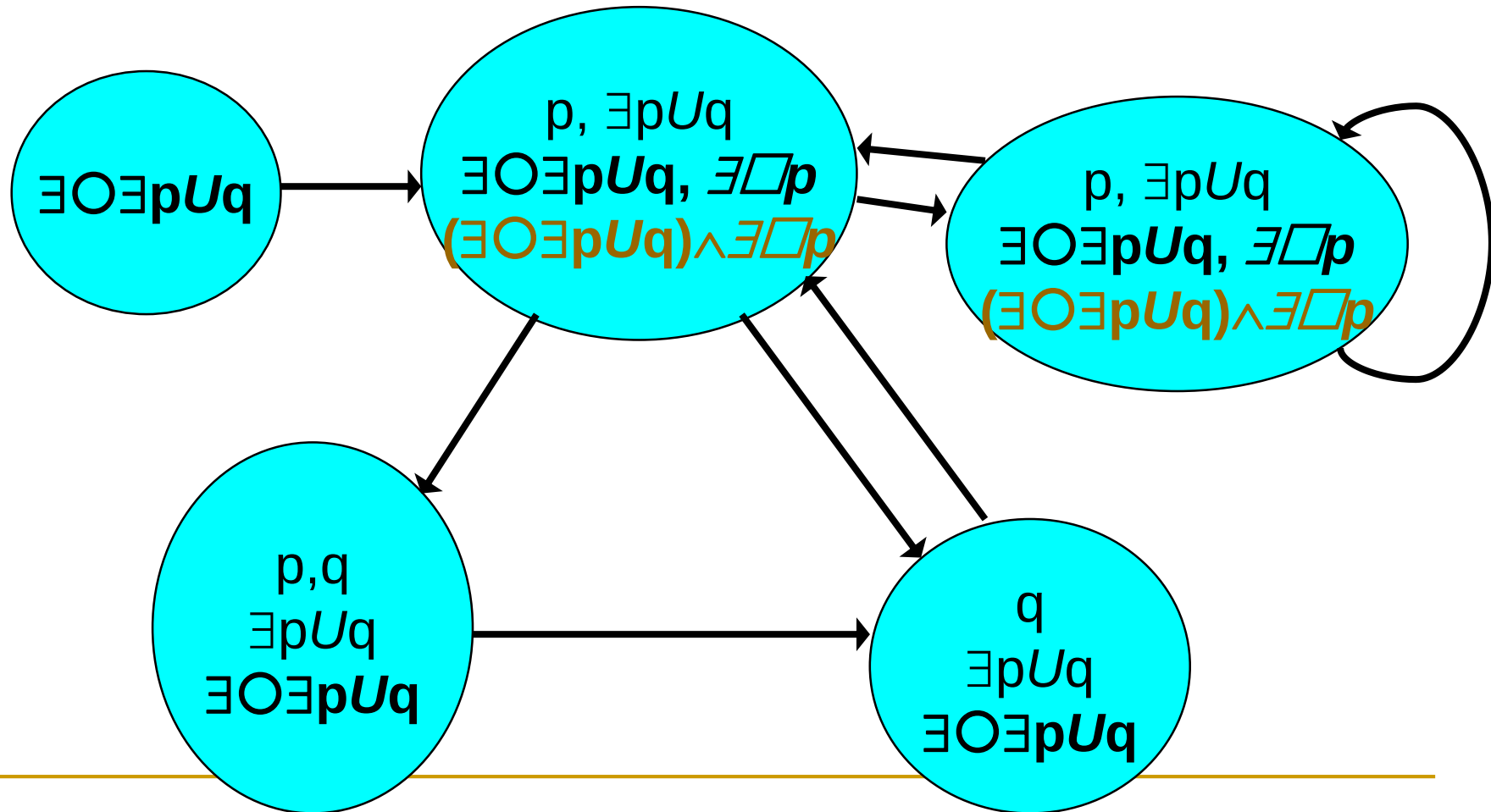
Evaluating $\exists \Box p$ using greatest fixpoint

Result:

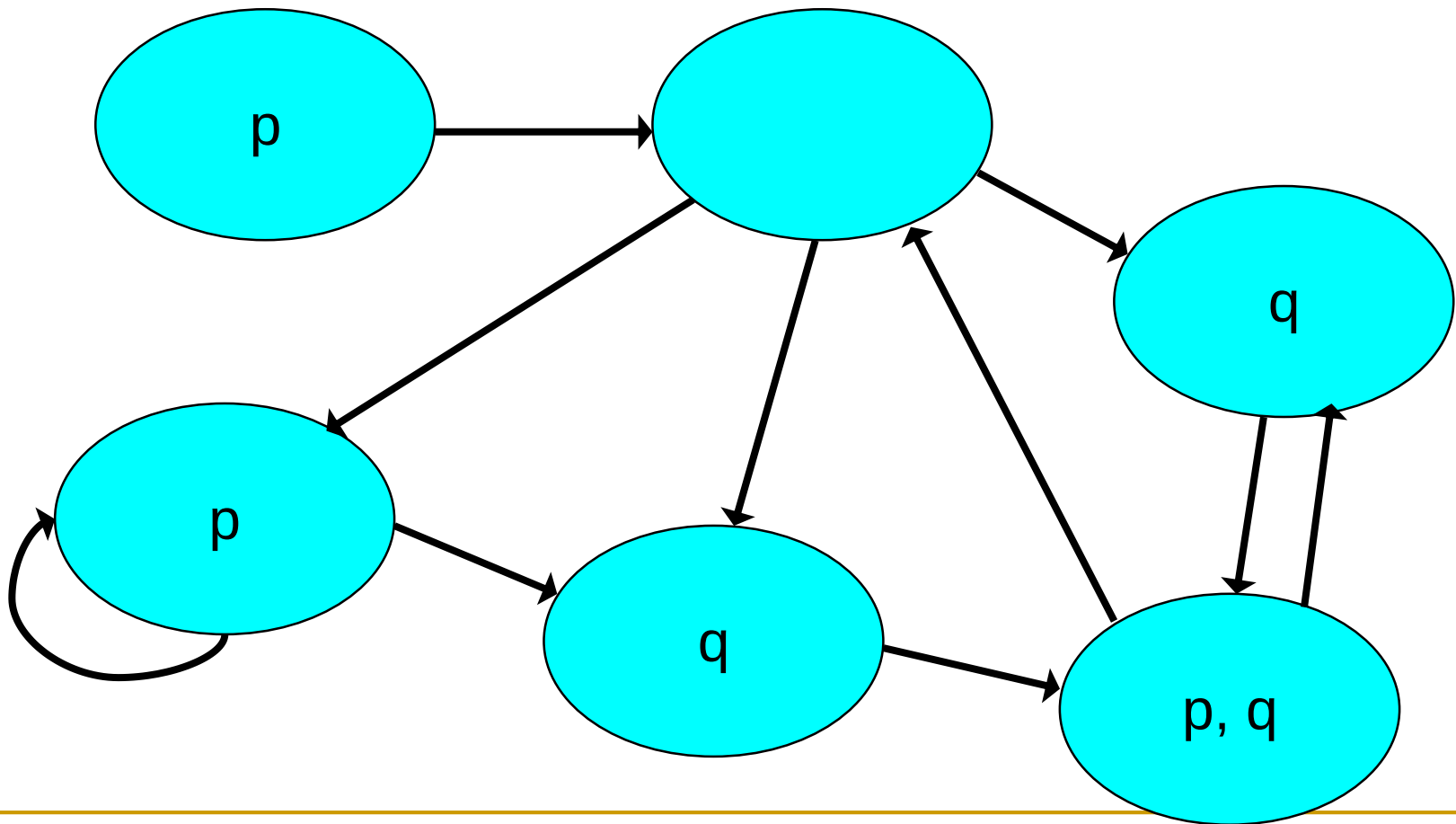


$$(\exists \bigcirc \exists p U q) \wedge \exists \Box p$$

Finally, evaluating $(\exists \bigcirc \exists p U q) \wedge \exists \Box p$



Workout: labelling $\exists \Diamond (p \wedge \exists \Box q)$



CTL

- model-checking problem complexities

- The PLTL model-checking problem is PSPACE-complete.
 - definition: Is there a run that satisfies the formula ?
- The PLTL without $\bigcirc$ (modal operator “next”) model-checking problem is NP-complete.
- The model-checking problem of CTL is PTIME-complete.
- The model-checking problem of CTL* is PSPACE-complete.

CTL

- symbolic model-checking with BDD

- System states are described with binary variables.

n binary variables $\rightarrow$ 2^n states

$x_1, x_2, \dots, x_n$

- we can use a BDD to describe legal states.

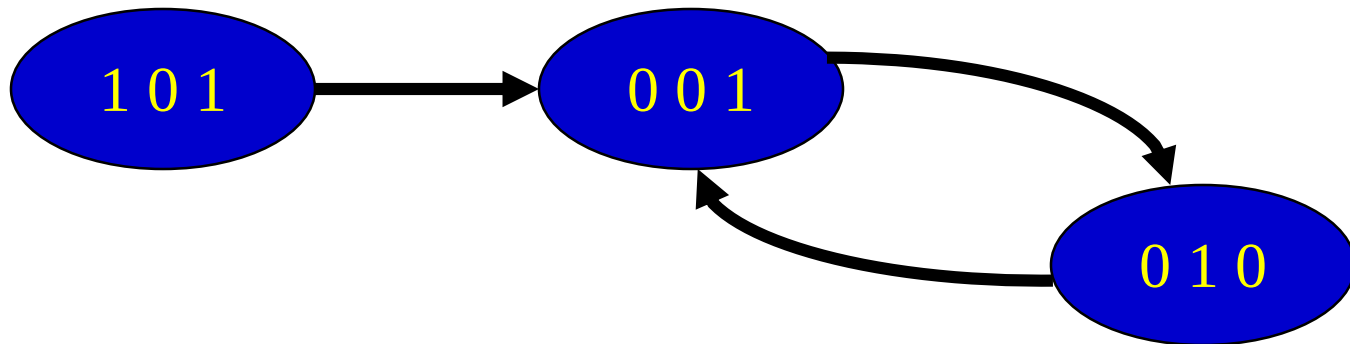
a Boolean function with n binary variables

$\text{state}(x_1, x_2, \dots, x_n)$

CTL - symbolic model-checking with Propositional logics

Example:

$x_1 \ x_2 \ x_3$

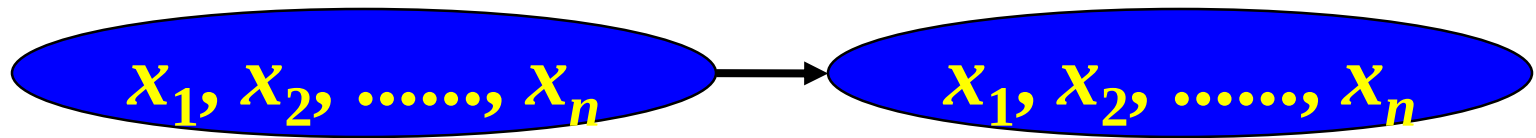


$$\begin{aligned} \text{state}(x_1, x_2, x_3) = & (x_1 \wedge \neg x_2 \wedge x_3) \\ \vee & (\neg x_1 \wedge \neg x_2 \wedge x_3) \\ \vee & (\neg x_1 \wedge x_2 \wedge \neg x_3) \end{aligned}$$

CTL - symbolic model-checking with Propositional logics

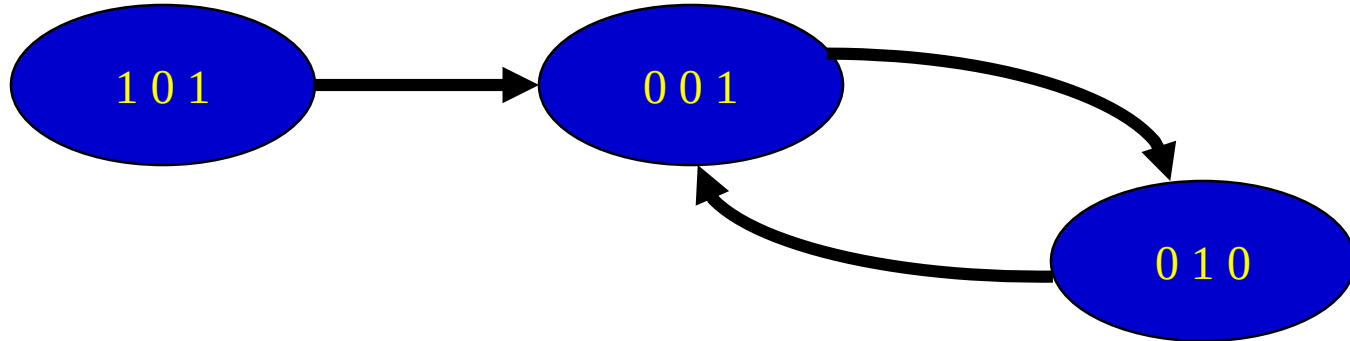
State transition relation as a logic function
with $2n$ parameters

$\text{transition}(x_1, x_2, \dots, x_n, x'_1, x'_2, \dots, x'_n)$



CTL - symbolic model-checking with Propositional logics

$x_1 \ x_2 \ x_3 \ x'_1 \ x'_2 \ x'_3$



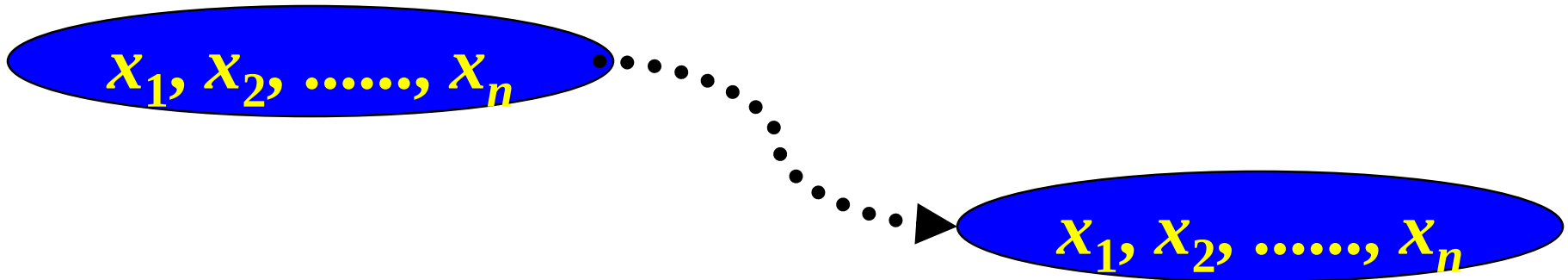
$\text{transition}(x_1, x_2, x_3, x'_1, x'_2, x'_3) =$

$$\begin{aligned}
 & (x_1 \wedge \neg x_2 \wedge x_3 \wedge \neg x'_1 \wedge \neg x'_2 \wedge x'_3) \\
 \vee & (\neg x_1 \wedge \neg x_2 \wedge x_3 \wedge \neg x'_1 \wedge x'_2 \wedge \neg x'_3) \\
 \vee & (\neg x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x'_1 \wedge \neg x'_2 \wedge x'_3)
 \end{aligned}$$

CTL - symbolic model-checking with Propositional logics

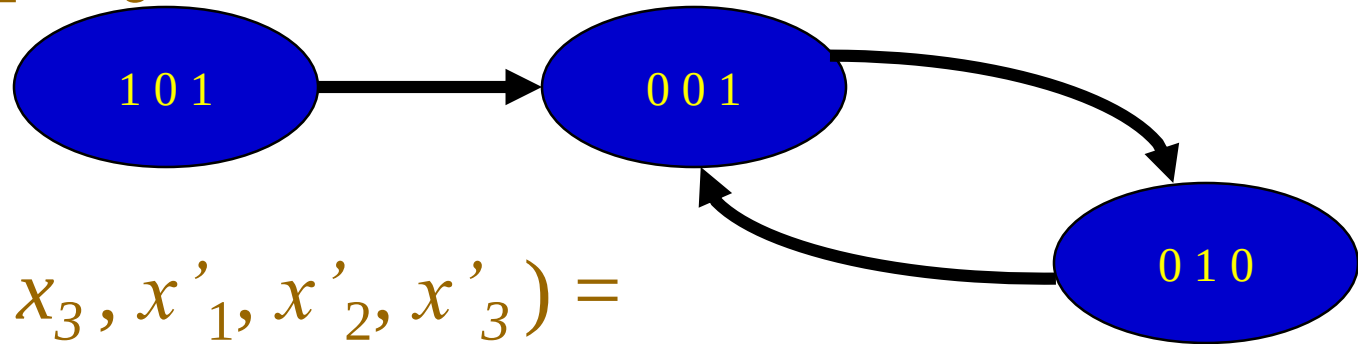
Path relation also as a logic function
with $2n$ parameters

$\text{reach}(x_1, x_2, \dots, x_n, x'_1, x'_2, \dots, x'_n)$



CTL - symbolic model-checking with Propositional logics

$x_1 \ x_2 \ x_3 \ x'_1 \ x'_2 \ x'_3$



$\text{reach}(x_1, x_2, x_3, x'_1, x'_2, x'_3) =$

$$\begin{aligned}
 & (x_1 \wedge \neg x_2 \wedge x_3 \wedge \neg x'_1 \wedge \neg x'_2 \wedge x'_3) \\
 \vee & (x_1 \wedge \neg x_2 \wedge x_3 \wedge \neg x'_1 \wedge x'_2 \wedge \neg x'_3) \\
 \vee & (\neg x_1 \wedge \neg x_2 \wedge x_3 \wedge \neg x'_1 \wedge x'_2 \wedge \neg x'_3) \\
 \vee & (\neg x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x'_1 \wedge \neg x'_2 \wedge x'_3) \\
 \vee & (\neg x_1 \wedge \neg x_2 \wedge x_3 \wedge \neg x'_1 \wedge \neg x'_2 \wedge x'_3) \\
 \hline
 \vee & (\neg x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x'_1 \wedge x'_2 \wedge \neg x'_3)
 \end{aligned}$$

Symbolic safety analysis

- I : initial condition with parameters

$X_1, X_2, \dots, X_n$

- η : safe condition with parameters

$X_1, X_2, \dots, X_n$

If $I \wedge \neg(\eta \uparrow) \wedge \text{reach}(x_1, x_2, \dots, x_n, x'_1, x'_2, \dots, x'_n)$

is not false,

- a risk state is reachable.
- *the system is not safe.*

change all
unprimed
variables in η
to primed.

Symbolic safety analysis

- construction of $\text{reach}(x_1, \dots, x_n, x'_1, \dots, x'_n)$

$$(x_1 = x'_1 \wedge \dots \wedge x_n = x'_n)$$

$$\vee \exists y_1, \dots, \exists y_n (\text{transition}(x_1, \dots, x_n, y_1, \dots, y_n) \\ \wedge \text{reach}(y_1, \dots, y_n, x'_1, \dots, x'_n))$$

$$\rightarrow \text{reach}(x_1, \dots, x_n, x'_1, \dots, x'_n)$$

This is a least fixpoint for backward analysis.

Symbolic safety analysis

- construction of $\text{reach}(x_1, \dots, x_n, x'_1, \dots, x'_n)$

$\text{transition}(x_1, \dots, x_n, x'_1, \dots, x'_n)$

$\forall \exists y_1, \dots, \exists y_n (\text{reach}(x_1, \dots, x_n, y_1, \dots, y_n)$
 $\quad \wedge \text{reach}(y_1, \dots, y_n, x'_1, \dots, x'_n)$
 $)$

$\rightarrow \text{reach}(x_1, \dots, x_n, x'_1, \dots, x'_n)$

This is *another* least fixpoint for speed-up.

Symbolic safety analysis

- construction of $\text{reach}(x_1, \dots, x_n, x'_1, \dots, x'_n)$

$\text{transition}(x_1, \dots, x_n, x'_1, \dots, x'_n)$

$\vee \exists y_1, \dots, \exists y_n (\text{reach}(x_1, \dots, x_n, y_1, \dots, y_n)$
 $\wedge \text{transition}(y_1, \dots, y_n, x'_1, \dots, x'_n)$
 $)$

$\rightarrow \text{reach}(x_1, \dots, x_n, x'_1, \dots, x'_n)$

This is *another* least fixpoint for forward analysis.

Symbolic model-checking

- based on reachability relation

Drawbacks

- Reachability relation is usually huge.
- BDD is exponential in size to the number of variables.
- $\text{State}()$ is an n -variable relation.
- $R()$ and $\text{reachable}()$ are both $2n$ -variable relations.

In fact, we do model-checking with only transition relation.

Symbolic safety analysis (backward)

Encode the states with variables $x_0, x_1, \dots, x_n$.

- the state set as a proposition formula: $s(x_0, x_1, \dots, x_n)$
- the risk state set as $r(x_0, x_1, \dots, x_n)$
- the initial state set as $i(x_0, x_1, \dots, x_n)$
- the transition set as $t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n)$

change all
unprimed
variables in b_{k-1}
to primed.

$b_0 = r(x_0, x_1, \dots, x_n) \wedge s(x_0, x_1, \dots, x_n); k = 1;$

repeat

$b_k = b_{k-1} \vee \exists x'_0 \exists x'_1 \dots \exists x'_n (t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \wedge (b_{k-1} \uparrow));$

$k = k + 1;$

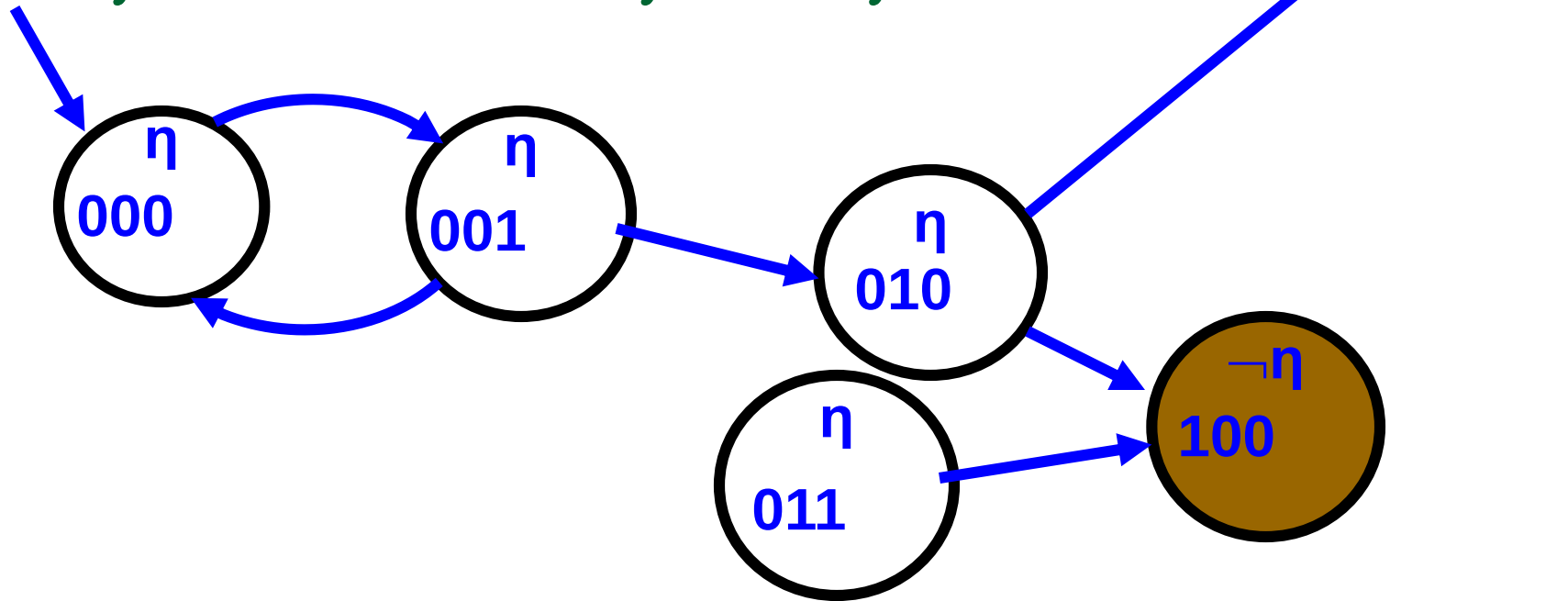
until $b_k \equiv b_{k-1};$

a least fixpoint procedure

if $(b_k \wedge i(x_0, x_1, \dots, x_n)) \equiv \text{false}$, return 'safe'; else return 'risky';

Kripke structure

- symbolic safety analysis



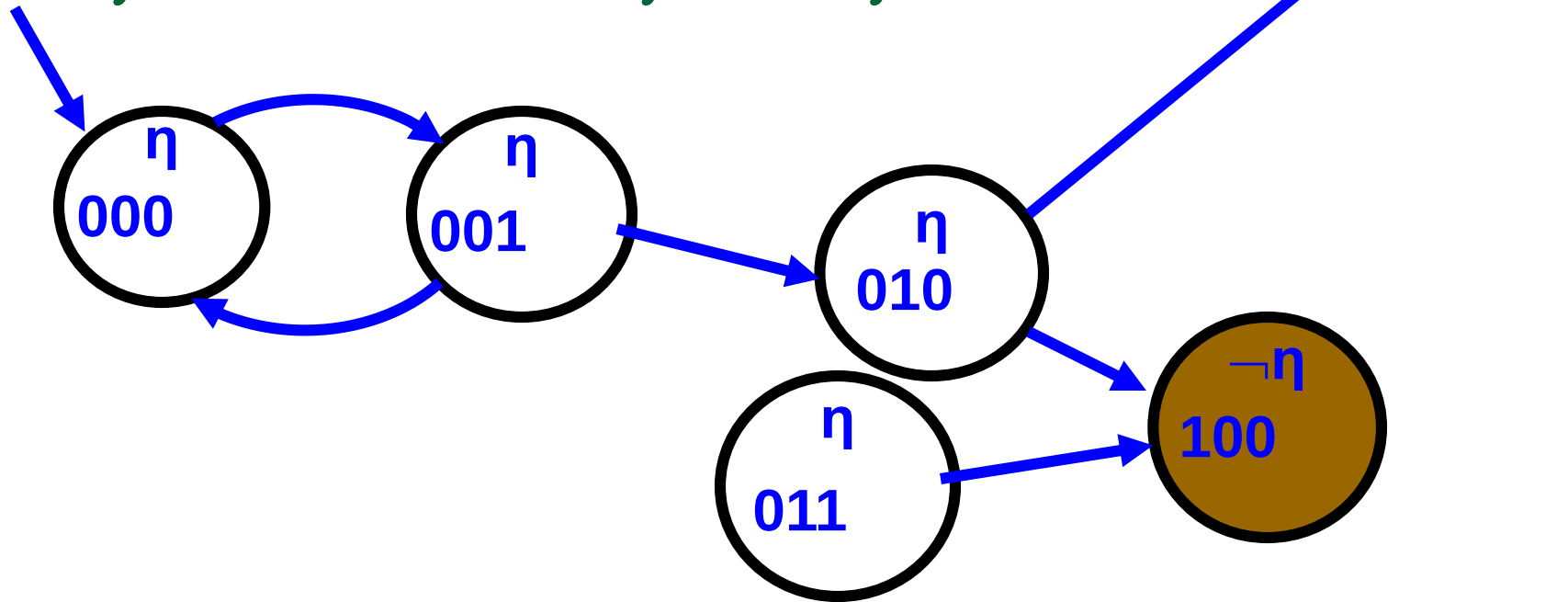
states: $s(x,y,z) \equiv (\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge y \wedge z) \vee (x \wedge \neg y \wedge \neg z) \vee (x \wedge \neg y \wedge z)$
 $\equiv (\neg x) \vee (x \wedge \neg y)$

initial state: $i(x,y,z) \equiv \neg x \wedge \neg y \wedge \neg z$

risk state: $r(x,y,z) \equiv x \wedge \neg y \wedge \neg z$

Kripke structure

- symbolic safety analysis



transitions: $T(x,y,z,x',y',z') \equiv$

$$\begin{aligned} & (\neg x \wedge \neg y \wedge \neg z \wedge \neg x' \wedge \neg y' \wedge z') \vee (\neg x \wedge \neg y \wedge z \wedge \neg x' \wedge \neg y' \wedge \neg z') \\ & \vee (\neg x \wedge \neg y \wedge z \wedge \neg x' \wedge y' \wedge \neg z') \vee (\neg x \wedge y \wedge \neg z \wedge x' \wedge \neg y' \wedge \neg z') \\ & \vee (\neg x \wedge y \wedge \neg z \wedge x' \wedge y' \wedge z') \vee (\neg x \wedge y \wedge z \wedge x' \wedge \neg y' \wedge \neg z') \end{aligned}$$

Symbolic safety analysis (backward)

$$b_0 = r(x,y,z) \equiv x \wedge \neg y \wedge \neg z$$

$$\begin{aligned} b_1 &= b_0 \vee \exists x' \exists y' \exists z' (t(x,y,z,x',y',z') \wedge b_0 \uparrow) \\ &= (x \wedge \neg y \wedge \neg z) \vee \exists x' \exists y' \exists z' (t(x,y,z,x',y',z') \wedge x' \wedge \neg y' \wedge \neg z') \\ &= (x \wedge \neg y \wedge \neg z) \vee \exists x' \exists y' \exists z' (((\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge y \wedge z)) \wedge x' \wedge \neg y' \wedge \neg z') \\ &= (x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge y \wedge z) \end{aligned}$$

$$\begin{aligned} b_2 &= b_1 \vee \exists x' \exists y' \exists z' (t(x,y,z,x',y',z') \wedge b_1 \uparrow) \\ &= (\neg x \wedge \neg y \wedge z) \vee (x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge y \wedge z) \end{aligned}$$

$$\begin{aligned} b_3 &= b_2 \vee \exists x' \exists y' \exists z' (t(x,y,z,x',y',z') \wedge b_2 \uparrow) \\ &= (\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z) \vee (x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge y \wedge z) \end{aligned}$$

$$\begin{aligned} b_4 &= b_3 \vee \exists x' \exists y' \exists z' (t(x,y,z,x',y',z') \wedge b_3 \uparrow) \\ &= (\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z) \vee (x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge y \wedge z) \end{aligned}$$

$$b_4 \wedge i(x,y,z) = (\neg x \wedge \neg y \wedge \neg z)$$

fixpoint

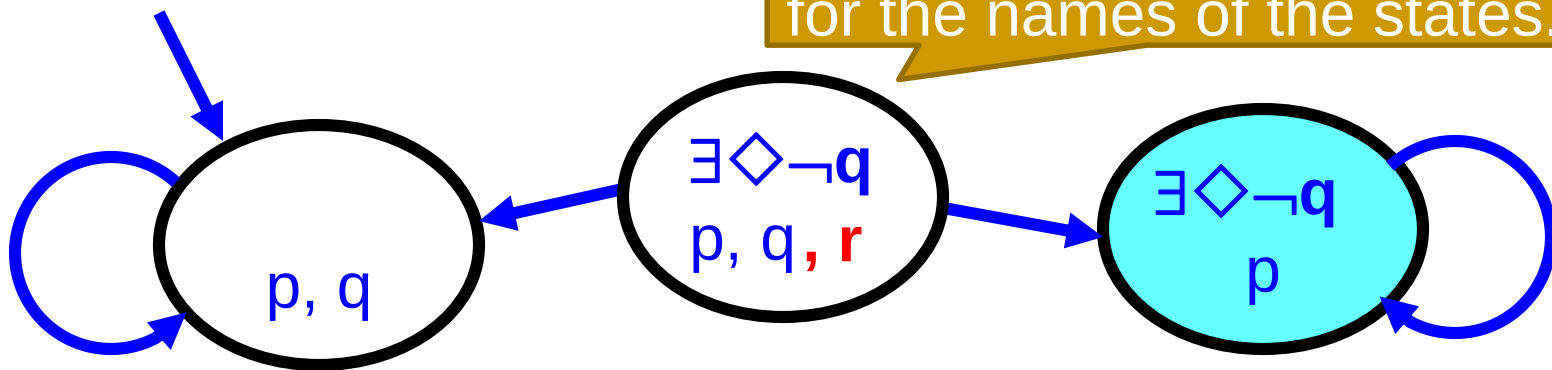
non-empty intersection
with the initial condition
→ risk detected.

Symbolic safety analysis (backward)

One assumption for the correctness!

- Two states cannot be with the same proposition labeling.
- Otherwise, the collapsing of the states may cause problem.

may need a few propositions for the names of the states.



Symbolic safety analysis (forward)

Encode the states with variables $x_0, x_1, \dots, x_n$.

- the state set as a proposition formula: $s(x_0, x_1, \dots, x_n)$
- the risk state set as $r(x_0, x_1, \dots, x_n)$
- the initial state set as $i(x_0, x_1, \dots, x_n)$
- the transition set as $t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n)$

change all
primed
variable to
unprimed.

$f_0 = i(x_0, x_1, \dots, x_n) \wedge s(x_0, x_1, \dots, x_n); k = 1;$

repeat

$f_k = f_{k-1} \vee (\exists x_0 \exists x_1 \dots \exists x_n (t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \wedge f_{k-1})) \downarrow;$

$k = k + 1;$

until $f_k \equiv f_{k-1};$

if $(f_k \wedge r(x_0, x_1, \dots, x_n)) \equiv \text{false}$, return 'safe'; else return 'risky';

Symbolic safety analysis (forward)

$$f_0 = i(x,y,z) \equiv \neg x \wedge \neg y \wedge \neg z$$

$$\begin{aligned} f_1 &= f_0 \vee (\exists x \exists y \exists z (t(x,y,z,x',y',z') \wedge f_0)) \downarrow \\ &= (\neg x \wedge \neg y \wedge \neg z) \vee (\exists x \exists y \exists z (t(x,y,z,x',y',z') \wedge \neg x \wedge \neg y \wedge \neg z)) \downarrow \\ &= (\neg x \wedge \neg y \wedge \neg z) \vee (\exists x \exists y \exists z (\neg x' \wedge \neg y' \wedge z' \wedge \neg x \wedge \neg y \wedge \neg z)) \downarrow \\ &= (\neg x \wedge \neg y \wedge \neg z) \vee (\neg x' \wedge \neg y' \wedge z') \downarrow \\ &= (\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z) = \neg x \wedge \neg y \end{aligned}$$

$$f_2 = f_1 \vee (\exists x \exists y \exists z (t(x,y,z,x',y',z') \wedge f_1)) \downarrow = (\neg x \wedge \neg y) \vee (\neg x \wedge y \wedge \neg z)$$

$$f_3 = f_2 \vee (\exists x \exists y \exists z (t(x,y,z,x',y',z') \wedge f_2)) \downarrow = (\neg y) \vee (\neg x \wedge y \wedge \neg z)$$

$$f_4 = f_3 \vee (\exists x \exists y \exists z (t(x,y,z,x',y',z') \wedge f_3)) \downarrow = (\neg y) \vee (\neg x \wedge y \wedge \neg z)$$

$$f_4 \wedge r(x,y,z) = ((\neg y) \vee (\neg x \wedge y \wedge \neg z)) \wedge (x \wedge \neg y \wedge \neg z) = (x \wedge \neg y \wedge \neg z)$$

fixpoint

non-empty intersection
with the risk condition
→ risk detected.

Bounded model-checking

The value
of x_n at
state k .

Encode the states with variables $x_{0,k}, x_{1,k}, \dots, x_{n,k}$.

- the state set as a proposition formula: $s(x_{0,k}, x_{1,k}, \dots, x_{n,k})$
- the risk state set as $r(x_{0,k}, x_{1,k}, \dots, x_{n,k})$
- the initial state set as $i(x_{0,0}, x_{1,0}, \dots, x_{n,0})$
- the transition set as $t(x_{0,k-1}, x_{1,k-1}, \dots, x_{n,k-1}, x_{0,k}, x_{1,k}, \dots, x_{n,k})$

$f_0 = i(x_{0,0}, x_{1,0}, \dots, x_{n,0}) \wedge s(x_{0,0}, x_{1,0}, \dots, x_{n,0}); k = 1;$

repeat

$f_k = t(x_{0,k-1}, x_{1,k-1}, \dots, x_{n,k-1}, x_{0,k}, x_{1,k}, \dots, x_{n,k}) \wedge f_{k-1};$

$k = k + 1;$

until $f_k \wedge r(x_{0,k}, x_{1,k}, \dots, x_{n,k}) \neq \text{false}$

When to stop ?

1. diameter of the state graph
2. explosion up to tens of steps.

Bounded model-checking

$$f_0 = i(x, y, z) \equiv \neg x_0 \wedge \neg y_0 \wedge \neg z_0$$

$$f_1 = t(x_0, y_0, z_0, x_1, y_1, z_1) \wedge f_0 = \neg x_0 \wedge \neg y_0 \wedge \neg z_0 \wedge \neg x_1 \wedge \neg y_1 \wedge z_1$$

$$\begin{aligned} f_2 &= t(x_1, y_1, z_1, x_2, y_2, z_2) \wedge f_1 \\ &= \neg x_0 \wedge \neg y_0 \wedge \neg z_0 \wedge \neg x_1 \wedge \neg y_1 \wedge z_1 \wedge ((\neg x_2 \wedge \neg y_2 \wedge \neg z_2) \vee (\neg x_2 \wedge y_2 \wedge \neg z_2)) \end{aligned}$$

$$\begin{aligned} f_3 &= t(x_2, y_2, z_2, x_3, y_3, z_3) \wedge f_2 \\ &= \neg x_0 \wedge \neg y_0 \wedge \neg z_0 \wedge \neg x_1 \wedge \neg y_1 \wedge z_1 \\ &\quad \wedge ((\neg x_2 \wedge \neg y_2 \wedge \neg z_2 \wedge \neg x_3 \wedge \neg y_3 \wedge z_3) \\ &\quad \vee (\neg x_2 \wedge y_2 \wedge \neg z_2 \wedge ((x_3 \wedge \neg y_3 \wedge \neg z_3) \vee (x_3 \wedge \neg y_3 \wedge z_3))) \\ &\quad) \end{aligned}$$

$$\begin{aligned} &= \neg x_0 \wedge \neg y_0 \wedge \neg z_0 \wedge \neg x_1 \wedge \neg y_1 \wedge z_1 \\ &\quad \wedge ((\neg x_2 \wedge \neg y_2 \wedge \neg z_2 \wedge \neg x_3 \wedge \neg y_3 \wedge z_3) \vee (\neg x_2 \wedge y_2 \wedge \neg z_2 \wedge x_3 \wedge \neg y_3)) \end{aligned}$$

$$f_3 \wedge r(x_3, y_3, z_3) = (x_3 \wedge \neg y_3 \wedge \neg z_3)$$

Symbolic inevitability analysis

Encode the states with variables $x_0, x_1, \dots, x_n$.

- the state set as a proposition formula: $s(x_0, x_1, \dots, x_n)$
- the negated inevitability state set as $b(x_0, x_1, \dots, x_n)$
- the initial state set as $i(x_0, x_1, \dots, x_n)$
- the transition set as $t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n)$

change all
unprimed
variable in b_{k-1}
to primed.

$b_0 = b(x_0, x_1, \dots, x_n) \wedge s(x_0, x_1, \dots, x_n); k = 1;$

repeat

$b_k = b_{k-1} \wedge \exists x'_0 \exists x'_1 \dots \exists x'_n (t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \wedge b_{k-1} \uparrow);$

$k = k + 1;$

until $b_k \equiv b_{k-1};$

if $(b_k \wedge i(x_0, x_1, \dots, x_n)) \equiv \text{false}$, return 'inevitable';

else return 'not inevitable';

Symbolic inevitability analysis

Encode the states with variables $x_0, x_1, \dots, x_n$.

- the state set as a proposition formula: $s(x_0, x_1, \dots, x_n)$
- the negated inevitability state set as $b(x_0, x_1, \dots, x_n)$
- the initial state set as $i(x_0, x_1, \dots, x_n)$
- the transition set as $t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n)$

change all
unprimed
variable in b_{k-1}
to primed.

$b_0 = b(x_0, x_1, \dots, x_n) \wedge s(x_0, x_1, \dots, x_n); k = 1;$

repeat

$b_k = b_{k-1} \wedge \neg \forall x'_0 \forall x'_1 \dots \forall x'_n \neg (t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \wedge b_{k-1} \uparrow);$

$k = k + 1;$

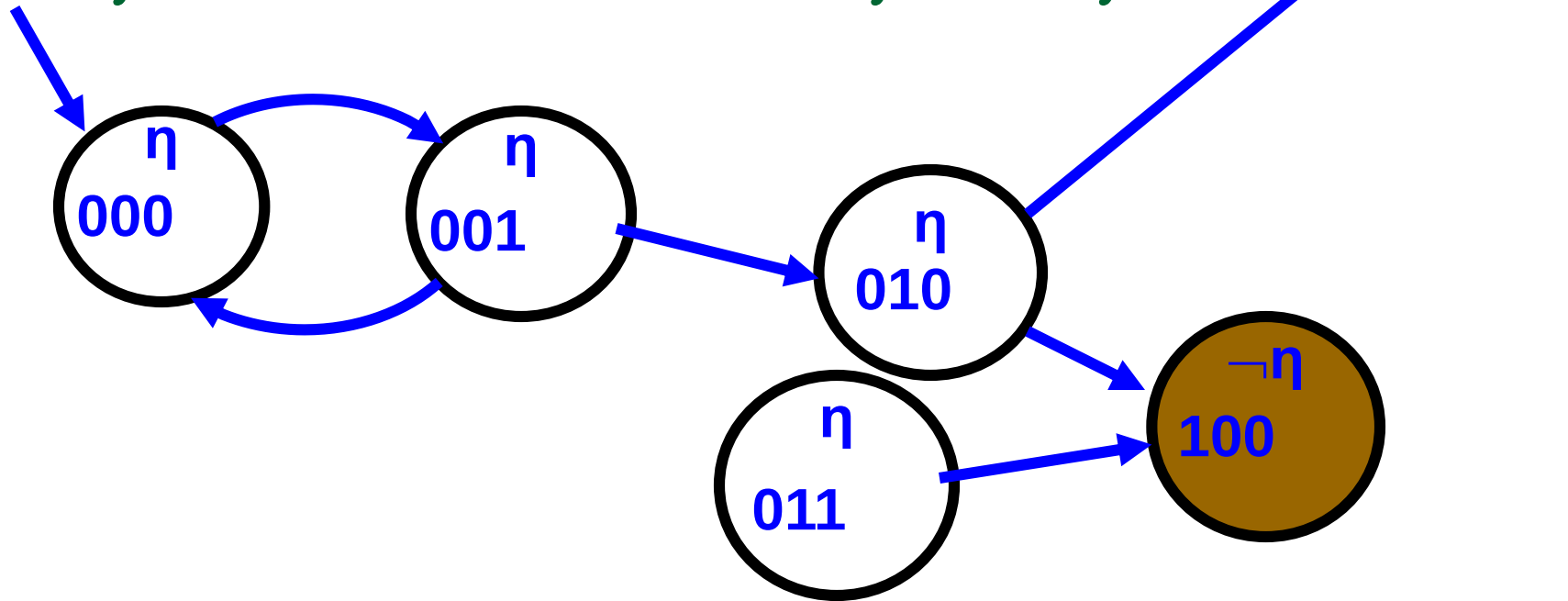
until $b_k \equiv b_{k-1};$

if $(b_k \wedge i(x_0, x_1, \dots, x_n)) \equiv \text{false}$, return 'inevitable';

else return 'not inevitable';

Kripke structure

- symbolic inevitability analysis



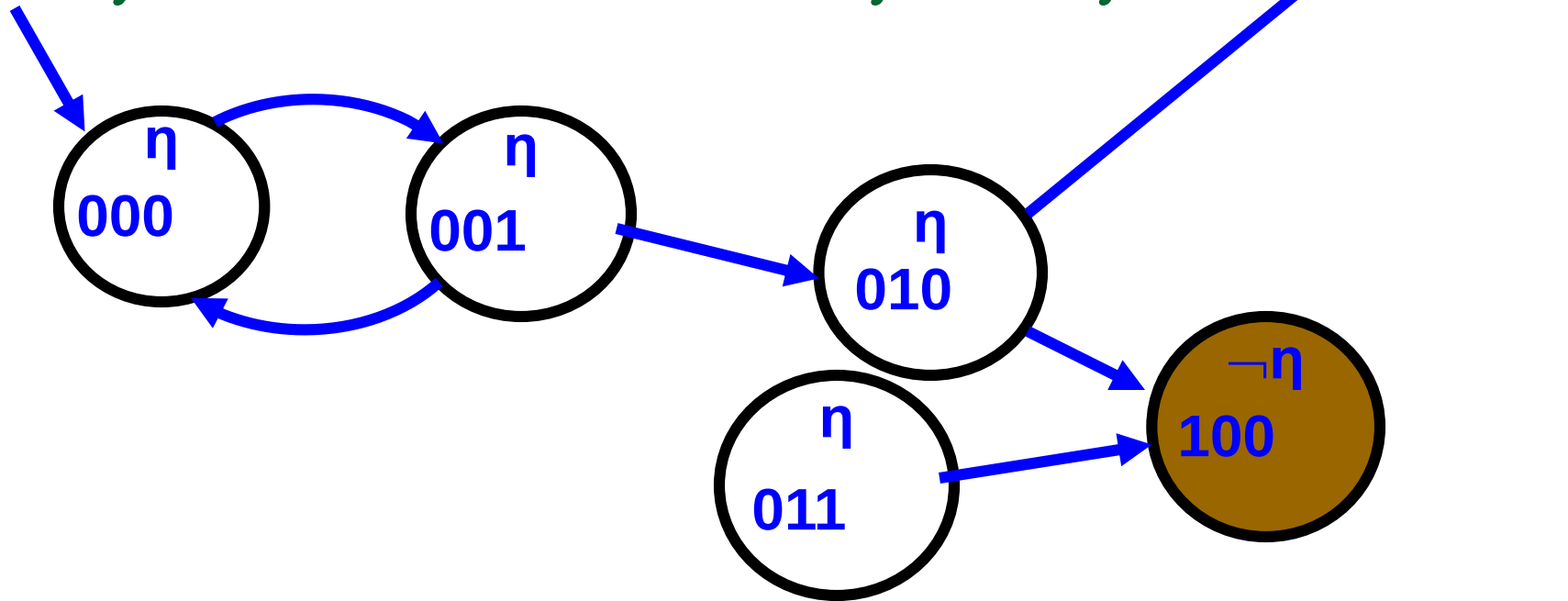
states: $s(x,y,z) \equiv (\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge y \wedge z) \vee (x \wedge \neg y \wedge \neg z) \vee (x \wedge \neg y \wedge z)$
 $\equiv (\neg x) \vee (x \wedge \neg y)$

initial state: $i(x,y,z) \equiv \neg x \wedge \neg y \wedge \neg z$

non-liveness state: $b(x,y,z) \equiv (\neg x) \vee (x \wedge \neg y \wedge z)$

Kripke structure

- symbolic inevitability analysis



transitions: $T(x,y,z,x',y',z') \equiv$

$$\begin{aligned} & (\neg x \wedge \neg y \wedge \neg z \wedge \neg x' \wedge \neg y' \wedge z') \vee (\neg x \wedge \neg y \wedge z \wedge \neg x' \wedge \neg y' \wedge \neg z') \\ & \vee (\neg x \wedge \neg y \wedge z \wedge \neg x' \wedge y' \wedge \neg z') \vee (\neg x \wedge y \wedge \neg z \wedge x' \wedge \neg y' \wedge \neg z') \\ & \vee (\neg x \wedge y \wedge \neg z \wedge x' \wedge y' \wedge z') \vee (\neg x \wedge y \wedge z \wedge x' \wedge \neg y' \wedge \neg z') \end{aligned}$$

Symbolic inevitability analysis

$$b0 = b(x,y,z) \equiv (\neg x) \vee (x \wedge \neg y \wedge z)$$

$$\begin{aligned} b1 &= b0 \wedge \exists x' \exists y' \exists z' (T(x,y,z,x',y',z') \wedge b0') \\ &= ((\neg x) \vee (x \wedge \neg y \wedge z)) \\ &\quad \wedge \exists x' \exists y' \exists z' (T(x,y,z,x',y',z') \wedge ((\neg x') \vee (x' \wedge \neg y' \wedge z'))) \\ &= ((\neg x) \vee (x \wedge \neg y \wedge z)) \wedge \\ &\quad \exists x' \exists y' \exists z' (((\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z)) \\ &\quad \wedge ((\neg x') \vee (x' \wedge \neg y' \wedge z'))) \\ &= (\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z) \end{aligned}$$

$$b2 = b1 \wedge \exists x' \exists y' \exists z' (T(x,y,z,x',y',z') \wedge b1')$$

$$= (\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z)$$

$$b3 = b2 \wedge \exists x' \exists y' \exists z' (T(x,y,z,x',y',z') \wedge b2')$$

$$= (\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z)$$

fixpoint

**non-empty
intersection with
the initial condition
→ non-inevitability
detected.**

Symbolic model-checking

- with real-world programs

Consider transition rules

Guard $\rightarrow$ Actions

- Guard is a propositional formula of state variables.
- Actions is a command of the following syntax.

$$C ::= \text{ACT} \mid \{C\} \mid C \ C \mid \textit{if} (B) \ C \ \textit{else} \ C \mid \textit{while} (B) \ C$$
$$\text{ACT} ::= ; \mid \textit{goto} \ \textit{name}; \mid x = E ;$$

Transition rules from programs

guarded commands

1: $w = 0;$		$(pc==1) \rightarrow w = 0; pc=2;$
2: $x = 0;$		$(pc==2) \rightarrow x = 0; pc=3;$
3: $y = z * z;$		$(pc==3) \rightarrow y = z * z; pc=4;$
4: $\text{while } (x < y) \{$		$(pc==4 \&\& x \geq y) \rightarrow pc=8;$
5: $w = w + x * z;$		$(pc==4 \&\& x < y) \rightarrow pc=5;$
6: $x = x + 1;$		$(pc==5) \rightarrow w = w + x * z; pc=6;$
7: $\}$		$(pc==6) \rightarrow x = x + 1; pc=4;$
8: $\text{if } (w > z * z * z) \ w = z * z * z;$		$(pc==8) \rightarrow \text{if } (w > z * z * z) \ w = z * z * z;$

program

A state-transition

- represented as a transition rules

8 rules in total:

(a1) $\rightarrow w = 0; \text{ goto } a2;$

(a2) $\rightarrow x = 0; \text{ goto } a3;$

(a3) $\rightarrow y = z * z; \text{ goto } a4;$

(a4&& $x \geq y$) $\rightarrow \text{ goto } a8;$

(a4&& $x < y$) $\rightarrow \text{ goto } a5;$

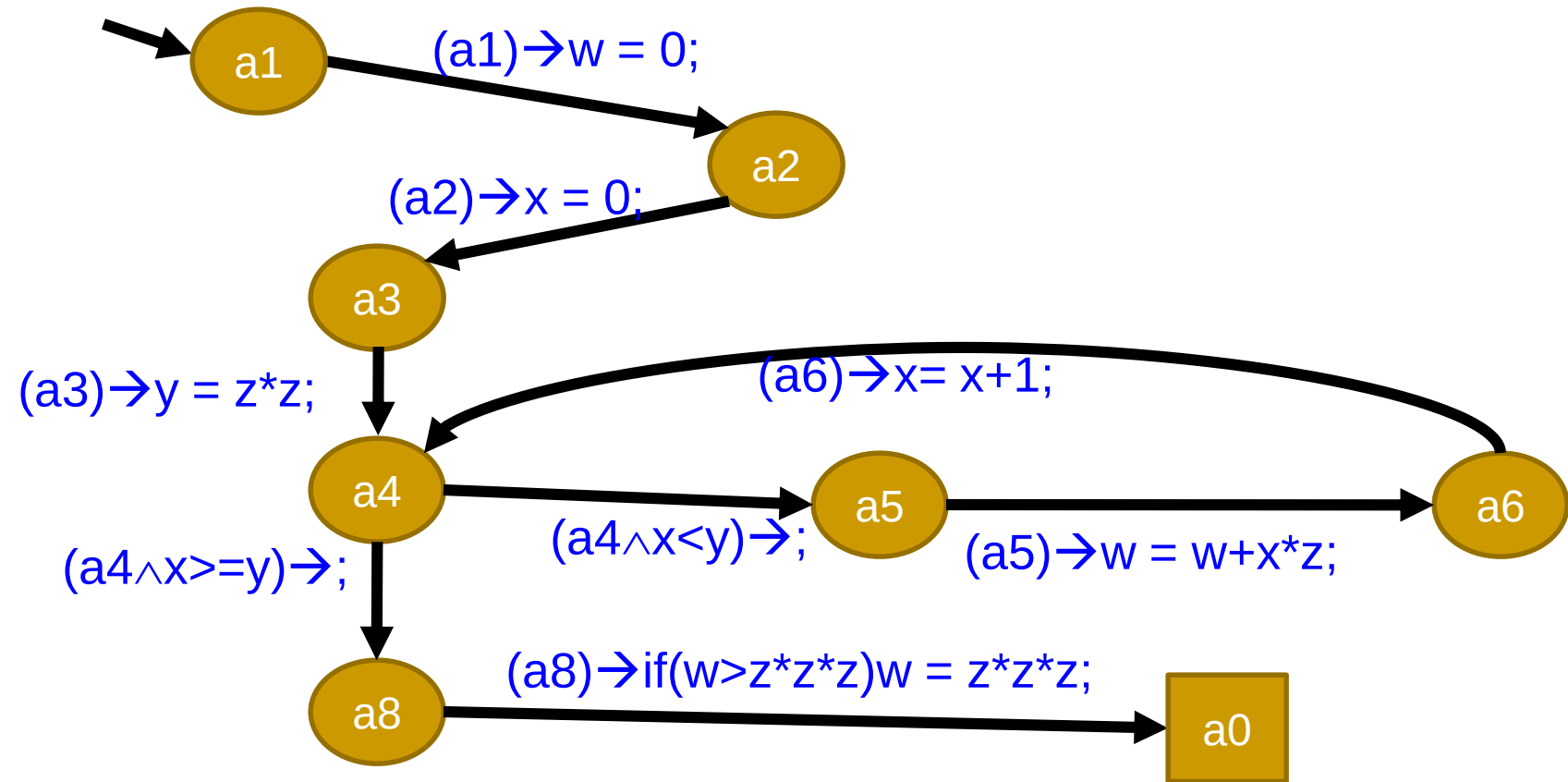
(a5) $\rightarrow w = w + x * z; \text{ goto } a6;$

(a6) $\rightarrow x = x + 1; \text{ goto } a4;$

(a8) $\rightarrow \text{ if } (w > z * z * z) \text{ } w = z * z * z; \}$

A state-transition

- represented as a transition rules



Transition relation

- from state-transition graphs

Given a set of rules $r_1, r_2, \dots, r_m$ of the form

r_k : when (τ_k) may $y_{k,0}=d_0; y_{k,1}=d_1; \dots; y_{k,nk}=d_{nk};$

$$\begin{aligned} & t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \\ & \equiv \bigvee_{k \in [1, m]} \left(\tau_k \wedge y'_{k,0} = d_0 \wedge y'_{k,1} = d_1 \wedge \dots \wedge y'_{k,nk} = d_{nk} \right. \\ & \quad \wedge \bigwedge_{h \in [1, n]} \left(x_h \notin \{y_{k,0}, y_{k,1}, \dots, y_{k,nk}\} \Rightarrow x_h = x'_h \right) \\ & \quad \left. \right) \end{aligned}$$

Transition relation

- from transition rules.

Given a set of rules for $X=\{x,y,z\}$

r_1 : when $(x < y \ \&\& \ y > 2)$ may $y = x + y$; $x = 3$;

r_2 : when $(z \geq 2)$ may $y = x + 1$; $z = 0$;

r_3 : when $(x < 2)$ may $x = 0$;

$t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n)$

$\equiv (x < y \wedge y > 2 \wedge y' == x + y \wedge x' == 3 \wedge z' == z)$

$\vee (z \geq 2 \wedge y' == x + 1 \wedge z' == 0 \wedge x' == x)$

$\vee (x < 2 \wedge x' == 0 \wedge y' == y \wedge z' == z)$

Transition relation

- from state-transition graphs

In general, transition relation is expensive to construct.

Can we do the following state-space construction

$$\exists x'_0 \exists x'_1 \dots \exists x'_n (t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \wedge (b_{k-1} \uparrow))$$

directly with the GCM rules ?

Yes, ***it is possible.***

On-the-fly precondition calculation

- with transition rules.

Given a set of rules $r_1, r_2, \dots, r_m$ of the form

$$r_k: \text{when } (\tau_k) \text{ may } y_{k,0}=d_0; y_{k,1}=d_1; \dots; y_{k,nk}=d_{nk};$$

$$\exists x'_0 \exists x'_1 \dots \exists x'_n (t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \wedge (b \uparrow))$$

$$\equiv \bigvee_{k \in [1, m]} \left(\tau_k \wedge \right. \\ \left. \exists y_{k,0} \exists y_{k,1} \dots \exists y_{k,nk} \left(b \wedge \bigwedge_{h \in [0, nk]} y_{k,h} == d_h \right) \right)$$

However, transition rules are more complex than that.

On-the-fly precondition calculation with transition rules.

$$\exists x'_0 \exists x'_1 \dots \exists x'_n (t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \wedge (b \uparrow))$$

$$\equiv \bigvee_{k \in [1, m]} (\tau_k \wedge \exists y_{k,0} \exists y_{k,1} \dots \exists y_{k,nk} (b \wedge \bigwedge_{h \in [0, nk]} y_{k,h} == d_h))$$

pre(b) {

 r = false;

 for k = 1 to m, {

 let f = b;

 for h=nk to 0, f = $\exists y_{k,h} (f \wedge y_{k,h} == d_h)$;

 r = r $\vee$ ($\tau_k \wedge f$);

 }

 return (r);

}

On-the-fly precondition calculation

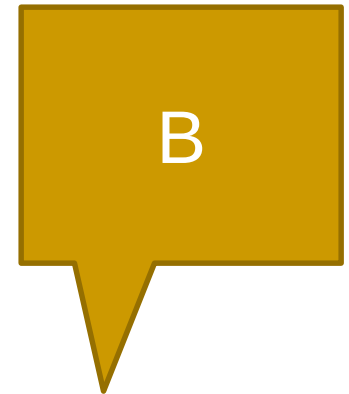
- with transition rules.

Given a set of rules for $X=\{x,y,z\}$

r_1 : when $(x < y \ \&\& \ y > 2)$ may $y = z; x = 3;$

r_2 : when $(z \geq 2)$ may $y = x + 1; z = 7;$

r_3 : when $(x < 2)$ may $z = 0;$



$$\begin{aligned} & \exists x'_0 \exists x'_1 \dots \exists x'_n (R(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \wedge (x < 4 \wedge z > 5)^\uparrow) \\ \equiv & (x < y \wedge y > 2 \wedge \exists y \exists x (x < 4 \wedge z > 5 \wedge y == z \wedge x == 3)) \\ & \vee (z \geq 2 \wedge \exists y \exists z (x < 4 \wedge z > 5 \wedge y == x + 1 \wedge z == 7)) \\ & \vee (x < 2 \wedge \exists z (x < 4 \wedge z > 5 \wedge z == 0)) \\ \equiv & (x < y \wedge y > 2 \wedge z > 5) \vee (z \geq 2 \wedge x < 4) \vee (x < 2 \wedge \exists z (\text{false})) \\ \equiv & (x < y \wedge y > 2 \wedge z > 5) \vee (z \geq 2 \wedge x < 4) \end{aligned}$$

On-the-fly precondition calculation with transition rules.

Given a set of rules $r_1, r_2, \dots, r_m$ of the form

r_k : when (τ_k) may s_k ;

$$\begin{aligned} & \exists x'_0 \exists x'_1 \dots \exists x'_n (t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \wedge (b \uparrow)) \\ & \equiv \bigvee_{k \in [1, m]} (\tau_k \wedge \text{pre}(s_k, b)) \end{aligned}$$

precondition
procedure

A general propositional formula

What is $\text{pre}(s, b)$?

A transition statement

Precondition calculation

- with substitution.

Given a set of rules $r_1, r_2, \dots, r_m$ of the form

$r_k: \text{when } (\tau_k) \text{ may } s_k;$

What is $\text{pre}(s, b)$?

new expression obtained from b by replacing every occurrence of x with E .

■ $\text{pre}(x = E;, b) \equiv b[x/E]$

Ex 1. $b: (x == y + 2 \wedge x < 4 \wedge z > 5)$ to $s: x = x + z;$

$(x == y + 2 \wedge x < 4 \wedge z > 5) [x/x+z] \equiv x + z == y + 2 \wedge x + z < 4 \wedge z > 5$

Ex 2. $b: (x == y + 2 \wedge x < 4 \wedge z > 5)$ to $s: x = 5;$

$(x == y + 2 \wedge x < 4 \wedge z > 5) [x/5] \equiv 5 == y + 2 \wedge 5 < 4 \wedge z > 5$

Ex 3. $b: (x == y + 2 \wedge x < 4 \wedge z > 5)$ to $s: x = 2 * x + 1;$

$(x == y + 2 \wedge x < 4 \wedge z > 5) [x/2 * x + 1] \equiv 2 * x + 1 == y + 2 \wedge 2 * x + 1 < 4 \wedge z > 5$

On-the-fly precondition calculation with transition rules.

Given a set of rules $r_1, r_2, \dots, r_m$ of the form

r_k : when (τ_k) may s_k ;

What is $\text{pre}(s, b)$?

new expression obtained from b by replacing every occurrence of x with E .

- $\text{pre}(x = E;, b) \equiv b[x/E]$
- $\text{pre}(s_1 s_2, b) \equiv \text{pre}(s_1, \text{pre}(s_2, b))$
- $\text{pre}(\text{if } (B) s_1 \text{ else } s_2) \equiv (B \wedge \text{pre}(s_1, b)) \vee (\neg B \wedge \text{pre}(s_2, b))$
- $\text{pre}(\text{while } (B) s, b) \equiv \dots$

Ex. the precondition to $x=x+z$;
 $(x==y+2 \wedge x<4 \wedge z>5) [x/x+z]$
 $\equiv x+z==y+2 \wedge x+z<4 \wedge z>5$

On-the-fly precondition calculation with transition rules.

Given a set of rules $r_1, r_2, \dots, r_m$ of the form

$$r_k: \text{when } (\tau_k) \text{ may } s_k;$$

What is $\text{pre}(s, b)$?

$\text{pre}(\text{while } (K) s, b) \equiv \text{formula } L_1 \vee L_2 \text{ for}$

L_1 : those states that reach $\neg K \wedge b$ with finite steps of s
through states in K ; and

L_2 : those states that never leave K with steps of s .

On-the-fly precondition calculation with transition rules.

L_1 : those states that reach $\neg K \wedge b$ with finite steps of
s through states in K

$w_0 = \neg K \wedge b$; $k = 1$;

repeat

also a least fixpoint procedure

$w_k = w_{k-1} \vee (K \wedge \text{pre}(s, w_{k-1}))$;

$k = k + 1$;

until $w_k \equiv w_{k-1}$;

return w_k ;

Precondition to b
through while (K) s;

Example: $b \equiv x == 2 \wedge y == 3$
while ($x < y$) $x = x + 1$;

```
w0 =  $\neg K \wedge b$ ; k = 1;  
repeat  
  wk = wk-1  $\vee$  ( $K \wedge \text{pre}(s, w_{k-1})$ );  
  k = k + 1;  
until wk  $\equiv$  wk-1;  
return wk;
```

L1 computation.

$w_0 \equiv x >= y \wedge x == 2 \wedge y == 3 \equiv \text{false} ; k = 1$;

$w_1 \equiv \text{false} \vee (x < y \wedge \text{pre}(x = x + 1, \text{false}))$;
 $\equiv \text{false} \vee (x < y \wedge \text{false})$;
 $\equiv \text{false}$;

On-the-fly precondition calculation with transition rules.

Given a set of rules $r_1, r_2, \dots, r_m$ of the form
 $\text{pre}(\text{while } (K) \text{ s, b})$

L_2 : those states that never leave K with steps of s .

$w_0 = K; k = 1;$

repeat

a greatest fixpoint procedure

$w_k = K \wedge \text{pre}(s, w_{k-1});$

$k = k + 1;$

until $w_k \equiv w_{k-1};$

return $w_k;$

Precondition to b through while (K) s;

Example:

while ($x < y \ \&\& \ x > 0$) $x = x + 1$;

L2 computation.

$$w_0 \equiv x < y \wedge x > 0 ; k = 1;$$

$$\begin{aligned} w_1 &\equiv x < y \wedge x > 0 \wedge \text{pre}(x = x + 1, x < y \wedge x > 0) \\ &\equiv x < y \wedge x > 0 \wedge x + 1 < y \wedge x + 1 > 0 \equiv x > 0 \wedge x + 1 < y \end{aligned}$$

$$\begin{aligned} w_2 &\equiv x + 1 < y \wedge x > 0 \wedge \text{pre}(x = x + 1, x + 1 < y \wedge x > 0) \\ &\equiv x + 1 < y \wedge x > 0 \wedge x + 2 < y \wedge x + 1 > 0 \equiv x > 0 \wedge x + 2 < y \end{aligned}$$

non-terminating for algorithms and protocols!

```
w0 = K; k = 1;  
repeat  
  wk = wk-1 ∧ pre(s, wk-1);  
  k = k + 1;  
until wk ≡ wk-1;  
return wk;
```

Precondition to b through while (K) s;

Example:

while ($x > y \ \&\& \ x > 0$) $x = x + 1$;

L2 computation.

$$w_0 \equiv x > y \wedge x > 0 ; k = 1;$$

$$\begin{aligned} w_1 &\equiv x > y \wedge x > 0 \wedge \text{pre}(x = x + 1, x > y \wedge x > 0) \\ &\equiv x > y \wedge x > 0 \wedge x + 1 > y \wedge x + 1 > 0 \equiv x > y \wedge x > 0 \end{aligned}$$

terminating for algorithms and protocols!

```
w0 = K; k = 1;  
repeat  
  wk = K ∧ pre(s, wk-1);  
  k = k + 1;  
until wk ≡ wk-1;  
return wk;
```

Precondition to b through while (K) s;

Example: $b \equiv x==2 \wedge y==3$

while ($x > y \ \&\& \ x > 0$) $x = x + 1$;

L_1 computation.

$$w_0 \equiv (x \leq y \vee x \leq 0) \wedge x == 2 \wedge y == 3 \equiv x == 2 \wedge y == 3;$$

$$w_1 \equiv (x == 2 \wedge y == 3) \vee (x > y \wedge x > 0 \wedge \text{pre}(x = x + 1, x == 2 \wedge y == 3));$$

$$\equiv (x == 2 \wedge y == 3) \vee (x > y \wedge x > 0 \wedge x == 1 \wedge y == 3);$$

$$\equiv (x == 2 \wedge y == 3) \vee \text{false}$$

$$\equiv x == 2 \wedge y == 3$$

```
w0 = ¬K ∧ b; k = 1;  
repeat  
  wk = wk-1 ∨ (K ∧ pre(s, wk-1));  
  k = k + 1;  
until wk ≡ wk-1;  
return wk;
```

Symbolic safety analysis (backward)

- without transition relation

Encode the states with variables $x_0, x_1, \dots, x_n$.

- the state set as a proposition formula: $S(x_0, x_1, \dots, x_n)$
- the risk state set as $\neg \eta(x_0, x_1, \dots, x_n)$
- the initial state set as $I(x_0, x_1, \dots, x_n)$
- the precondition procedure $\text{pre}(x_0, x_1, \dots, x_n)$

$b_0 = \neg \eta(x_0, x_1, \dots, x_n) \wedge S(x_0, x_1, \dots, x_n); k = 1;$

repeat

$b_k = b_{k-1} \vee \text{pre}(b_{k-1});$

$k = k + 1;$

until $b_k \equiv b_{k-1};$

a least fixpoint procedure

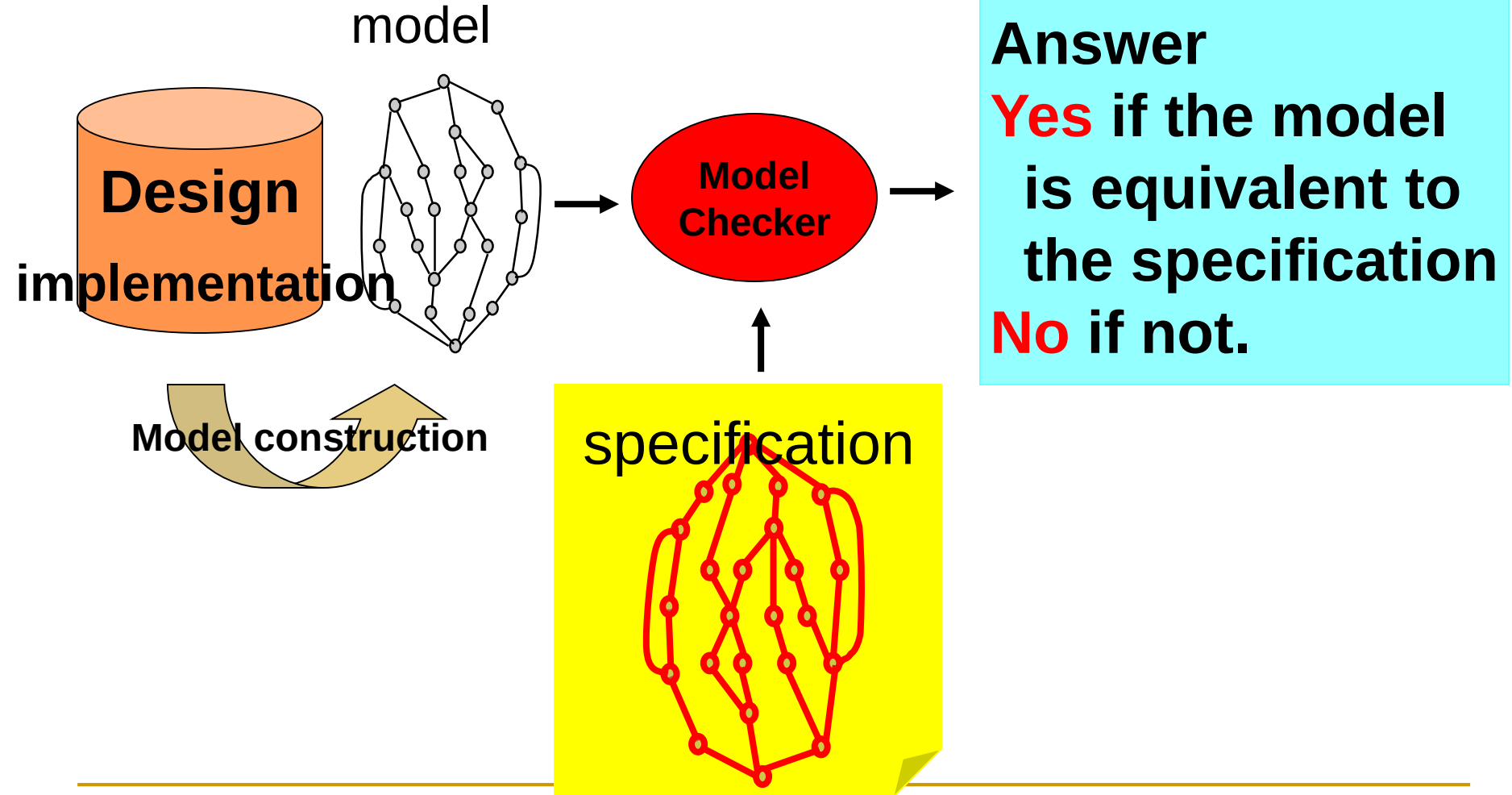
if $(b_k \wedge I(x_0, x_1, \dots, x_n)) \equiv \text{false}$, return 'safe'; else return 'risky';

CTL

- symbolic model-checking algorithm

```
label( $\varphi$ ) {  
  case true, return  $s(x_1, \dots, x_n)$ ;  
  case p, return  $p \wedge s(x_1, \dots, x_n)$ ;  
  case  $\neg \varphi$ , return  $s(x_1, \dots, x_n) \wedge \neg \text{label}(\varphi)$ ;  
  case  $\varphi \vee \psi$ , return  $s(x_1, \dots, x_n) \wedge (\text{label}(\varphi) \vee \text{label}(\psi))$ ;  
  case  $\exists \bigcirc \varphi$ , return  
     $s(x_1, \dots, x_n)$   
     $\wedge \exists x'_0 \exists x'_1 \dots \exists x'_n (t(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \wedge \text{label}(\varphi)^\uparrow)$ ;  
  case  $\exists \psi_1 \bigcup \psi_2$ , return  $\text{lfp}(\text{label}(\psi_1), \text{label}(\psi_2))$ ;  
  case  $\exists \Box \varphi$ , return  $\text{gfp}(\text{label}(\varphi))$ ;  
}
```

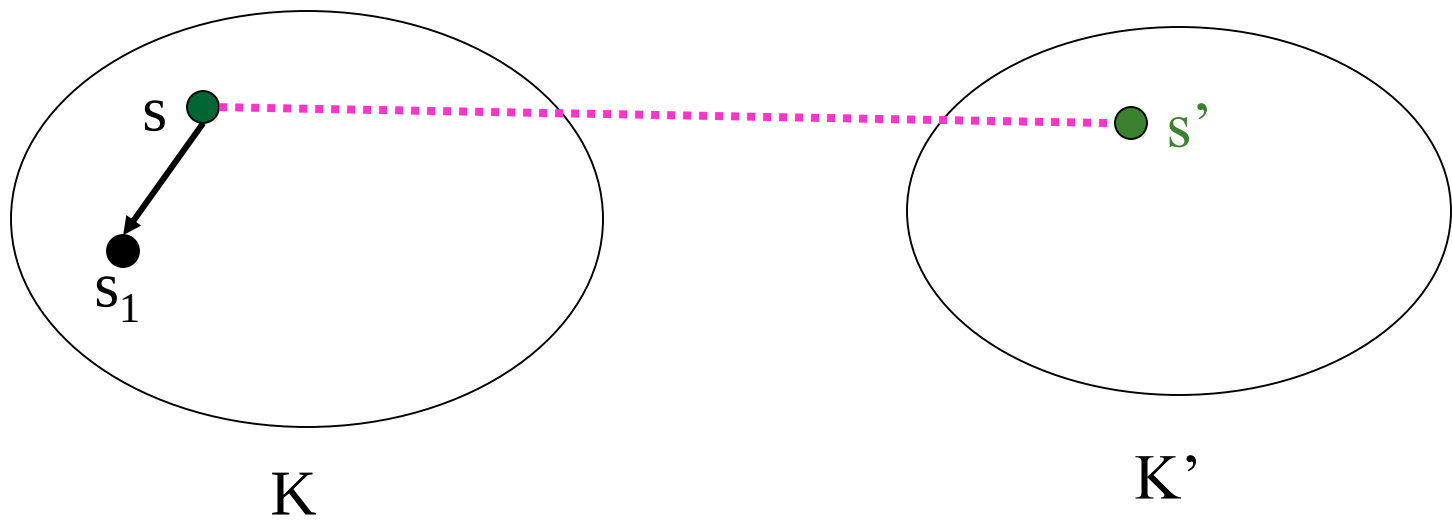
Bisimulation Framework



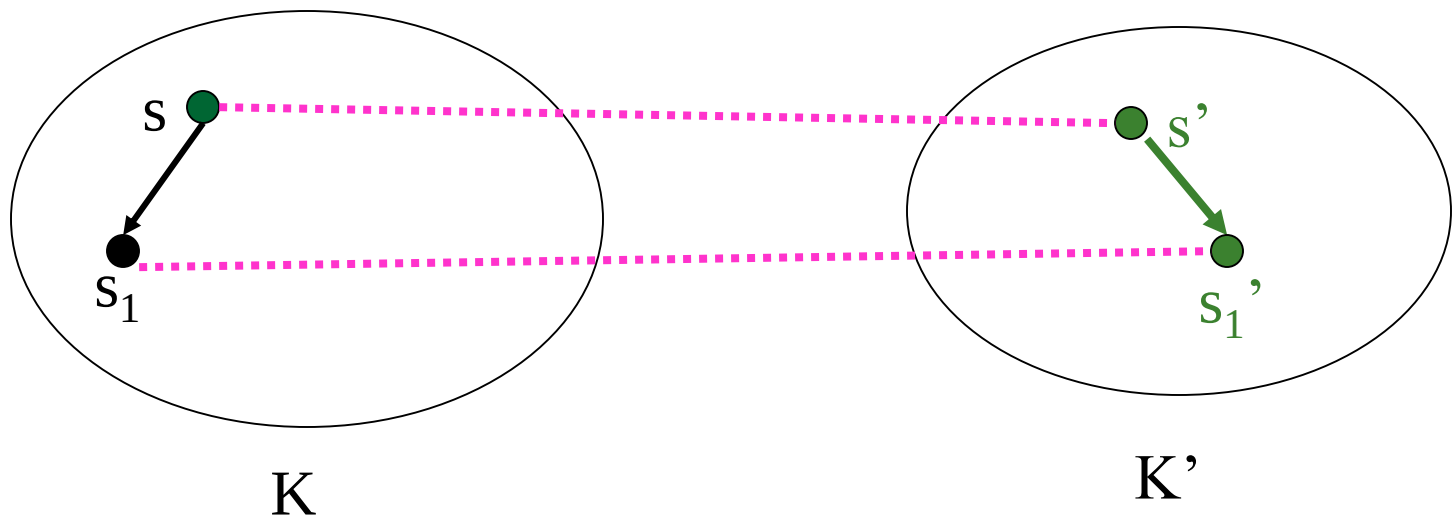
Bisimulation-checking

- $K = (S, S_0, R, AP, L)$
 $K' = (S', S'_0, R', AP, L')$
- Note K and K' use the same set of atomic propositions AP .
- $B \in S \times S'$ is a **bisimulation relation** between K and K' iff for every $B(s, s')$:
 - $L(s) = L'(s')$ (**BSIM 1**)
 - If $R(s, s_1)$, then there exists s'_1 such that $R'(s', s'_1)$ and $B(s_1, s'_1)$. (**BISIM 2**)
 - If $R(s', s'_2)$, then there exists s_2 such that $R(s, s_2)$ and $B(s_2, s'_2)$. (**BISIM 3**)

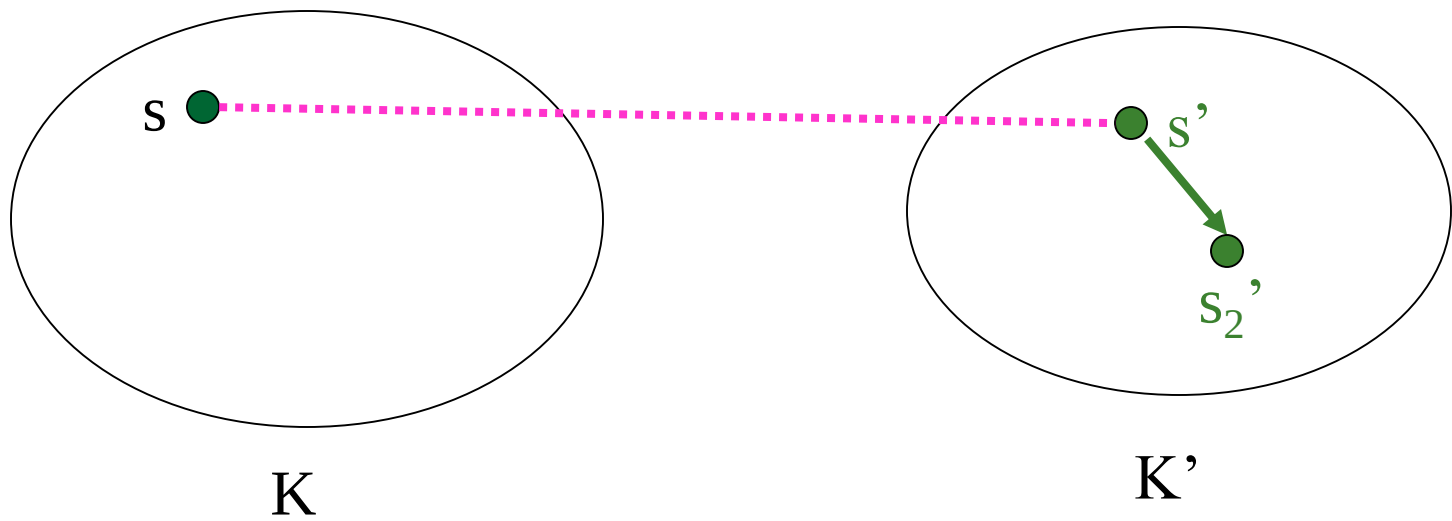
Bisimulations



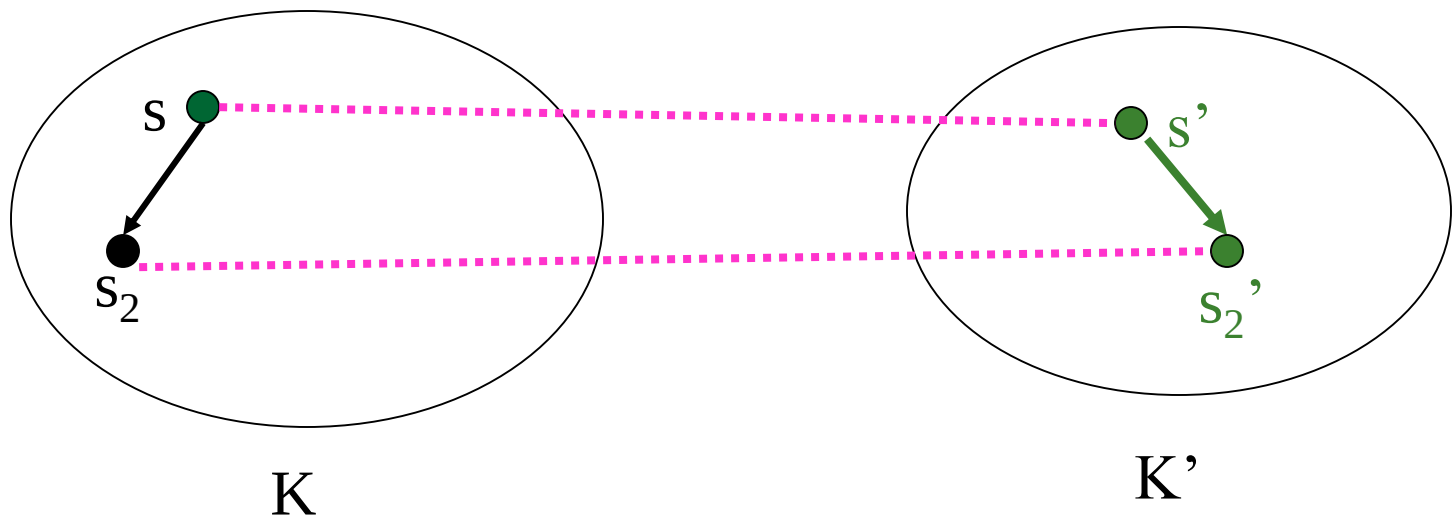
Bisimulations



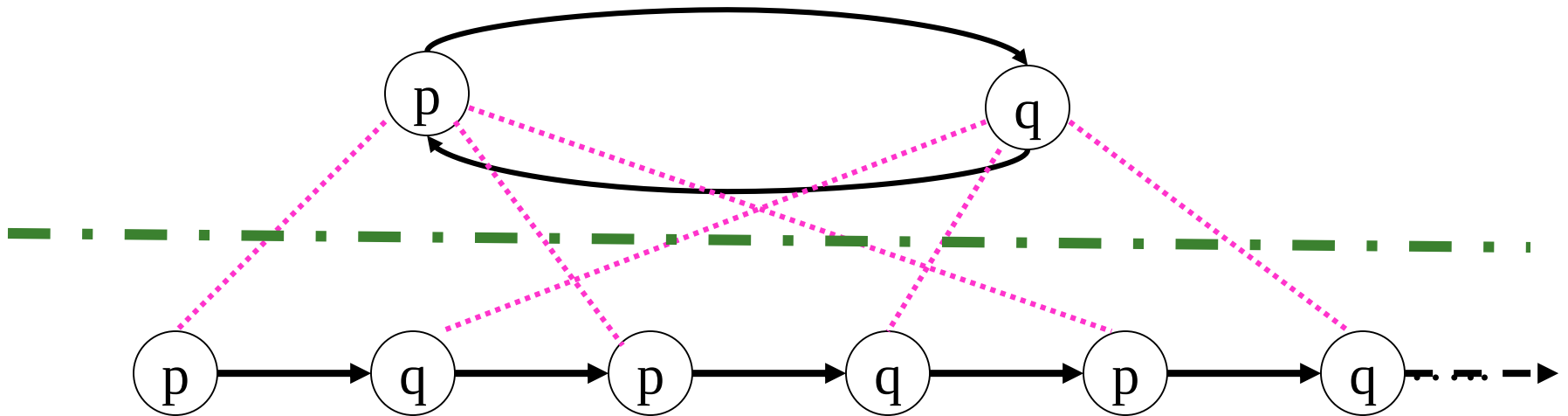
Bisimulations



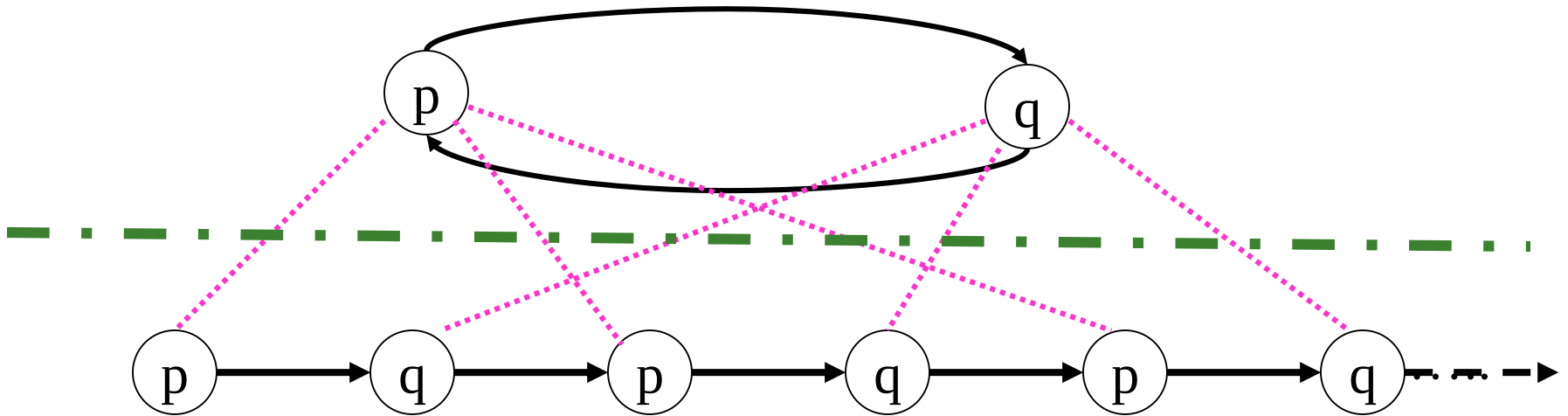
Bisimulations



Examples

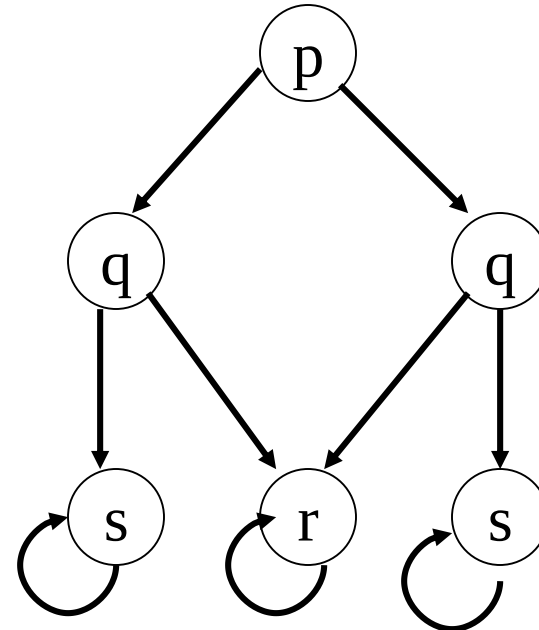
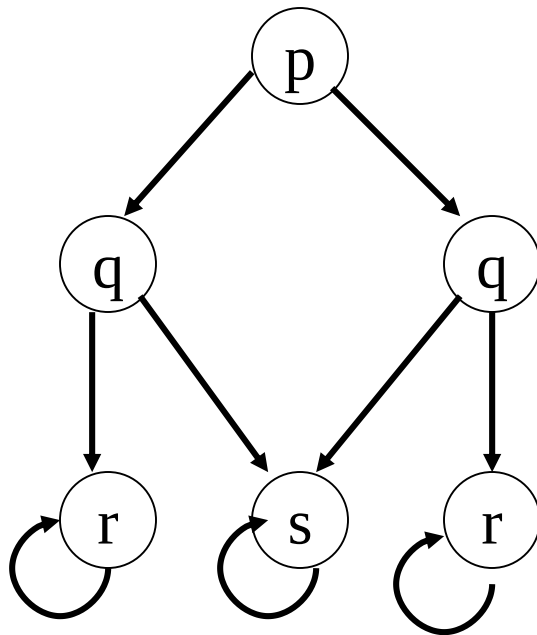


Examples

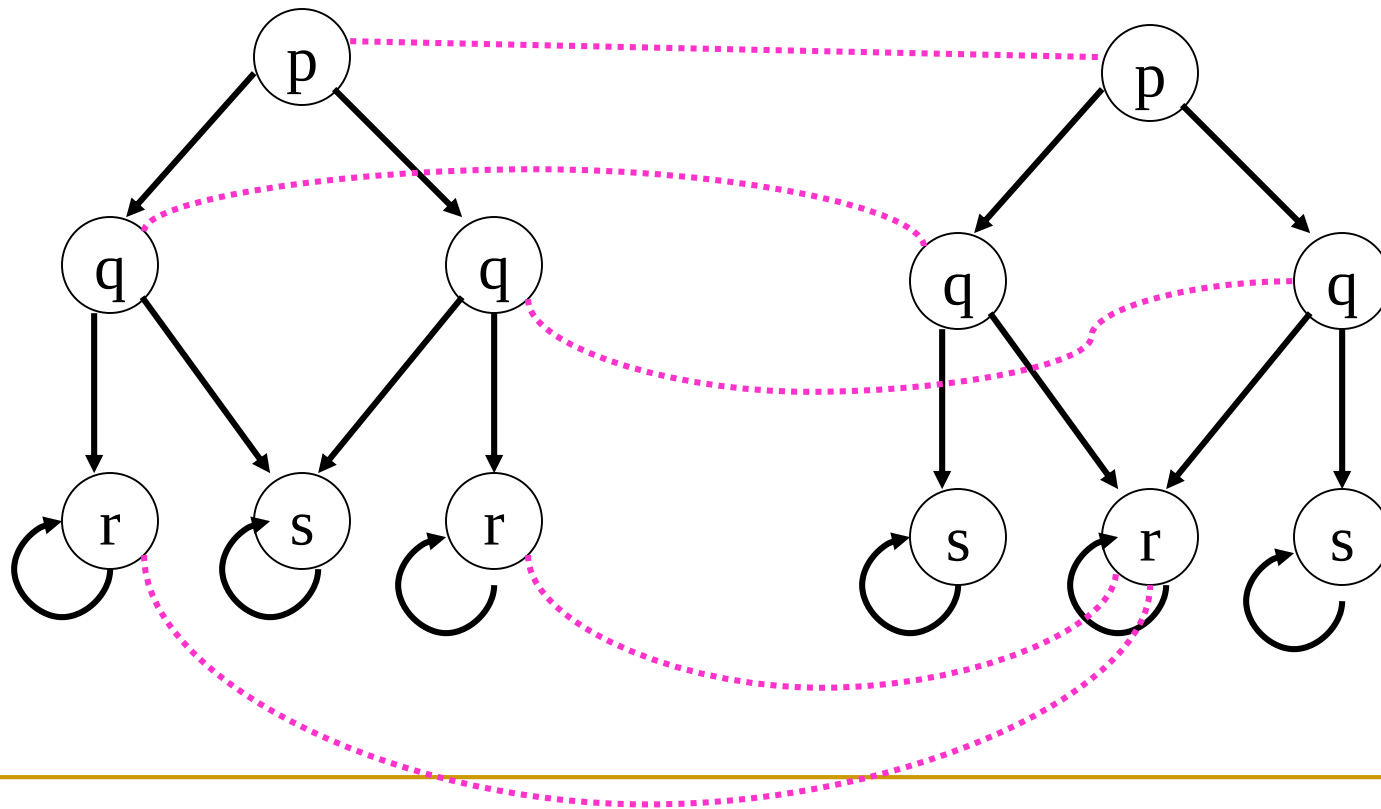


Unwinding preserves bisimulation

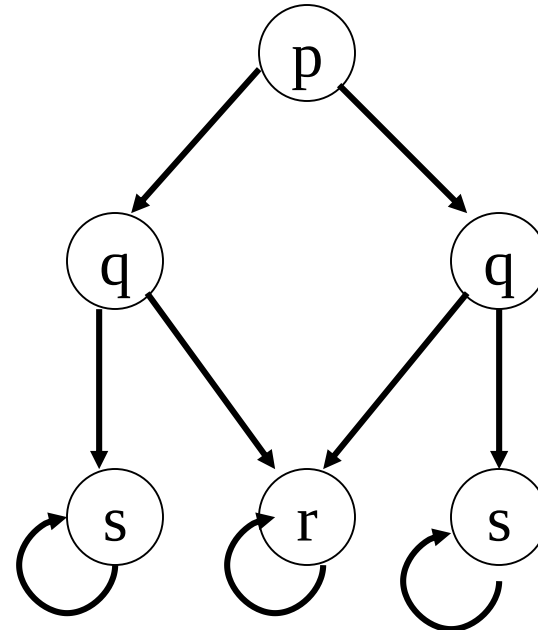
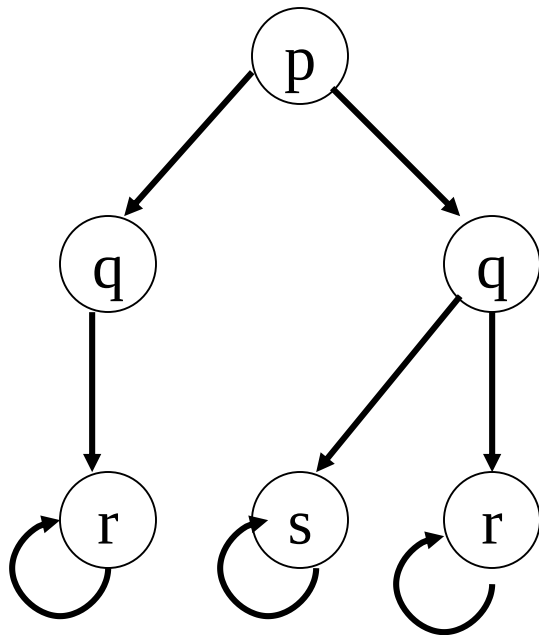
Examples



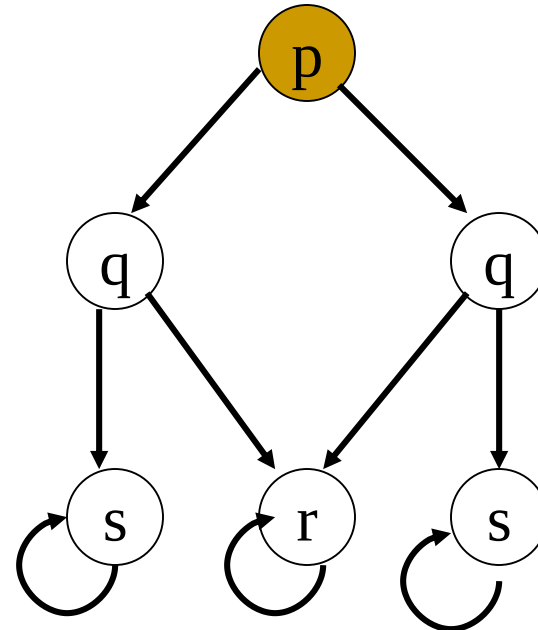
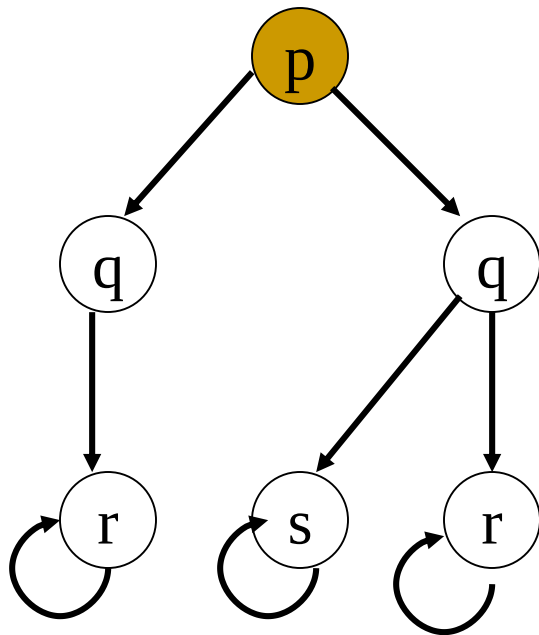
Examples



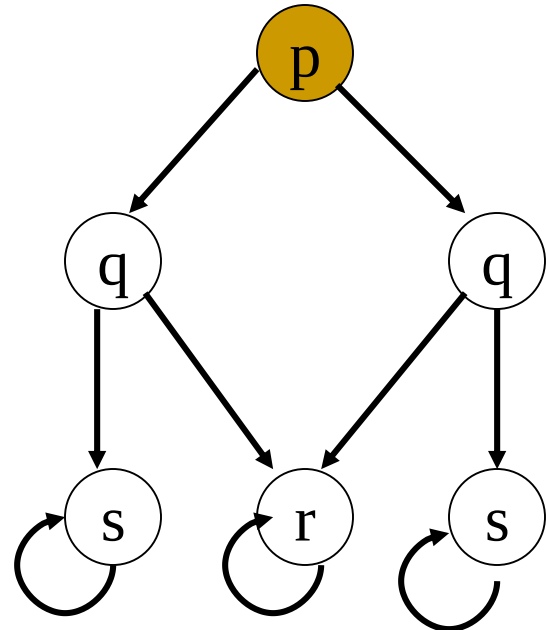
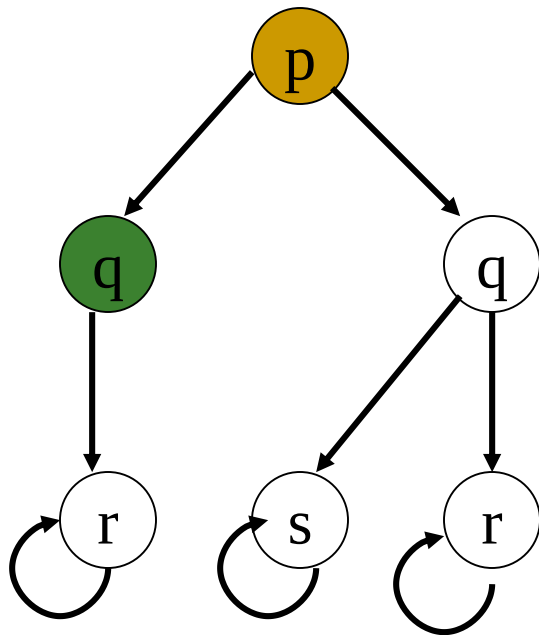
Examples



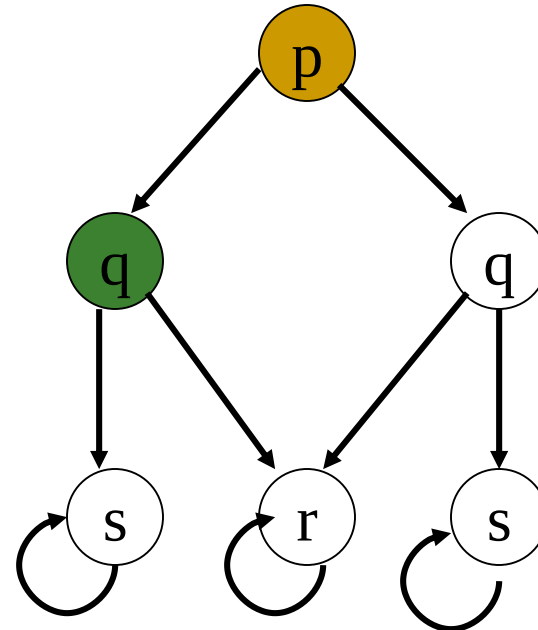
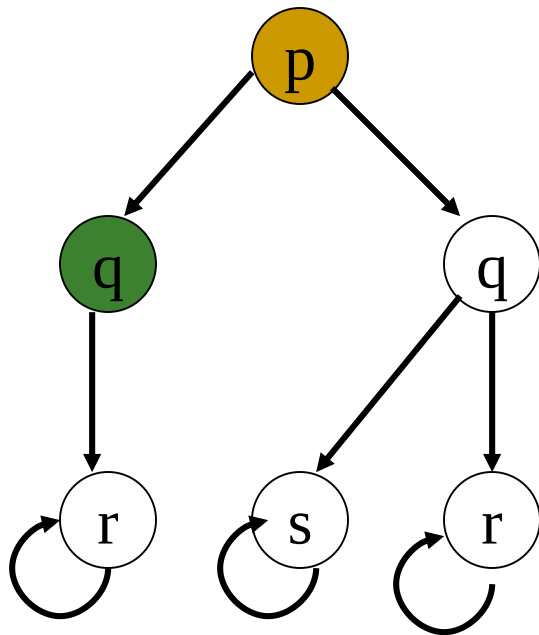
Examples



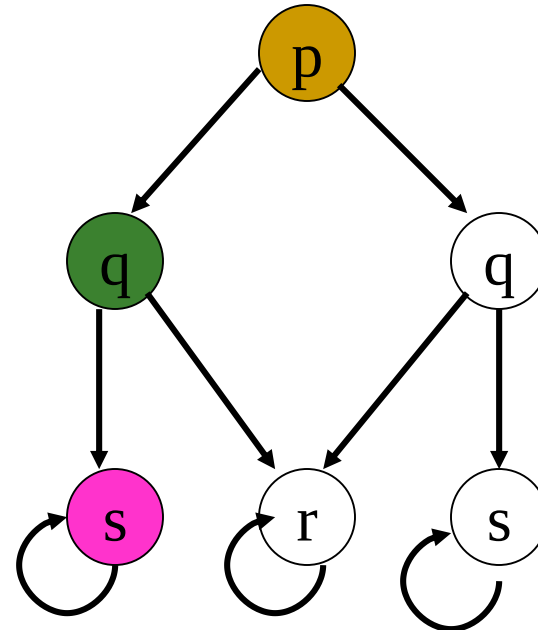
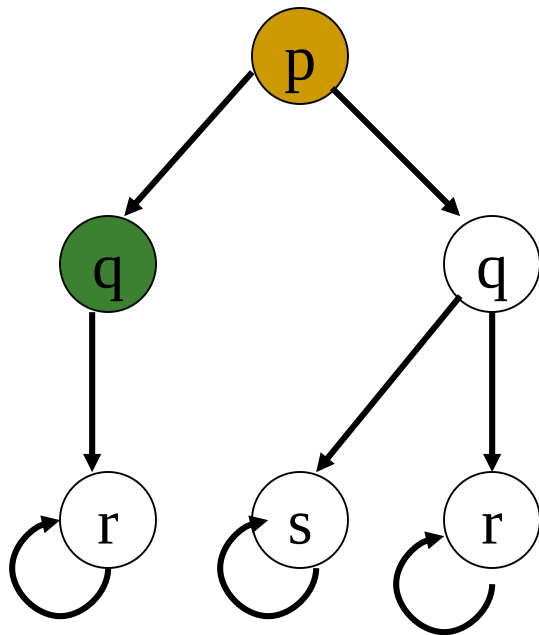
Examples



Examples



Examples



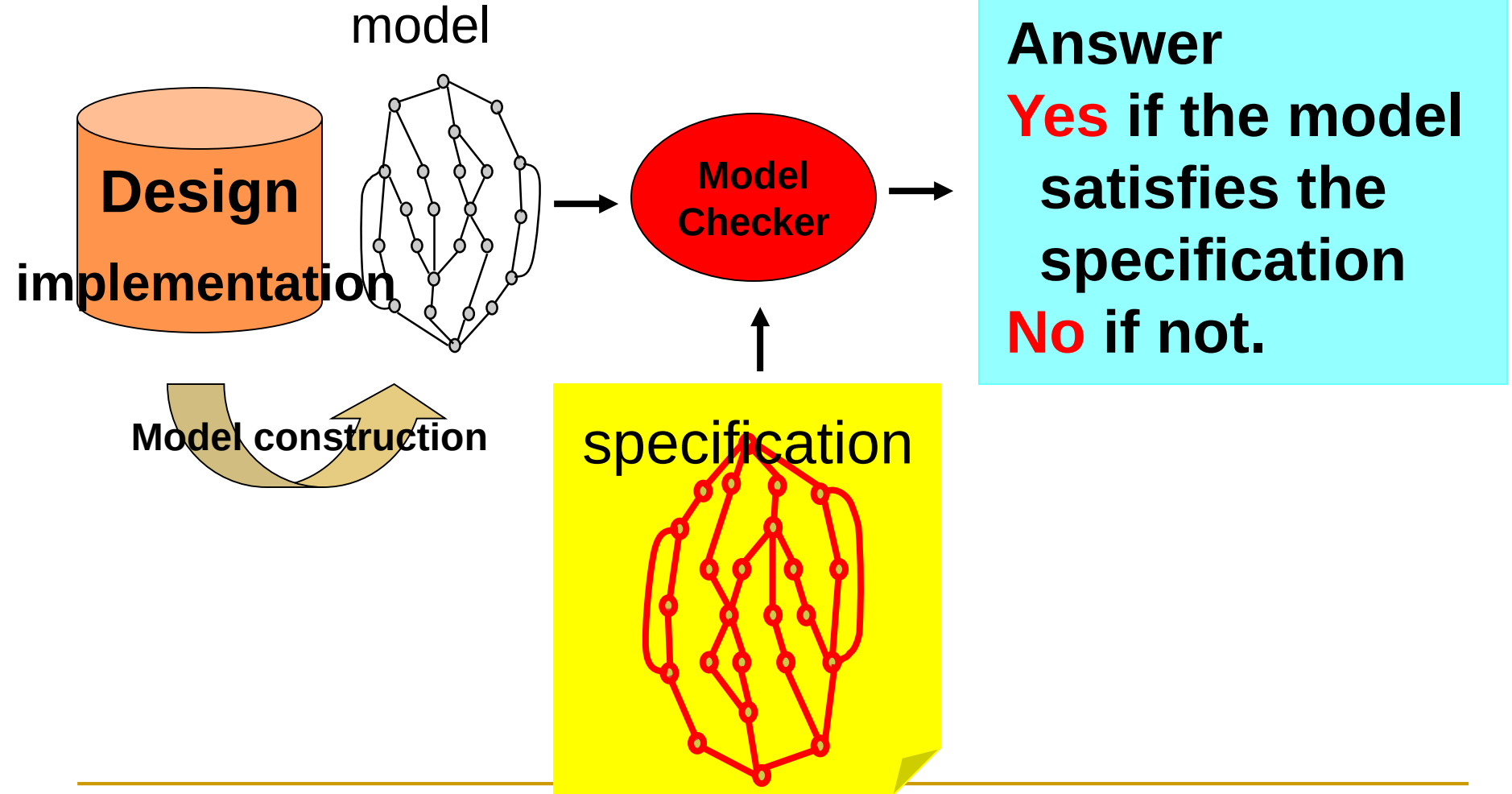
Bisimulations

- $K = (S, S_0, R, AP, L)$
- $K' = (S', S_0', R', AP, L')$
- K and K' are **bisimilar** (bisimulation equivalent) iff there exists a bisimulation relation $B \subseteq S \times S'$ between K and K' such that:
 - For each s_0 in S_0 there exists s_0' in S_0' such that $B(s_0, s_0')$.
 - For each s_0' in S_0' there exists s_0 in S_0 such that $B(s_0, s_0')$.

The Preservation Property.

- $K = (S, S_0, R, AP, L)$
 $K' = (S', S'_0, R', AP, L')$
- $B \subseteq S \times S'$, a bisimulation.
- Suppose $B(s, s')$.
- **FACT**: For any CTL* (or proposition mu-calculus) formula ψ (over AP), $K, s \models \psi$ iff $K', s' \models \psi$.
- If K' is smaller than K this is worth something.

Simulation Framework



Simulation-checking

- $K = (S, S_0, R, AP, L)$
 $K' = (S', S'_0, R', AP, L')$
- Note K and K' use the same set of atomic propositions AP .
- $B \in S \times S'$ is a **simulation relation** between K and K' iff for every $B(s, s')$:
 - $L(s) = L'(s')$ (**BSIM 1**)
 - If $R(s, s_1)$, then there exists s'_1 such that $R'(s', s'_1)$ and $B(s_1, s'_1)$. (**BISIM 2**)

Simulations

- $K = (S, S_0, R, AP, L)$
- $K' = (S', S_0', R', AP, L')$
- K is simulated by (implements or refines) K' iff there exists a simulation relation $B \subseteq S \times S'$ between K and K' such that for each s_0 in S_0 there exists s_0' in S_0' such that $B(s_0, s_0')$.

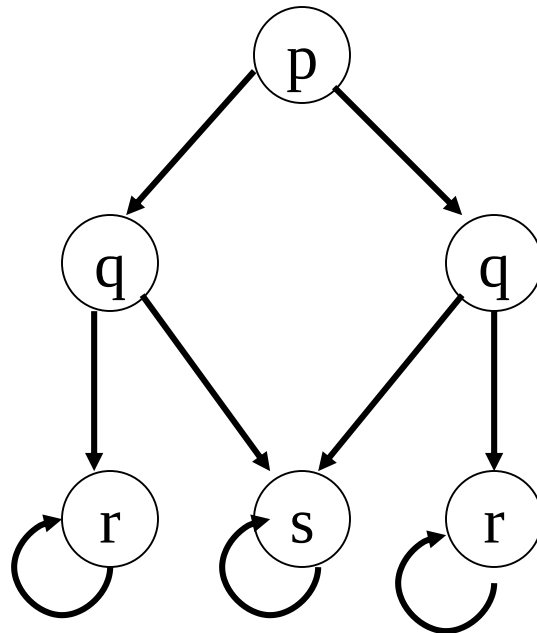
Bisimulation Quotients

- $K = (S, S_0, R, AP, L)$
- There is a maximal simulation $B \subseteq S \times S$.
 - Let R be this bisimulation.
 - $[s] = \{s' \mid s R s'\}$.
- R can be computed “easily”.
- $K' = K / R$ is the bisimulation quotient of K .

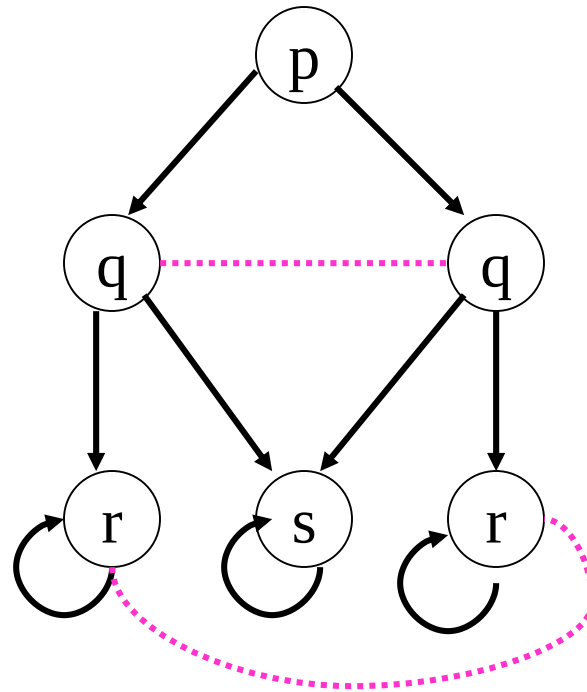
Bisimulation Quotient

- $K = (S, S_0, R, AP, L)$
- $[s] = \{s' \mid s R s'\}.$
- $K' = K / R = (S', S'_0, R', AP, L').$
 - $S' = \{[s] \mid s \in S\}$
 - $S'_0 = \{[s_0] \mid s_0 \in S_0\}$
 - $R' = \{([s], [s']) \mid R(s_1, s_1'), s_1 \in [s], s_1' \in [s']\}$
 - $L'([s]) = L(s).$

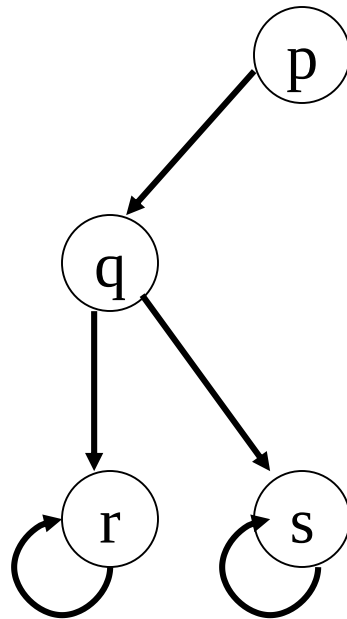
Examples



Examples



Examples



Facts About a (Bi)Simulation

- The empty set is always a (bi)simulation
- If R, R' are (bi)simulations, so is $R \cup R'$
- Hence, there always exists a *maximal* (bi)simulation:
 - Checking if $DB_1 = DB_2$: compute the maximal bisimulation R , then test $(\text{root}(DB_1), \text{root}(DB_2))$ in R

Kripke structure

- simulation-checking

```
/* Given model  $A = (S, S_0, R, L)$ , spec.  $A' = (S', S'_0, R', L')$  */  
Simulation-checking( $A, A'$ ) /* using greatest fixpoint algorithm */ {  
  Let  $B = \{(s, s') \mid s \in S, s' \in S', L(s) = L'(s')\}$  ;  
  repeat {  
     $B = B - \{(s, s') \mid (s, s') \in B, \exists (s, t) \in R \forall (s', t') \in R' ((t, t') \notin B)\}$ ;  
  } until no more changes to B.  
  if there is an  $s_0 \in S_0$  with  $\forall s'_0 \in S'_0 ((s_0, s'_0) \notin B)$ ,  
    return 'no simulation',  
    else return 'simulation exists.'  
}
```

The procedure terminates since B is finite in the Kripke structure.

Kripke structure

- bisimulation-checking

/* Given model $A = (S, S_0, R, L)$, spec. $A' = (S', S'_0, R', L')$ */

Bisimulation-checking(A, A') /* using greatest fixpoint algorithm */ {

Let $B = \{(s, s') \mid s \in S, s' \in S', L(s) = L'(s')\}$;

repeat {

$B = B - \{(s, s') \mid (s, s') \in B, \exists (s, t) \in R \forall (s', t') \in R' ((t, t') \notin B)\}$;

$B = B - \{(s, s') \mid (s, s') \in B, \exists (s', t') \in R' \forall (s, t) \in R ((t, t') \notin B)\}$;

} until no more changes to B.

if there is an $s_0 \in S_0$ with $\forall s'_0 \in S'_0 ((s_0, s'_0) \notin B)$,

return 'no simulation,'

if there is an $s'_0 \in S'_0$ with $\forall s_0 \in S_0 ((s_0, s'_0) \notin B)$,

return 'no simulation,'

else return 'simulation exists.'

}

(Bi)Simulation

- complexities

- Bisimulation: $O((m+n)\log(m+n))$
- Simulation: $O(m \cdot n)$
- In contrast, finding a graph homeomorphism is NP-complete.

Symbolic simulation-checking

- Encode the states with variables

- $x_0, x_1, \dots, x_n$ (for the model) and
- $y_0, y_1, \dots, y_m$ (for the spec.)

Usually there are shared variables

between $\{x_0, x_1, \dots, x_n\}$ and $\{y_0, y_1, \dots, y_m\}$.

$L(s) = L'(s')$ means that the shared variables are of the same values.

- the state sets as proposition formulas:

- $s(x_0, x_1, \dots, x_n) \ \& \ s(y_0, y_1, \dots, y_m)$

- the initial state set as

- $i(x_0, x_1, \dots, x_n) \ \& \ i'(y_0, y_1, \dots, y_m)$

- the transition set as

- $R(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n) \ \& \ R'(y_0, y_1, \dots, y_n, y'_0, y'_1, \dots, y'_n)$

Symbolic simulation-checking

$B_0 = \bigwedge_{L(x_0, x_1, \dots, x_n) = L(y_0, y_1, \dots, y_m)} s(x_0, x_1, \dots, x_n) \wedge s(y_0, y_1, \dots, y_m);$

for ($k = 1, B_1 = \text{false}; B_k \neq B_{k-1}; k = k + 1$)

$B_k = B_{k-1} \wedge \neg \exists x'_0 \exists x'_1 \dots \exists x'_n ($
 $R(x_0, x_1, \dots, x_n, x'_0, x'_1, \dots, x'_n)$
 $\wedge \neg \exists y'_0 \exists y'_1 \dots \exists y'_m ($
 $R'(y_0, y_1, \dots, y_m, y'_0, y'_1, \dots, y'_m) \wedge (B_{k-1} \uparrow)$
 $)) ;$

if ($(i(x_0, x_1, \dots, x_n) \neq \exists y_0 \exists y_1 \dots \exists y_m (B_k))$,

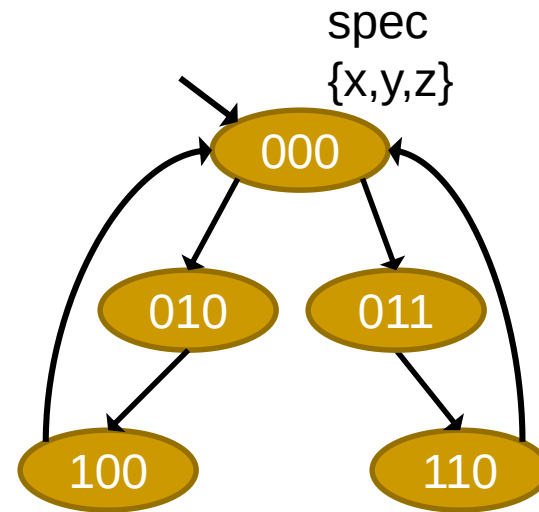
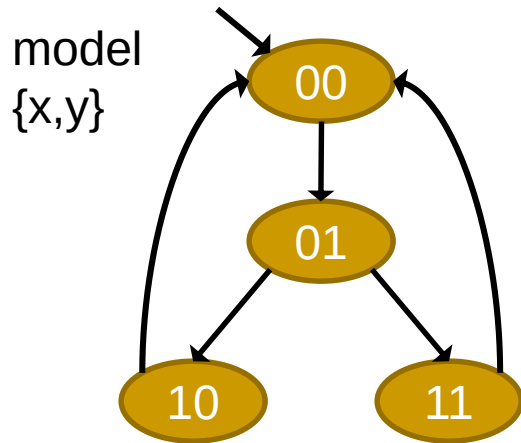
return 'no simulation';

else return 'a simulation exists';

change all
unprimed
variable in B_{k-1}
to primed.

Symbolic simulation-checking

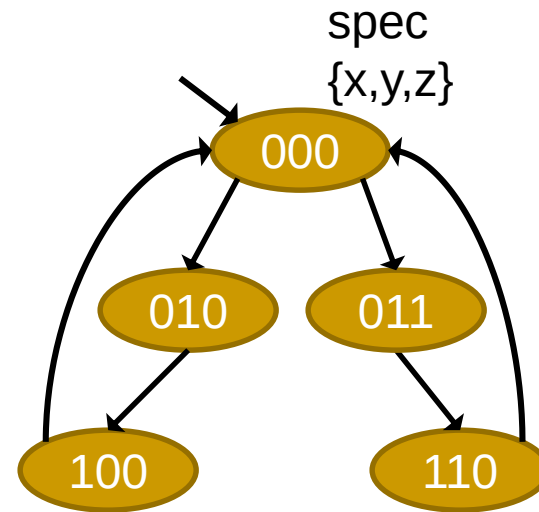
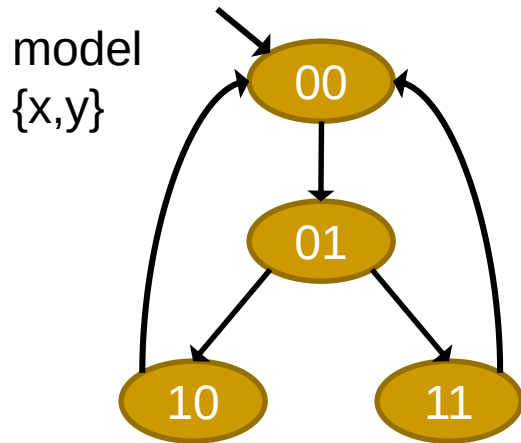
- an example



- $s(x,y) \equiv \text{true}$, $s'(x,y,z) \equiv \neg z \vee (\neg x \wedge y \wedge z)$
- $i(x,y) \equiv \neg x \wedge \neg y$, $i'(x,y,z) \equiv \neg x \wedge \neg y \wedge \neg z$
- $R(x,y,x',y') \equiv \dots\dots\dots$, $R'(x,y,z,x',y',z') \equiv \dots\dots\dots$

Symbolic simulation-checking

- an example



- $R(x,y,x',y') \equiv (\neg x \wedge \neg y \wedge \neg x' \wedge y') \vee (\neg x \wedge y \wedge x' \wedge \neg y') \vee (\neg x \wedge y \wedge x' \wedge y') \vee (x \wedge \neg y \wedge \neg x' \wedge \neg y') \vee (x \wedge y \wedge \neg x' \wedge \neg y')$
- $R'(x,y,z,x',y',z') \equiv (\neg x \wedge \neg y \wedge \neg z \wedge \neg x' \wedge y') \vee (\neg x \wedge y \wedge \neg z \wedge x' \wedge \neg y' \wedge \neg z') \vee (\neg x \wedge y \wedge z \wedge x' \wedge y' \wedge \neg z') \vee (x \wedge \neg y \wedge \neg z \wedge \neg x' \wedge \neg y' \wedge \neg z') \vee (x \wedge y \wedge \neg z \wedge \neg x' \wedge \neg y' \wedge \neg z')$

Symbolic simulation-checking

- an example

$$B_0 = s(x,y) \wedge s'(x,y,z) = \neg z \vee (\neg x \wedge y \wedge z)$$

$$\begin{aligned}
 B_1 &= (\neg z \vee (\neg x \wedge y \wedge z)) \wedge \neg \exists x' \exists y' (\\
 &\quad ((\neg x \wedge \neg y \wedge \neg x' \wedge y') \vee (\neg x \wedge y \wedge x' \wedge \neg y') \\
 &\quad \vee (\neg x \wedge y \wedge x' \wedge y') \vee (x \wedge \neg y \wedge \neg x' \wedge \neg y') \vee (x \wedge y \wedge \neg x' \wedge \neg y') \\
 &\quad) \\
 &\quad \wedge \neg \exists x' \exists y' \exists z' (\\
 &\quad ((\neg x \wedge \neg y \wedge \neg z \wedge \neg x' \wedge y') \\
 &\quad \vee (\neg x \wedge y \wedge \neg z \wedge x' \wedge \neg y' \wedge \neg z') \vee (\neg x \wedge y \wedge z \wedge x' \wedge y' \wedge \neg z') \\
 &\quad \vee (x \wedge \neg y \wedge \neg z \wedge \neg x' \wedge \neg y' \wedge \neg z') \vee (x \wedge y \wedge \neg z \wedge \neg x' \wedge \neg y' \wedge \neg z') \\
 &\quad) \wedge (\neg z' \vee (\neg x' \wedge y' \wedge z'))) \\
 &= (\neg z \vee (\neg x \wedge y \wedge z)) \wedge \neg \exists x' \exists y' (((\neg x \wedge \neg y \wedge z \wedge \neg x' \wedge y') \vee (\neg x \wedge y \wedge x' \wedge y') \\
 &\quad \vee (x \wedge \neg y \wedge z \wedge \neg x' \wedge \neg y') \vee (x \wedge y \wedge z \wedge \neg x' \wedge \neg y')))) \\
 &= (\neg z \vee (\neg x \wedge y \wedge z)) \wedge \neg ((\neg x \wedge \neg y \wedge z) \vee (\neg x \wedge y) \vee (x \wedge \neg y \wedge z) \vee (x \wedge y \wedge z))
 \end{aligned}$$

Symbolic simulation-checking

- an example

$$\begin{aligned} B_1 &= (\neg z \vee (\neg x \wedge y \wedge z)) \wedge \neg((\neg x \wedge \neg y \wedge z) \vee (\neg x \wedge y) \vee (x \wedge \neg y \wedge z) \vee (x \wedge y \wedge z)) \\ &= (\neg z \vee (\neg x \wedge y \wedge z)) \wedge \neg((\neg x \wedge \neg y \wedge z) \vee (\neg x \wedge y) \vee (x \wedge \neg y \wedge z) \vee (x \wedge y \wedge z)) \\ &= (\neg z \vee (\neg x \wedge y \wedge z)) \wedge \neg(z \vee (\neg x \wedge y \wedge \neg z)) \\ &= (\neg z \vee (\neg x \wedge y \wedge z)) \wedge \neg(z) \wedge \neg(\neg x \wedge y \wedge \neg z) \\ &= (\neg z \vee (\neg x \wedge y \wedge z)) \wedge \neg(z) \wedge \neg(\neg x \wedge y \wedge \neg z) \\ &= (\neg x \wedge \neg y \wedge \neg z) \vee (x \wedge \neg y \wedge \neg z) \vee (x \wedge y \wedge \neg z) \end{aligned}$$

Symbolic simulation-checking

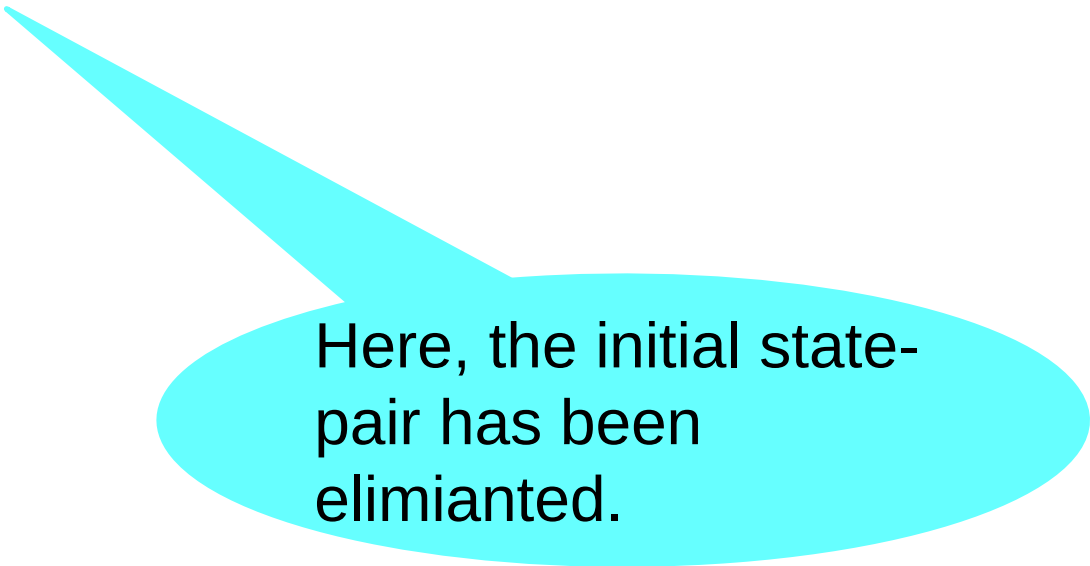
- an example

$$\begin{aligned}
 B_2 &= ((\neg x \wedge \neg y \wedge \neg z) \vee (x \wedge \neg y \wedge \neg z) \vee (x \wedge y \wedge \neg z)) \wedge \neg \exists x' \exists y' (\\
 &\quad ((\neg x \wedge \neg y \wedge \neg x' \wedge y') \vee (\neg x \wedge y \wedge x' \wedge \neg y') \\
 &\quad \vee (\neg x \wedge y \wedge x' \wedge y') \vee (x \wedge \neg y \wedge \neg x' \wedge \neg y') \vee (x \wedge y \wedge \neg x' \wedge \neg y') \\
 &\quad) \\
 &\quad \wedge \neg \exists x' \exists y' \exists z' (\\
 &\quad ((\neg x \wedge \neg y \wedge \neg z \wedge \neg x' \wedge y') \\
 &\quad \vee (\neg x \wedge y \wedge \neg z \wedge x' \wedge \neg y' \wedge \neg z') \vee (\neg x \wedge y \wedge z \wedge x' \wedge y' \wedge \neg z') \\
 &\quad \vee (x \wedge \neg y \wedge \neg z \wedge \neg x' \wedge \neg y' \wedge \neg z') \vee (x \wedge y \wedge \neg z \wedge \neg x' \wedge \neg y' \wedge \neg z') \\
 &\quad) \wedge ((\neg x' \wedge \neg y' \wedge \neg z') \vee (x' \wedge \neg y' \wedge \neg z') \vee (x' \wedge y' \wedge \neg z')))) \\
 &= ((\neg x \wedge \neg y \wedge \neg z) \vee (x \wedge \neg y \wedge \neg z) \vee (x \wedge y \wedge \neg z)) \wedge \neg \exists x' \exists y' (\\
 &\quad ((\neg x \wedge \neg y \wedge \neg x' \wedge y') \vee (x \wedge \neg y \wedge z \wedge \neg x' \wedge \neg y') \vee (x \wedge y \wedge z \wedge \neg x' \wedge \neg y'))) \\
 &= ((\neg x \wedge \neg y \wedge \neg z) \vee (x \wedge \neg y \wedge \neg z) \vee (x \wedge y \wedge \neg z)) \wedge \neg ((\neg x \wedge \neg y) \vee (x \wedge \neg y \wedge z) \vee (x \wedge y \wedge z))
 \end{aligned}$$

Symbolic simulation-checking

- an example

$$\begin{aligned} B_2 &= ((\neg x \wedge \neg y \wedge \neg z) \vee (x \wedge \neg y \wedge \neg z) \vee (x \wedge y \wedge \neg z)) \wedge \neg((\neg x \wedge \neg y) \vee (x \wedge \neg y \wedge z) \vee (x \wedge y \wedge z)) \\ &= (x \wedge \neg y \wedge \neg z) \vee (x \wedge y \wedge \neg z) \end{aligned}$$



Here, the initial state-pair has been eliminated.

Tool & library available

- REDLIB version 3 at Sourceforge

 - <http://sourceforge.net/projects/redlib/>

 - ❑ GUI for timed automata editor & simulator
 - ❑ Timed automata model/simulation-checking
 - ❑ Pre/post-condition calculation
 - ❑ Dense-space manipulation

- RED version 8

 - ❑ a timed automata model/simulation-checker
 - ❑ an LHA parametric analyzer
 - ❑ built on top of REDLIB.